MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Tran Cong An, my thesis advisor, for his guidance, continuous feedback, and support throughout this research.

I also thank the members of the Thesis Defense Committee for their time, thorough evaluation, and constructive comments.

My sincere appreciation goes to the Faculty of Information Technology, Can Tho University, for providing the academic foundation that supported this work, and to the High-Quality Program for fostering the engineering discipline applied throughout this thesis.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and unwavering support throughout my studies and the completion of this work.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

EVALUATION OF ADVISOR ..... **III**
ACKNOWLEDGEMENTS ..... **IV**
TABLE OF CONTENTS ..... **V**
LIST OF FIGURES ..... **VIII**
LIST OF TABLES ..... **X**
LIST OF ABBREVIATIONS ..... **XIII**
ABSTRACT ..... **XIV**
TÓM TẮT ..... **XV**
PART 1: INTRODUCTION **2**
I. Context and Motivation 2
II. Problem Statement 2
III. Objectives 3
IV. Scope and Limitations 5
V. Research Methodology 6
VI. Thesis Outline 6
PART 2: THESIS CONTENT **8**
CHAPTER 1. BACKGROUND AND RELATED WORK **8**
1.1 Fashion E-Commerce 8
1.2 Content-Based Image Retrieval 8
1.2.1 Visual Search Concepts 9
1.2.2 The Semantic Gap 9
1.2.3 Mathematical Foundations of Embeddings 9
1.3 Machine Learning Models 10
1.3.1 Convolutional Neural Networks 11
1.3.2 Vision Transformers 14
1.3.3 CLIP and Fashion-CLIP 16
1.3.4 Model Selection and Justification 20
1.4 Vector Databases 22
1.4.1 The Search Challenge and Approximate Nearest Neighbours 22
1.4.2 HNSW: Hierarchical Navigable Small World 23
1.4.3 IVFFlat: Inverted File with Flat Compression 23
1.4.4 pgvector: Vector Search in PostgreSQL 24
1.4.5 Architectural Decision and Trade-offs 25
1.5 Platform Architecture and Technology Stack 26
1.5.1 Architectural Patterns 26
1.5.2 .NET Backend 27

1.5.3 Vue.js Frontend 28
1.5.4 PostgreSQL and pgvector 29
1.5.5 Redis Caching 29
1.5.6 Python ML Sidecar 29
1.5.7 .NET Aspire Orchestration 30
1.5.8 Hangfire Background Jobs 30
1.5.9 Identity, Authentication, and Authorisation 31
1.5.10 Benchmark Framework 31
1.5.11 Technology Stack Summary 32
1.6 Related Work and Research Gap 33
1.6.1 Academic Research 33
1.6.2 Commercial Systems 33
1.6.3 Contribution Differentiators 34
CHAPTER 2. DESIGN AND IMPLEMENTATION 35
2.1 Requirements Analysis 35
2.1.1 System Actors 35
2.1.2 Functional Requirements 36
2.1.3 Non-Functional Requirements 45
2.1.4 Feature Classification 47
2.2 Functional Decomposition and Use Cases 49
2.2.1 Functional Decomposition 49
2.2.2 Use Case Overview 50
2.2.3 Use Case Summary Matrix 52
2.2.4 Administrator Use Cases 54
2.2.5 Customer Use Cases 70
2.2.6 System Use Cases 81
2.3 System Architecture & Design 84
2.3.1 System Overview 84
2.3.2 Domain-Driven Design 86
2.3.3 C4 Architecture 96
2.3.4 Database Design 101
2.3.5 API Design 106
2.3.6 Security Design 113
2.3.7 Summary 116
2.4 Implementation 116
2.4.1 Vertical Slice Architecture 116
2.4.2 ML Embedding Pipeline 117
2.4.3 CBIR Search Flow 120
2.4.4 Model Configuration 122
2.4.5 E-commerce Core 123
CHAPTER 3. TESTING AND EVALUATION 126
3.1 Testing Strategy 126
3.1.1 Unit Testing 126

3.1.2 Integration Testing 126
3.1.3 End-to-End Testing 127
3.2 Benchmark Protocol 127
3.2.1 Dataset 127
3.2.2 Models Evaluated 128
3.2.3 Metrics 129
3.2.4 Methodology 130
3.2.5 Hardware Environment 131
3.3 Retrieval Performance 131
3.4 Efficiency Metrics 133
3.5 Model Comparison & Discussion 135
3.5.1 Accuracy-Efficiency Trade-off 135
3.5.2 Deployment Recommendations 136
3.5.3 Discussion of Limitations 137
3.5.4 Lessons Learned 138
PART 3: CONCLUSION AND FUTURE WORK 140
CHAPTER 5. CONCLUSION & FUTURE WORK 140
5.1 Summary of Work 140
5.1.1 Answering the Research Questions 141
5.1.2 Achievement of Technical Objectives 142
5.2 Contributions 143
5.3 Limitations 144
5.4 Future Work 145
5.5 Requirements Traceability 147
REFERENCES 150
APPENDIX A. APPENDIX A, FULL BENCHMARK RESULTS 153
A.1 A.1 Category-Only Ground Truth (5 Categories) 153
A.2 A.2 Category + Colour Ground Truth 153
A.3 A.3 Category + Colour + Pattern Ground Truth 154
A.4 A.4 Operational Efficiency (3-Fold CV) 155
A.5 A.5 Per-Fold Variability 156
APPENDIX B. APPENDIX B, DATASET COMPOSITION 156
B.1 B.1 Source and Selection 156
B.2 B.2 Category Distribution 157
B.3 B.3 Ground-Truth Label Schemes 157
B.4 B.4 Image Preprocessing 158
APPENDIX C. APPENDIX C, HARDWARE SPECIFICATIONS 158
C.1 C.1 Compute Hardware 158
C.2 C.2 Software Stack 159
C.3 C.3 Precision and Determinism Settings 159
C.4 C.4 Reproducibility Notes 160

LIST OF FIGURES

Figure 1 ResNet architecture with residual skip connections enabling gradient flow across very deep networks 12
Figure 2 EfficientNet-B0 architecture showing the flow from input image to feature vector 13
Figure 3 DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector 15
Figure 4 CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison 17
Figure 5 Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space 19
Figure 6 Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions. 50
Figure 7 System use case diagram showing all 26 use cases from the summary matrix, grouped by business module with actor-to-module associations. 51
Figure 8 Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace. 87
Figure 9 Order checkout state machine: five sequential states with cancellation available from any pre-confirmation state. The forward-only constraint prevents regressing to earlier checkout stages. 94
Figure 10 Payment intent lifecycle: the state machine reflects Stripe gateway states while maintaining a parallel system copy for offline operations and Bogus gateway compatibility. Terminal states are Failed, Canceled, and Refunded. 95
Figure 11 System Context diagram: ReSys.Shop as a single system boundary with customer and administrator users on one side and external payment, email, storage, identity, and ML services on the other. The modular monolith internally handles all eight business domains within one boundary. 97
Figure 12 Container diagram showing the deployable units of ReSys.Shop: two Vue 3 SPAs, the .NET 10 API backend, the Python ML sidecar, PostgreSQL with pgvector, and Redis. Arrows indicate communication protocols between containers. 98

Figure 13 Component diagram of the API Backend showing the Carter endpoint layer, the MediatR pipeline, the feature handlers, and eight supporting infrastructure components. The Python ML Sidecar is shown with its internal three-layer architecture alongside. 99
Figure 14 Deployment diagram showing containerised services within an Aspire orchestration boundary. The API backend is horizontally scalable; the embedding sidecar is stateless; Redis enables distributed state across API replicas. 101
Figure 15 Core entity-relationship model across all eight bounded contexts. 103
Figure 16 ML embedding pipeline: the step-by-step flow from image bytes received by the FastAPI interface to a 512-dimensional float vector returned as JSON to the .NET backend. 119
Figure 17 CBIR search sequence: the complete end-to-end flow from customer image upload through embedding generation and vector search to ranked results display, spanning the Vue frontend, .NET API, Python ML sidecar, and PostgreSQL pgvector. 121

LIST OF TABLES

Table 1 What different CNN layers learn to detect in fashion images 11
Table 2 CNN-based models evaluated 13
Table 3 DINOv2 model specifications evaluated in this project 16
Table 4 CLIP variants evaluated 19
Table 5 Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available. 20
Table 6 Comparison of pgvector with specialised vector databases 25
Table 7 Technology stack of the ReSys.Shop platform 32
Table 8 Comparison of commercial visual search products 33
Table 9 Catalog module functional requirements. 37
Table 10 Identity module functional requirements. 39
Table 11 Inventory module functional requirements. 40
Table 12 Ordering module functional requirements. 41
Table 13 Payment module functional requirements. 43
Table 14 Shipping module functional requirements. 43
Table 15 Profile module functional requirements. 44
Table 16 Location module functional requirements. 45
Table 17 Dashboard module functional requirements. 45
Table 18 Performance: latency targets for CBIR search and general API responses 45
Table 19 Security: authentication, authorisation, rate limiting, upload validation, and transport controls 46
Table 20 Modularity: module isolation, communication boundary, and testability 46
Table 21 Observability: distributed tracing, correlation logging, and health monitoring 46
Table 22 Reliability: job durability, cart expiry, idempotency, and timeout guarantees 47
Table 23 Classification of feature areas into Core Research and Supporting Infrastructure. Core Research features represent the thesis's original contributions; Supporting Infrastructure features provide the realistic e-commerce context necessary for meaningful evaluation. 47
Table 24 Use case summary matrix. All use cases are listed with their actor, associated business module, and traceability to functional requirements defined in Section 2.1. 52
Table 25 Manage Products. 54
Table 26 Manage Variants. 55
Table 27 Manage Images and Embeddings. 56
Table 28 Manage Taxonomies and Classification. 57
Table 29 Manage Option Types. 58

Table 30 Manage Orders. 59
Table 31 Manage Order Details. 61
Table 32 Manage Payments. 61
Table 33 Manage Payment Methods. 63
Table 34 Manage Stock Locations. 63
Table 35 Manage Stock. 64
Table 36 Manage Users. 66
Table 37 Manage Roles and Permissions. 67
Table 38 Manage Shipping. 68
Table 39 Manage Reference Data. 69
Table 40 Browse and Search Catalog. 70
Table 41 Visual Search. 71
Table 42 Manage Cart. 73
Table 43 Checkout. 74
Table 44 Order History. 75
Table 45 Payment Processing. 76
Table 46 Authentication. 77
Table 47 Session Management. 79
Table 48 Profile Management. 79
Table 49 Embedding Operations. 81
Table 50 System Maintenance. 82
Table 51 System services and their technology stacks. Each service is independently deployable and communicates through well-defined HTTP contracts. 84
Table 52 Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates with other contexts through MediatR dispatch only. 85
Table 53 Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace. 87
Table 54 Ubiquitous Language Glossary: core domain terms, their owning bounded context, and their precise definitions as used throughout the codebase and this thesis. 92
Table 55 Key API endpoints representing the primary user-facing capabilities of the platform. All endpoints in the Storefront surface serve customer interactions; Admin surface endpoints (not shown) mirror these with full CRUD capabilities on all modules. 107
Table 56 Embedding models supported by the benchmark framework, grouped by architecture family. Four representative models were evaluated in the thesis. All models use pre-trained weights from public model repositories. 128
Table 57 Hardware environment for benchmark evaluation. 131

Table 58 Aggregate Retrieval Metrics, 3-Fold Cross-Validation 131
Table 59 Efficiency Metrics, 3-Fold Cross-Validation 133
Table 60 Combined Accuracy and Efficiency Comparison 135
Table 61 Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met. 147
Table 62 Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD) 153
Table 63 Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD) 154
Table 64 Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD) 154
Table 65 Operational Efficiency, All Models (3-Fold CV, Mean ± SD) . . . 155
Table 66 Per-Fold Breakdown, Category + Colour Ground Truth 156
Table 67 Dataset Category Distribution 157
Table 68 Benchmark Workstation Configuration 158
Table 69 Benchmark Software Environment 159

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation.

A systematic benchmark evaluates four pre-trained deep learning models across both retrieval accuracy and operational efficiency on a curated fashion product dataset. Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, achieved the highest mean Average Precision at 0.8788 (SD 0.0022), outperforming the generic CLIP (mAP 0.8341, SD 0.0043) by 5.4%, EfficientNet-B0 (mAP 0.8158, SD 0.0007) by 7.7%, and ResNet-50 (mAP 0.8120, SD 0.0052) by 8.2%. At the opposite end of the speed spectrum, EfficientNet-B0 delivered inference at 23.9 ms per image (SD 2.5) while maintaining competitive retrieval quality, representing a 3.8x speed advantage over Fashion-CLIP (92.0 ms, SD 5.8) for a 7.7% accuracy trade-off.

Results demonstrate that domain-specific pre-training provides measurable advantages for visual fashion retrieval. The pluggable model architecture, switchable via a single environment variable, allows production deployments to select the optimal accuracy-speed profile for their infrastructure. The thesis provides a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt chính xác qua từ khóa. Hệ thống được xây dựng theo hướng module, bao gồm ba thành phần chính: backend .NET xử lý logic nghiệp vụ, frontend Vue.js phục vụ giao diện người dùng, và dịch vụ Python chuyên biệt cho tác vụ trích xuất đặc trưng hình ảnh từ các mô hình học sâu tiền huấn luyện.

Thực nghiệm được thiết lập trên bộ dữ liệu ảnh sản phẩm thời trang với quy trình đánh giá chéo ba lần (3-fold cross-validation), so sánh bốn mô hình embedding trên hai chiều: chất lượng truy xuất (đo bằng mAP, Precision at K, Recall at K) và hiệu năng vận hành (đo bằng độ trễ suy luận, thông lượng, và dung lượng lưu trữ). Mô hình Fashion-CLIP, được tinh chỉnh trên tập dữ liệu hơn 700.000 ảnh thời trang, đạt mAP cao nhất 0,8788 với độ lệch chuẩn 0,0022. CLIP gốc đạt mAP 0,8341 (SD 0,0043) và EfficientNet-B0 đạt 0,8158 (SD 0,0007). Ở chiều ngược lại, mô hình EfficientNet-B0 cho tốc độ suy luận nhanh nhất, 23,9 ms mỗi ảnh (SD 2,5), nhanh gấp 3,8 lần Fashion-CLIP (92,0 ms, SD 5,8) trong khi chỉ đánh đổi 7,7% về chất lượng truy xuất.

Kết quả thực nghiệm khẳng định giá trị của huấn luyện chuyên biệt theo lĩnh vực trong bài toán truy xuất hình ảnh thời trang, đồng thời cung cấp dữ liệu định lượng cho việc lựa chọn mô hình dựa trên ràng buộc hạ tầng thực tế. Kiến trúc mô hình hoán đổi linh hoạt, được điều khiển qua biến môi trường, cho phép hệ thống thích ứng với nhiều cấu hình triển khai khác nhau mà không cần thay đổi mã nguồn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

Global fashion e-commerce revenue exceeded **770 billion USD** in 2024, with projections surpassing **one trillion by 2030** [1]. Yet its dominant interface, keyword search, fails where the domain succeeds: fashion products are defined by silhouette, drape, print density, and colour relationships – attributes that resist textual description. This **semantic gap**, the discrepancy between a garment's visual richness and a user's ability to express it in words, is well documented. A customer can recognise a desired aesthetic instantly from a photograph yet cannot produce query terms that retrieve it. When catalogue indexing uses inconsistent terminology (one vendor tags a pattern as "floral," another as "botanical," a third as "flower print"), relevant products are systematically excluded. Industry estimates place the **session abandonment rate** after an unsuccessful search at approximately **30 percent** [2].

Content-Based Image Retrieval (CBIR) addresses this gap by replacing textual intermediaries with direct visual comparison. Products are indexed not by human-authored labels but by **dense vector embeddings** computed from images, with similarity measured through mathematical distance functions. A query image of a dress with a particular neckline and print pattern retrieves visually similar products without any keyword translation step. Pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models [6] have substantially advanced this capability.

The contribution of this work is **architectural**, not algorithmic. It investigates how to embed existing CBIR capabilities into a practical e-commerce system built with conventional web technologies, and provides empirical data on which embedding models deliver the optimal balance of accuracy, latency, and resource efficiency. The work bridges two distinct software ecosystems, the **Python machine learning stack** and the **.NET enterprise web stack**, under real-time latency constraints appropriate for interactive search.

II. PROBLEM STATEMENT

Keyword-reliant fashion search suffers from four compounding inefficiencies.

Catalogue vocabulary mismatch. Descriptors vary across vendors, such that a single visual pattern appears under multiple labels and fragments result sets. Users must reformulate queries iteratively as the gap between catalogue

indexing vocabulary and customer search vocabulary silently excludes relevant products.

Visual inexpressibility. Attributes such as fabric drape, texture gradient, silhouette proportion, and pattern rhythm cannot be captured reliably through text queries. A customer identifies a desired aesthetic instantly in a photograph yet cannot produce keywords that retrieve it. The search engine cannot match what the user cannot name.

Cold-start invisibility. Recommendation models based on collaborative filtering depend on historical user-item interactions. Newly listed products lack this data at their point of highest commercial value: initial release. Visual feature extraction bypasses this limitation, as product images are available from catalogue ingestion and embeddings enable similarity-based discovery without interaction history.

Polyglot integration cost. Integrating pre-trained vision models into a .NET transactional backend introduces a recurring engineering challenge in applied machine learning. The Python deep learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET enterprise stack. Achieving **sub-second response latency** across this boundary requires architectural design that isolates the ML workload from the main application while bridging incompatible package managers, runtime environments, and deployment conventions.

III. OBJECTIVES

This project builds a functional fashion e-commerce platform with integrated image-based search and evaluates the effectiveness of pre-trained deep learning models within that system. The contribution is not a novel AI architecture but the **engineering demonstration** of embedding existing models into a conventional web application stack.

Technical Objectives

- **Model integration.** Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack, establishing a reference pattern for teams with existing web infrastructure.
- **Polyglot architecture.** Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- **Vector storage validation.** Validate **pgvector** (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, evaluating whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.

- **Empirical benchmarking.** Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, producing empirical guidance for model selection in resource-constrained deployments.

Research Questions

Three questions guide the investigation and are answered empirically in Chapter 3.

RQ1: Model comparison. How do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval? This question tests whether domain-specific training on fashion data yields measurable improvements over models trained on generic image corpora.

RQ2: Accuracy-speed trade-off. What trade-offs exist between retrieval accuracy and inference latency across pre-trained embedding models, and which model offers the best balance for real-time search? The most accurate model is rarely the fastest; deployment decisions require weighing both dimensions.

RQ3: Architecture viability. Can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times? This question evaluates whether the chosen polyglot pattern (Python ML service alongside a .NET application) is practical for production use.

Tasks Completed

- **Build an AI service.** A Python FastAPI service that loads multiple pre-trained embedding models and generates feature vectors from product images within interactive latency bounds.
- **Set up vector search.** PostgreSQL configured with pgvector to store and index high-dimensional embeddings, with similarity queries validated for correctness and performance on a catalogue of representative size.
- **Connect the services.** A .NET backend layer that orchestrates image upload, embedding generation, vector database query, and result assembly into a single end-to-end search flow.
- **Create the user interface.** A Vue.js storefront with drag-and-drop image upload and a results grid displaying visually similar products with similarity scores.
- **Evaluate the results.** A systematic benchmark measuring retrieval accuracy via standard information retrieval metrics, comparing inference speed across models, and analyzing operational trade-offs that inform production model selection.

IV. SCOPE AND LIMITATIONS

The thesis encompasses the design, implementation, and empirical evaluation of a fashion e-commerce platform with integrated visual search. Five areas define the included scope:

- Visual search from the storefront, with image upload via drag-and-drop or file selection.
- Product recommendations derived from embedding similarity scores.
- Core e-commerce functionality: product catalogue browsing, shopping cart, and simulated checkout.
- Multi-model comparison spanning several CNN and transformer architectures.
- End-to-end latency, throughput, and storage footprint measurement.

The following areas are explicitly excluded:

- Real payment processing (transactions are simulated for demonstration purposes).
- Shipping, logistics, and warehouse management workflows.
- User authentication via social login providers.
- Mobile application development (the system targets desktop and tablet web browsers).
- Custom model training or fine-tuning (all models are used as published).

Known Limitations

Four limitations define the boundaries of this work and are revisited in the concluding chapter.

- **Dataset size.** Evaluation uses 5,000 fashion product images from the Fashion Product Images dataset [7]. Controlled comparative benchmarking is feasible at this scale, but results may not extrapolate to production catalogues containing millions of items.
- **Hardware.** Experiments ran on consumer-grade hardware (Intel i7-1165G7, 16 GB RAM) with all inference executed on CPU. Reported latency and throughput figures are relative to this CPU-only profile. A dedicated inference server with GPU acceleration would likely improve both metrics.
- **Evaluation method.** Evaluation is exclusively quantitative: retrieval accuracy, inference latency, and throughput. Search output was reviewed qualitatively through visual inspection, but no formal user experience study was conducted. The relationship between measured accuracy metrics and subjective user satisfaction remains an open question.
- **Model training.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific

fine-tuning, particularly for models pre-trained on generic image corpora rather than fashion data, might improve retrieval quality but was beyond scope.

V. RESEARCH METHODOLOGY

This thesis follows a **Design Science Research (DSR)** methodology [8], [9], a problem-solving paradigm that produces and evaluates an IT artifact (here, the e-commerce platform with integrated visual search) to address a defined problem domain.

The project progressed through four phases:

- **Research and planning.** Literature survey on visual search in fashion and e-commerce architectures; exploration of available pre-trained embedding models; evaluation of vector database options; technology stack selection.
- **Design.** Formalization of a modular .NET monolith architecture with a Python AI sidecar communicating via HTTP; database schema design supporting both relational and vector data within a single PostgreSQL instance.
- **Implementation.** Construction of three principal components: the .NET backend following vertical slice architecture, the Python embedding service with lazy model loading, and the Vue.js storefront.
- **Test and evaluation.** Systematic benchmark measuring retrieval accuracy through standard information retrieval metrics; latency and throughput measurement on consumer-grade hardware; qualitative review of search output.

This methodology suits the project's engineering focus: the primary output is not a theoretical contribution but a working system accompanied by empirical data on performance trade-offs practitioners encounter when integrating academic models into production web stacks.

VI. THESIS OUTLINE

The thesis is organized into five chapters across three parts.

Part I: Introduction. Chapter 1 establishes research context, defines the problem, states objectives and research questions, delineates scope and limitations, describes the methodology, and provides the present outline.

Part II: Thesis Content contains three chapters:

- **Chapter 1: Background and Related Work.** Surveys vector embeddings, convolutional and transformer-based neural architectures, vector database technologies including pgvector, prior work in fashion image retrieval, and the technology stack selected for this project.
- **Chapter 2: Design and Implementation.** Translates the problem into functional and non-functional requirements (Section 2.1), presents the system

architecture including domain-driven design, C4 diagrams, database design, API design, and security model (Section 2.2), and describes the concrete implementation of the .NET backend, Python ML sidecar, and Vue.js storefront (Section 2.3).

- **Chapter 3: Testing and Evaluation.** Reports a systematic benchmark comparing retrieval accuracy and inference efficiency across multiple embedding models using a cross-validation protocol on 5,000 fashion product images, and analyzes the accuracy-speed trade-offs that inform model selection.

Part III: Conclusion. Chapter 4 synthesizes findings against each research objective, evaluates contributions and limitations, and proposes directions for future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter surveys the theoretical foundations and technical prerequisites for the project.

- **Fashion E-commerce.** Market context, semantic gap, business case.
- **Content-Based Image Retrieval.** Embeddings, cosine similarity, CBIR pipeline.
- **Machine Learning Models.** CNN, ViT, CLIP, Fashion-CLIP architectures.
- **Vector Databases.** ANN search, HNSW, IVFFlat, pgvector.
- **Platform Architecture and Technology Stack.** Modular monolith, vertical slice, .NET, Vue, PostgreSQL, Redis, Python sidecar, orchestration, auth, benchmarks.
- **Related Work and Research Gap.** Academic research, commercial systems, contribution differentiators.

1.1 FASHION E-COMMERCE

Global fashion e-commerce reached **770 billion USD** in 2024 and is projected to surpass **one trillion USD by 2030** [1]. However, traditional text search bars cause a **30 percent search abandonment rate** because keyword queries struggle to match visual intent [2].

Fashion is fundamentally visual. Attributes like silhouette, drape, color harmony, and pattern density do not translate easily into search terms. A customer can easily recognize a garment in an image but often fails to find it using keywords.

Major platforms like ASOS, Zalando, and Shopee invest in visual search because millions of catalog items rely on inconsistent vendor metadata. Identical patterns often carry different labels across suppliers, hiding relevant products from keyword search. Visual search solves this metadata problem, directly preventing lost revenue from search abandonment.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical

distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Rather than requiring users to describe what they want in words, CBIR lets them search by example.

The system encodes a query image into a **dense vector embedding**: a fixed-length sequence of numbers that captures shape, texture, colour, and pattern. It then retrieves catalog items whose embeddings are nearest in vector space. Visually similar products produce similar vectors; dissimilar products produce distant ones.

This approach bypasses the need for consistent textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them.

1.2.2 The Semantic Gap

The semantic gap, introduced in Section 1.1, is the discrepancy between the visual richness of a garment and a user's ability to express that richness in keywords. CBIR bridges this gap by operating directly on visual content rather than on human-authored metadata. The embedding serves as a universal descriptor that captures attributes such as fabric texture, silhouette proportion, and colour gradients automatically, without any keyword translation step.

1.2.3 Mathematical Foundations of Embeddings

An embedding model processes an input image and transforms its visual content into a fixed-length numerical array:

$$\boldsymbol{e} = [e_1, e_2, e_3, ..., e_d]^T \in \mathbb{R}^d$$

For a model with a 512-dimensional output, this vector $\boldsymbol{e}$ contains 512 continuous numbers. These arrays are called **vector embeddings**. Visually similar items yield nearby numerical values across these dimensions, converting visual comparison into a standard mathematical distance calculation.

1.2.3.1 The Latent Space

Embeddings act as coordinates within a high-dimensional continuous space known as the **latent space**. A 512-dimensional vector occupies a coordinate system with 512 orthogonal axes.

Within this space:

- Visually and semantically similar items sit close to one another.
- Dissimilar items lie further apart.
- The term "latent" indicates that individual axes rarely map directly to single human visual labels like "stripes" or "red." Instead, each dimension captures abstract, non-linear feature combinations learned by the model during training.

1.2.3.2 Measuring Similarity: Cosine Similarity

To evaluate how closely two images match, the system measures the directional angle between their corresponding embedding vectors $\boldsymbol{A}$ and $\boldsymbol{B}$ using **cosine similarity**:

$$\text{Cosine Similarity}(\boldsymbol{A}, \boldsymbol{B}) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\|_2 \times \|\boldsymbol{B}\|_2}$$

Where $\boldsymbol{A} \cdot \boldsymbol{B}$ represents the vector dot product, and $\|\boldsymbol{A}\|_2$ and $\|\boldsymbol{B}\|_2$ denote the Euclidean norms (L_2 norms) of each vector.

Cosine similarity produces values ranging from $+1.0$ (identical vector orientation) to 0.0 (orthogonal vectors) down to -1.0 (opposite orientations). For normalized fashion embeddings, scores above 0.70 generally correspond to strong visual similarity perceptible to human shoppers.

1.2.3.3 The CBIR Pipeline

A standard Content-Based Image Retrieval system operates through four core sequential stages:

1. **Image Input:** The user uploads or selects a target query photograph.
2. **Embedding Generation:** A deep learning model processes the image to output its feature vector.
3. **Vector Comparison:** The database compares the query vector against pre-indexed catalog vectors using cosine distance or cosine similarity.
4. **Ranking & Retrieval:** The system orders products by their similarity scores and renders the top nearest neighbors to the user.

1.3 MACHINE LEARNING MODELS

This section surveys the three families of architectures used for visual feature extraction: convolutional neural networks, vision transformers, and multimodal CLIP-based models. Each subsection explains how the architecture works, introduces the specific variants evaluated, and identifies their trade-offs for fashion retrieval. The section concludes with the model selection decision.

1.3.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) have been the dominant architecture for computer vision. This section explains how CNNs extract features and introduces the specific variants evaluated: ResNet and EfficientNet.

1.3.1.1 Hierarchical Feature Extraction

A CNN processes an image through a stack of learned filters. Each filter is a small window (typically 3 by 3 pixels) that slides across the image detecting local patterns [3]. This process builds increasingly complex representations:

Table 1: What different CNN layers learn to detect in fashion images

Layer	What It Detects
Early layers	Simple patterns: edges, colours, corners
Middle layers	Combinations: textures, shapes, parts (lapels, sleeves)
Late layers	High-level features: garment categories, styles

Early layers might detect the edge of a sleeve, middle layers recognise a striped pattern, and late layers understand "a formal button-down shirt." This hierarchical organisation gives CNNs an **inductive bias** toward local patterns: they excel at detecting texture and colour but may miss relationships between distant image regions.

1.3.1.2 ResNet and Skip Connections

Deeper networks capture richer features but suffer from **vanishing gradients**: training signals decay as they propagate backward through many layers. ResNet (Residual Network) solves this with **skip connections**: identity paths that bypass convolutional blocks and add the block input directly to its output [3]. This identity path lets gradients flow unimpeded, enabling networks of 50, 101, or 152 layers to train effectively.

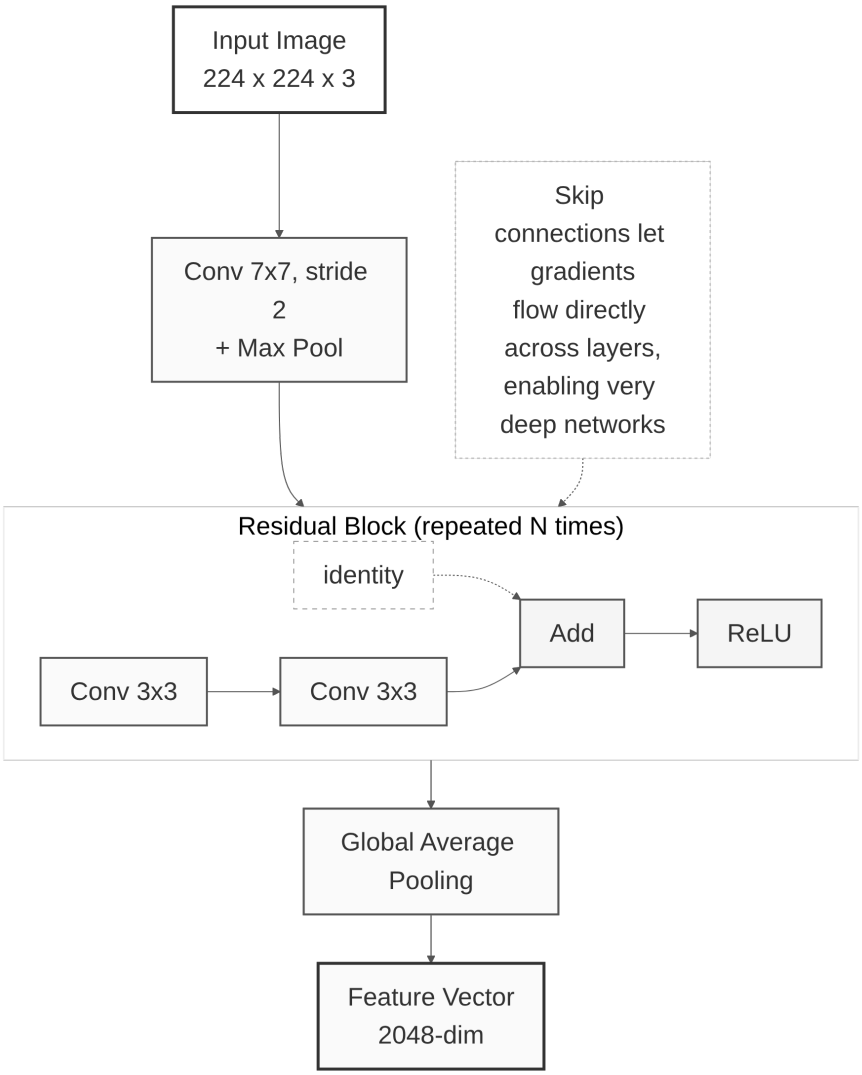

Figure 1: ResNet architecture with residual skip connections enabling gradient flow across very deep networks

ResNet-50, with 25.6 million parameters and 2,048-dimensional embeddings, remains a strong baseline for image retrieval. ResNet-101 (44.5M parameters) provides additional depth for comparison.

1.3.1.3 EfficientNet and Compound Scaling

Traditional scaling strategies enlarge a network along a single dimension: depth (more layers), width (more channels), or resolution (larger inputs). EfficientNet introduces **compound scaling**, which balances all three dimensions simultaneously using a learned coefficient [4]. This produces a family of models (B0 through B7) that achieve competitive accuracy with far fewer parameters than conventional scaling.

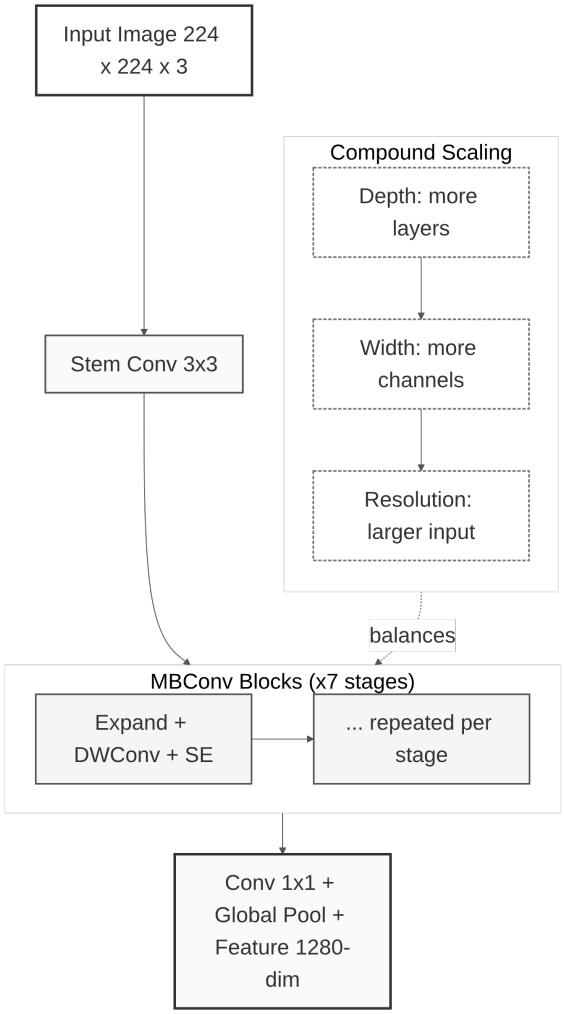

Figure 2: EfficientNet-B0 architecture showing the flow from input image to feature vector

EfficientNet-B0, the smallest variant, uses 5.3 million parameters and produces 1,280-dimensional embeddings. Its compact design makes it well suited to CPU-only deployments. EfficientNet-B4 (19.3M parameters, 1,792-dimensional embeddings) provides higher capacity at increased computational cost.

1.3.1.4 Evaluated CNN Variants

Table 2: CNN-based models evaluated

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

While CNNs excel at detecting local patterns through their convolutional layers, they have limitations in capturing long-range dependencies and global context. This motivated researchers to explore alternative architectures: vision transformers.

1.3.2 Vision Transformers

Vision Transformers (ViTs) apply the transformer architecture from natural language processing to images. This section explains how ViTs work and introduces DINOv2, a self-supervised model evaluated in this project.

1.3.2.1 Patch Embedding and Tokenization

Transformers were originally developed for NLP tasks such as translation and text generation. The key innovation was **self-attention**, which allows the model to consider relationships between all parts of the input simultaneously.

In 2020, researchers showed this approach could also work for images [10]. The idea is straightforward:

- Split the image into a grid of fixed-size patches (e.g., 14 by 14 or 16 by 16 pixels).
- Flatten each patch into a vector and treat it as a token, analogous to a word in a sentence.
- Add position encodings to preserve spatial information.
- Pass the sequence through transformer layers with multi-head self-attention.

1.3.2.2 Global Context via Self-Attention

Unlike CNNs, which focus on local patterns, self-attention captures relationships across the entire image from the first layer. A CNN processes patches in order and mainly compares nearby regions. A ViT can directly compare any two patches, even if they are far apart.

For fashion, this means a ViT can better understand that the collar of a shirt and the cuffs should match, even though they are on opposite sides of the image. This capability is valuable for garment retrieval, where silhouette and drape matter as much as local texture.

1.3.2.3 DINOv2 and Self-Supervised Pre-Training

Most AI models are trained with supervised learning, where humans label images (“this is a dress,” “this is a shoe”), and the model learns from those labels. This requires expensive manual annotation.

Self-supervised learning takes a different approach: the model learns from the images themselves, without human labels. DINOv2 uses a student-teacher self-distillation framework [11]:

- Take an image and create two different views (different crops, slightly different colours).
- Pass them through student and teacher networks with identical architecture.
- Train the student to match the teacher's output.
- Update the teacher as an exponential moving average of the student weights.
- Repeat across 142 million uncurated images.

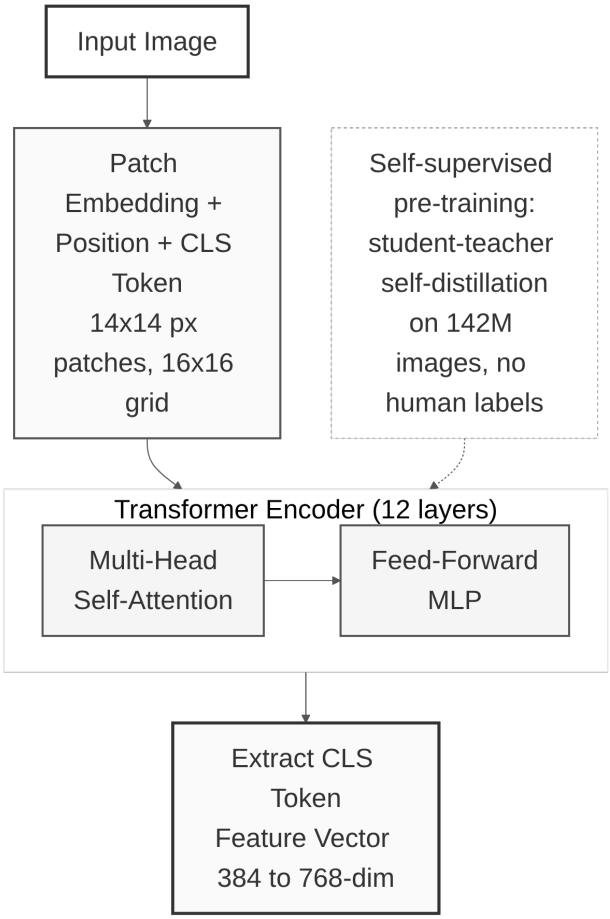

Figure 3: DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector

DINOv2 produces features that exhibit strong object-level structure: silhouettes, part geometry, and garment boundaries, without being trained on category labels. This makes it adaptable to fashion domains where curated labels are scarce.

1.3.2.4 Structural Fidelity for Fashion Retrieval

DINOv2 is particularly good at capturing **structural fidelity**: the shapes, silhouettes, and proportions of objects. For fashion, this means:

- Matching garments with similar cuts (A-line, fitted, oversized).
- Finding items with similar proportions (cropped vs. full-length).

- Ignoring colour differences when the shape is similar.

This is valuable for users who might want "a dress shaped like this one, but in a different colour."

1.3.2.5 DINOv2 Model Specifications

Table 3: DINOv2 model specifications evaluated in this project

Property	**DINOv2 ViT-S/14**	**DINOv2 ViT-B/14**
Architecture	Vision Transformer (Small)	Vision Transformer (Base)
Parameters	21M	86M
Patch size	14 by 14 pixels	14 by 14 pixels
Embedding dimension	384	768
Training data	142M images	142M images
Training method	Self-supervised	Self-supervised

1.3.2.6 Trade-offs and Limitations

Vision Transformers have different trade-offs compared to CNNs:

Advantages:

- Better at understanding global structure and long-range relationships.
- Learned without human labels, so potentially more general.
- Strong structural fidelity: captures shapes and silhouettes.

Disadvantages:

- Slower than CNNs (more computation required).
- Require larger input images for best results.
- May be overkill for simple colour/pattern matching.

Both CNNs and Vision Transformers learn from images alone, mapping visual features to embedding vectors. However, fashion search often involves natural language queries like "red floral summer dress" or "casual weekend outfit." This requires models that can understand both images and text. The next section introduces CLIP and Fashion-CLIP, which bridge this gap through multimodal learning.

1.3.3 CLIP and Fashion-CLIP

CLIP (Contrastive Language-Image Pre-training) bridges the gap between images and natural language, enabling search using both visual and textual queries. This section explains how CLIP works and introduces Fashion-CLIP, the domain-specialized variant used for visual search.

1.3.3.1 Contrastive Language-Image Pre-Training

Traditional image models classify images into fixed categories ("cat," "dog," "dress"). CLIP takes a different approach: it learns to match images with text descriptions [5].

During training, CLIP processed 400 million image-text pairs from the public web. For example, an image of a floral dress paired with the caption "colourful floral summer dress," or sneakers paired with "white running shoes on grass." From these pairs, the model learned to:

- Convert images into vectors (image encoder).
- Convert text into vectors (text encoder).
- Make matching image-text pairs produce similar vectors.

1.3.3.2 Dual-Tower Architecture

CLIP has two separate towers:

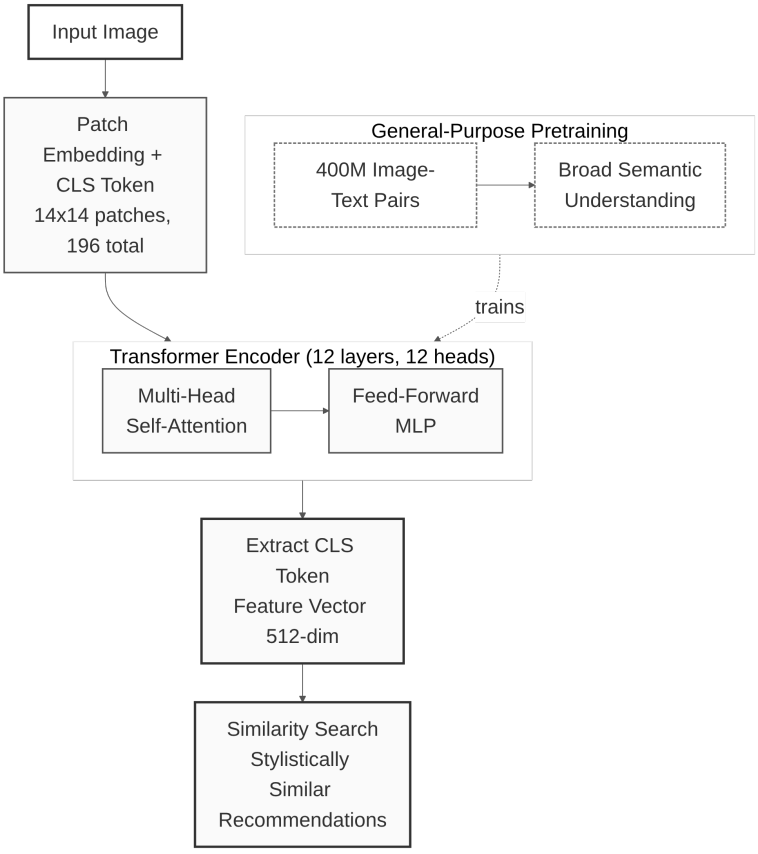

Figure 4: CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison

- **Image Tower.** Processes the image using a Vision Transformer (ViT-B/16, ViT-B/32, or ViT-L/14).
- **Text Tower.** Processes text using a transformer.

Both towers output vectors of the same size (512 dimensions for ViT-B variants, 768 for ViT-L), so images and text can be directly compared using cosine similarity.

1.3.3.3 Multimodal Embedding Space

CLIP's ability to understand natural language makes it powerful for fashion:

- It understands concepts like "bohemian style" or "minimalist design."
- It can match images to abstract descriptions ("something for a casual Friday").
- It captures semantic meaning beyond just visual appearance.

However, the general CLIP model was trained on diverse internet images, not specifically fashion. It may not distinguish "A-line dress" from "sheath dress" or "Bohemian style" from "vintage aesthetic." This is where Fashion-CLIP comes in.

1.3.3.4 Multimodal Query Capabilities

The dual-tower design enables query modalities unavailable in vision-only models such as DINOv2 or EfficientNet:

- **Text-to-image search.** A user types "red floral summer dress"; the text encoder maps the description into the same embedding space as catalog images.
- **Hybrid queries.** An uploaded photo can be combined with a textual refinement ("like this, but in blue") by encoding both modalities and merging the results.

This flexibility makes CLIP-based models the primary choice for the visual search feature.

1.3.3.5 Fashion-CLIP and Domain-Specific Fine-Tuning

Fashion-CLIP is a version of CLIP further trained on fashion-specific data [6]. The researchers used over 700,000 fashion product images paired with detailed descriptions covering garment categories, fabric textures, style descriptors, and occasion labels.

This specialization helps Fashion-CLIP understand:

- Fashion-specific vocabulary ("A-line," "empire waist," "distressed denim").
- Style categories ("streetwear," "preppy," "athleisure").
- Occasion suitability ("office wear," "cocktail party," "beach vacation").

Fashion-CLIP inherits the ViT-B/16 architecture, producing 512-dimensional embeddings. The original paper reports a 15-to-20% improvement on fashion retrieval over general CLIP, a result confirmed in the benchmark evaluation presented in Chapter 3.

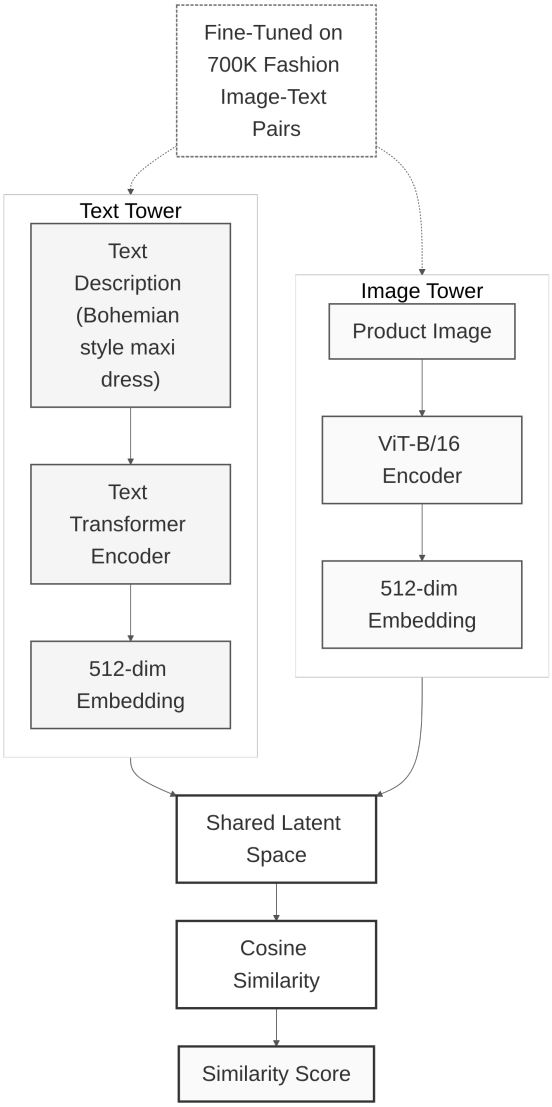

Figure 5: Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space

1.3.3.6 Evaluated CLIP Variants

Table 4: CLIP variants evaluated

Model	Architecture	Training	Domain
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General

Model	Architecture	Training	Domain
Fashion-CLIP	Dual-tower (ViT + text transformer)	Contrastive, fine-tuned on 700K fashion images	Fashion-specific

Having introduced four model families (CNN, ViT, general CLIP, and Fashion-CLIP), the next section presents a comparative analysis to determine which model best balances retrieval quality, inference speed, and feature capabilities for the fashion e-commerce use case.

1.3.4 Model Selection and Justification

This section presents the comparative analysis of the embedding models and the rationale for selecting Fashion-CLIP as the primary model for visual search.

1.3.4.1 Candidate Models

Eleven pre-trained models spanning three architectural families were evaluated:

Table 5: Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available.

Model	Architecture	Dim	Parameters	Training Method
ResNet-50	CNN	2048	25.6M	Supervised (ImageNet)
ResNet-101	CNN	2048	44.5M	Supervised (ImageNet)
EfficientNet-B0	CNN	1280	5.3M	Supervised (ImageNet)
EfficientNet-B4	CNN	1792	19.3M	Supervised (ImageNet)
DINOv2 ViT-S/14	ViT	384	21M	Self-supervised (142M images)
DINOv2 ViT-B/14	ViT	768	86M	Self-supervised (142M images)
CLIP ViT-B/32	ViT + Text	512	151M	Contrastive (400M pairs)
CLIP ViT-B/16	ViT + Text	512	150M	Contrastive (400M pairs)
CLIP ViT-L/14	ViT + Text	768	428M	Contrastive (400M pairs)
Fashion-CLIP	ViT + Text	512	150M	Fine-tuned (700K fashion)

1.3.4.2 Evaluation Methodology

To ensure fair comparison, all models were evaluated under identical conditions. The evaluation used 5,000 fashion product images from the Fashion Product Images dataset, split into training and query sets. Hardware was consumer-

grade: Intel i7-1165G7 CPU with 16 GB RAM, with all inference executed on CPU.

Metrics included:

- **Mean Average Precision (mAP@10).** Primary metric measuring overall retrieval quality.
- **Precision at K (P@1, P@5, P@10).** Accuracy of the top-K results.
- **Recall at 10 (R@10).** Coverage of relevant items in the top-10 results.
- **Inference latency.** Average time to generate one embedding (milliseconds).

A retrieved product was considered relevant if it belonged to the same category as the query image. The full evaluation protocol, benchmark results, and cross-validation methodology are presented in Chapter 3.

1.3.4.3 Weighted Selection Criteria

The model selection was based on four criteria:

1. **Retrieval quality.** mAP@10 and P@K scores measuring how well the model retrieves visually similar products.
2. **Inference latency.** Must support real-time search with sub-300 ms total response time.
3. **Multimodal capability.** The ability to search by both image and text, enabling text-to-image queries.
4. **Hardware compatibility.** Must operate within the memory and compute constraints of commodity hardware.

1.3.4.4 Selection Decision

Fashion-CLIP was selected as the primary embedding model for the visual search feature. Three factors drove this decision:

Retrieval quality. Fashion-CLIP achieved the highest mAP@10 among the evaluated models, with a 15-to-20% improvement over general CLIP on fashion-specific queries. This result was confirmed through the systematic benchmark presented in Chapter 3.

Multimodal capability. Fashion-CLIP's dual-tower architecture enables search by image, by text description, and by hybrid image-plus-text queries. This capability is unavailable in vision-only models such as DINOv2 and EfficientNet.

Domain specialization. Fine-tuning on 700,000 fashion images gives Fashion-CLIP an understanding of fashion-specific vocabulary, styles, and garment attributes that general-purpose models lack.

While EfficientNet-B0 offers faster inference (suitable for ultra-low-latency mobile deployments) and DINOv2 excels at structural fidelity (useful

for silhouette-based matching), Fashion-CLIP provides the best overall balance of retrieval quality, search flexibility, and inference performance for the target deployment scenario.

1.3.4.5 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred:

- **Ultra-low latency.** EfficientNet-B0 provides the fastest inference at 5.3M parameters. Trade-off: 3.4% lower mAP@10 and no text-to-image search.
- **Maximum structural accuracy.** DINOv2 excels at shape and silhouette matching. Trade-off: no multimodal capability.
- **Non-fashion e-commerce.** General CLIP variants are suitable for multi-category marketplaces. Trade-off: lower accuracy on fashion-specific queries.
- **High-resource environments.** CLIP ViT-L/14 offers the largest model capacity. Trade-off: 428M parameters, requiring GPU with significant VRAM.

The complete numerical comparison and error analysis across all 11 models are presented in Chapter 3.

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. This section explains the challenge of vector similarity search at scale, introduces two indexing algorithms, and describes pgvector, the PostgreSQL extension used in this project.

1.4.1 The Search Challenge and Approximate Nearest Neighbours

Consider a catalog of 10,000 fashion products, each represented by a 512-dimensional embedding vector. When a user uploads a query image, the system must compare that query vector to all 10,000 product vectors to find the most similar ones. This is called **nearest neighbour search**.

A naive approach computes the distance to every vector: 10,000 comparisons per search. For a catalog of 100,000 products, that becomes 100,000 comparisons. For one million products, one million comparisons. For real-time search (responding under one second), this brute-force approach is too slow.

The solution is **Approximate Nearest Neighbour** (ANN) search. Rather than checking every vector, ANN algorithms use index structures that organise vectors into navigable graphs or clusters. A query navigates directly toward the neighbourhood of likely matches, skipping the vast majority of irrelevant vectors.

The key insight is simple: we do not need the absolute best match. If the true best match has a similarity of 0.95 and the returned result has 0.93, that

is acceptable for product search. The accuracy trade-off is modest: ANN algorithms typically achieve **97 to 99% recall** of the true top matches, while reducing search time by several orders of magnitude. For product search, where returning 20 visually similar items matters far more than guaranteeing the absolute 21st best match, this trade-off is entirely acceptable.

Two ANN indexing algorithms are used in this project: HNSW for production-scale search and IVFFlat for rapid evaluation.

1.4.2 HNSW: Hierarchical Navigable Small World

HNSW is one of the most effective ANN algorithms and the preferred index for production-scale vector search in this project [12]. It builds a multi-layered graph structure where each vector is a node connected to its nearest neighbours.

The graph has multiple layers. The top layers are sparse and connect distant regions of the embedding space, enabling long-range jumps. The bottom layers are dense and refine the search locally. To search, the algorithm starts at a node in the top layer, greedily traverses edges toward the nearest neighbour, descends to the next layer, and repeats. The process converges on the query neighbourhood in logarithmic time: search cost scales with the logarithm of the catalog size rather than the size itself.

HNSW has two main configuration parameters:

- **M.** The number of connections per node. Higher values improve recall but increase memory usage and build time. The default value of 16 provides a good balance for most use cases.
- **ef_construction.** How many candidates to consider during index construction. Higher values produce a better index at the cost of longer build time.

HNSW consistently exceeds 95% recall at query latencies under 10 milliseconds, sustained across catalog scales of up to 10 million vectors. Its logarithmic query cost makes it suitable for interactive fashion retrieval at millions of catalog items, where sub-100 ms response time is required.

1.4.3 IVFFlat: Inverted File with Flat Compression

IVFFlat is a simpler ANN algorithm used during evaluation. It partitions the embedding space into clusters using **k-means** and stores each vector in the cluster whose centroid it is nearest to. A query computes the distance to all cluster centroids, selects the nearest few clusters, and searches exhaustively only within those selected clusters.

IVFFlat has two configuration parameters:

- **lists.** The number of clusters. More lists mean smaller clusters and faster search, but risk missing relevant vectors in unexamined clusters.
- **probes.** The number of clusters searched per query. More probes improve recall at the cost of query latency.

Compared to HNSW, IVFFlat's build cost is low: clustering completes in under one second for 5,000 vectors. However, recall is moderate: 65 to 72% at sub-10 ms latency for catalog-scale data. Recall degrades as the catalog grows because each cluster contains more candidates.

IVFFlat is used for the model comparison benchmarks in Chapter 3, where the objective is ranking embedding models rather than optimising index performance. Its fast build time and simple configuration suit rapid evaluation. For production deployment, HNSW is the designated long-term index.

1.4.4 pgvector: Vector Search in PostgreSQL

pgvector is an open-source extension that adds vector operations to PostgreSQL [13]. It allows storing vectors alongside regular product data (names, prices, images) in the same database, using standard SQL.

Key features of pgvector:

- **Vector column type.** `VECTOR(512)` stores 512-dimensional embeddings. The extension accommodates the different embedding dimensions produced by different models.
- **Similarity operators.** The `<=>` operator computes cosine distance (values in the range [0, 2], where 0 means identical and 2 means maximally dissimilar). The `<->` operator computes Euclidean distance.
- **Index support.** Both HNSW and IVFFlat indexing are supported for fast approximate search.

1.4.4.1 Why pgvector Over a Separate Vector Database

The critical advantage of pgvector is **transactional consistency**. Vectors and product metadata live in the same database. A product update and its embedding update occur within a single ACID transaction. If an admin changes a product image, the new embedding is committed atomically with the catalog update. This eliminates the dual-database problem: a separate vector store that can drift out of sync with the relational source of truth.

Combined queries present a second advantage. Vector similarity and relational filtering can be combined in a single SQL statement: find products visually similar to a query image, restricted to a specific category and within a price range, using one query plan. With separate databases, this requires querying the vector store, collecting result IDs, and querying the relational store in a second pass.

1.4.4.2 Example Usage

A simplified example of how vectors are stored and searched:

```
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    price DECIMAL,
    image_embedding VECTOR(512)
);

CREATE INDEX ON products
USING hnsw (image_embedding vector_cosine_ops);

SELECT id, name, price,
       1 - (image_embedding <=> query_vector) AS similarity
FROM products
ORDER BY image_embedding <=> query_vector
LIMIT 10;
```

The HNSW index enables the `ORDER BY ... <=>` clause to execute in logarithmic time rather than scanning the entire table.

1.4.5 Architectural Decision and Trade-offs

Several specialised vector databases exist (Pinecone, Milvus, Weaviate), each optimised for large-scale vector search. pgvector was selected for practical reasons suited to this project's scope.

Table 6: Comparison of pgvector with specialised vector databases

Feature	Specialised Vector DB	pgvector
Setup complexity	Moderate to high	Low (PostgreSQL extension)
Data consistency	Separate from main database	Same transaction as product data
Query language	Custom API or query language	Standard SQL
Cost	Often paid service (SaaS)	Free and open source
Scale limit	Billions of vectors	Millions of vectors

For a system with thousands to tens of thousands of products, pgvector's simplicity and transactional consistency outweigh the scaling advantages of specialised databases. If the system needed to scale to tens of millions of products, migration to a dedicated vector database would be considered.

1.4.5.1 Trade-offs Acknowledged

Using pgvector has limitations:

- **Scale ceiling.** pgvector performs well for millions of vectors but is not designed for billion-vector deployments.

- **Less mature.** Fewer features and optimisation options than dedicated vector databases.
- **Single-node.** pgvector does not natively distribute across multiple servers.

For this project's scope (5,000 products in the evaluation, with a target of tens of thousands in production), these limitations are acceptable. The primary contribution is the architectural integration of vector search within a conventional e-commerce stack, not massive-scale infrastructure.

1.5 PLATFORM ARCHITECTURE AND TECHNOLOGY STACK

Building a production-capable e-commerce platform with integrated machine learning requires deliberate architectural and technology choices. This section surveys architectural patterns, describes each technology in the ReSys.Shop stack, and provides the rationale for each selection.

1.5.1 Architectural Patterns

An application's **architecture** determines how code is partitioned and how components communicate. Three patterns govern system-level structure; a fourth, **vertical slice architecture**, organises code within each module.

1.5.1.1 Monolith

All code runs in a single deployable unit.

- Single codebase, build, and deployment; no network overhead between components.
- Components entangle over time: modifying one area requires understanding others, and any deployment restarts the entire application.

Suitable for small applications with one team and few business domains.

1.5.1.2 Microservices

Independent services each own one business capability, communicating over the network.

- Teams operate autonomously with different technology stacks; services scale and deploy independently.
- Introduces service discovery, inter-service authentication, partial failure handling, and distributed transaction coordination. Each network hop adds latency.

Suitable for large organisations where teams and domains evolve independently.

1.5.1.3 Modular Monolith

Code partitions into independent modules within a single process. Modules communicate through an **in-process message bus**; compile-time rules prevent direct cross-module references.

- Preserves module independence without networking cost. One deployment and one database serve all modules.
- Established boundaries simplify future migration to microservices.
- The single database may limit extreme-scale growth; all modules restart on any deployment.

Suitable for single-team projects requiring logical separation without distributed infrastructure.

1.5.1.4 Vertical Slice Architecture

Each feature is a self-contained folder containing its handler, validation, data access, and endpoint definition. This pattern is orthogonal to those above: it organises code **within** a module, not **across** the system.

- All code for one feature is co-located; features are added, modified, or removed in isolation.
- Cross-cutting concerns (authentication, logging) require explicit extraction to avoid duplication across slices.

Suitable when features are numerous, independent, and evolve at different cadences.

1.5.1.5 Architectural Decision

ReSys.Shop combines three patterns. At the system level, a **modular monolith** partitions nine business modules (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard) into a single process with an **in-process message bus**. Within each module, **vertical slice architecture** organises features as self-contained folders. A dedicated **Python sidecar**, the only cross-process component, handles embedding generation to isolate PyTorch dependencies from the .NET runtime; a GPU failure in the sidecar does not affect e-commerce API availability. This combination provides the deployment simplicity of a monolith, the code isolation of microservices, and machine learning capability without distributed infrastructure overhead.

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.
- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage. Migrations are version-controlled and applied at startup.
- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** (described in Section 1.5.1) organises each feature as a self-contained folder containing the handler, request, response, endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database.

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

The pgvector extension, HNSW indexing, and transactional consistency between product data and embeddings are described in detail in Section 1.4.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement.

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.
- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR).

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn serves as the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** `POST /embeddings` accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. `GET /health` reports

the currently loaded model, its embedding dimension, and the most recent inference latency.

- **Security.** Requests require an `X-API-Key` header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 .NET Aspire Orchestration

.NET Aspire coordinates the multi-container development and deployment environment.

- **Service discovery.** Components resolve each other by name: the .NET backend reaches the Python sidecar at `http://ml-service`, PostgreSQL at `postgres`, and Redis at `redis`. Configuration is injected via environment variables managed by the Aspire host.
- **Lifecycle.** Aspire enforces startup ordering (database before API, sidecar before backend health check) and gates each component behind a readiness probe. Containers restart on failure with configurable back-off.
- **Observability.** OpenTelemetry SDKs in both .NET and Python services export distributed traces, request metrics, and structured logs to the Aspire dashboard. Correlation IDs propagate across the HTTP boundary between backend and sidecar, linking a search request to its embedding generation span.

1.5.8 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs daily, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.
- **Index maintenance.** A periodic job rebuilds the HNSW index on the embedding column, optimising search performance as the catalog grows.

1.5.9 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP.
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.
- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.
- **Permission model.** Authorisation uses granular claims in the format `domain:category:action` (e.g., `catalog:products:create`). Roles aggregate commonly used permission sets. Endpoint-level attributes evaluate claims at the middleware layer, so handler code contains no authorisation logic.

1.5.10 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic, reproducible evaluation of embedding models on fashion product retrieval. It is separate from the application codebase and produces the experimental results reported in Chapter 3.

- **Modes.** Three evaluation modes are supported: a one-shot comparison across all configured models, a three-fold cross-validation protocol with stratified category-based splits (the default for thesis results), and a pgvector pipeline mode that measures end-to-end latency including database query and network round-trip time.
- **Models.** 11 pre-trained architectures are implemented: CNN-based (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), vision transformers (DINOv2 ViT-S/14, DINOv2 ViT-B/14), and CLIP variants (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, Fashion-CLIP). Each model has a thin adapter implementing a common `generate_embeddings` interface.
- **Caching.** Embeddings are cached to disk per model, per fold, and per split (query vs. catalog), avoiding recomputation when re-running evaluation on the same configuration.

- **Metrics.** Retrieval accuracy is measured through mAP, Precision at K, Recall at K, and nDCG. Operational efficiency is measured through inference latency (mean and SD per image), throughput (images per second), model load time, on-disk storage, and RAM usage.
- **Outputs.** Results are exported in JSON, CSV, Markdown, and Typst table formats. The Typst output files (`thesis_aggregate.typ`, `thesis_efficiency.typ`) embed directly into the thesis without manual transcription.
- **Multi-label pipeline.** A separate enriched-dataset mode supports evaluation across three label schemes of increasing granularity (category only, category and colour, and category, colour and pattern), enabling analysis of how embedding quality varies with annotation detail.

1.5.11 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table 7 consolidates the complete stack.

Table 7: Technology stack of the ReSys.Shop platform

Layer	Technology	Role
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface; reactive UI with Pinia state management
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs; CQRS command-query separation across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings in a single ACID database with HNSW-indexed similarity search
Caching	Redis, HybridCache	Two-tier cache (in-memory L1 and Redis L2); Hangfire job queue and session state backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Dedicated embedding generation service with lazy model loading and GPU acceleration
Orchestration	.NET Aspire	Container lifecycle management, service discovery, and reproducible local development environment
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance tasks
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation, guest sessions

Layer	Technology	Role
Benchmarking	Python 3.12, PyTorch	Systematic 11-model comparison with cross-validation, accuracy and efficiency metrics

1.6 RELATED WORK AND RESEARCH GAP

This section positions the ReSys.Shop platform within the landscape of fashion image retrieval research and commercial visual search systems, and identifies the engineering gap that this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [14]. This dataset catalysed much of the subsequent work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback ("like this dress but shorter") [15]. While compelling, the interactive dialogue paradigm requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval by 15 to 20% over the general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the evaluation to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 8: Comparison of commercial visual search products

Product	Key Strength	Limitation
Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only

ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share common limitations for independent projects: they are proprietary and cannot be studied or modified, API access incurs costs at query volume, and reliance on external services creates vendor lock-in. This thesis demonstrates that comparable functionality is achievable with open-source tools, providing both a reference implementation and a cost-effective alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project distinguishes itself from prior work by addressing the **engineering gap** between model research and production systems. Four contributions define this gap:

1. Polyglot architecture. Python's machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar, combining .NET's type safety and transactional integrity with Python's access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, eliminating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis evaluates 11 models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Rather than chasing leaderboard metrics, this thesis compares models within realistic deployment constraints (inference latency budget, memory limits, storage cost). The resulting accuracy-efficiency trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

2 DESIGN AND IMPLEMENTATION

This chapter translates requirements into a concrete system design and describes the implementation of its principal components.

- **Requirements Analysis.** System actors, 88 functional requirements, non-functional requirements, feature classification.
- **Functional Decomposition and Use Cases.** 26 capability-grouped use cases across administration, storefront, and system services.
- **System Architecture and Design.** Domain-driven design, C4 diagrams, database schema, API design, security model.
- **Implementation.** Vertical slice organisation, ML pipeline, CBIR search flow, .NET backend, Python sidecar, Vue.js storefront.

2.1 REQUIREMENTS ANALYSIS

This section defines the scope of the ReSys.Shop platform by identifying its actors, functional and non-functional requirements, and the classification of features into research contributions and supporting infrastructure. The analysis establishes what the system must do before proceeding to its architectural design and implementation. Use cases are treated separately in Section 2.2.

2.1.1 System Actors

The platform serves three categories of actors, defined by their access level, responsibilities, and interaction surface.

2.1.1.1 Customer: Product Discovery and Purchase

The **Customer** accesses the platform through the browser-based **storefront** and is the primary beneficiary of the research contribution.

- **Product Discovery.** Browse the catalog with faceted filters and hierarchical category navigation. Perform **keyword searches** by product name or attribute. Perform **visual searches** by uploading a reference image; the system returns visually similar products ranked by embedding similarity.
- **Cart and Checkout.** Add products to a persistent cart (guest or authenticated). Complete a **multi-step checkout** spanning **address selection**, **delivery method choice**, **payment confirmation**, and **order finalisation**.
- **Account.** Register with email and password or **Google OAuth**. Manage **profile details**, **shipping addresses**, and **wishlists**. View **order history** and track fulfilment status.
- **Guest capability.** Browse, search, and manage a cart without registration. On login or registration, the **guest session is promoted** to the authenticated context, preserving cart contents.

2.1.1.2 Administrator: Data Management and Operational Oversight

The **Administrator** operates the **administration interface**, a separate application surface with independent authentication.

- **Product Management.** Create, update, archive, and delete products. Define **fashion-specific metadata**: style code, season, material composition, department, gender target. Manage **variants** (size and colour combinations) with SKUs, barcodes, dimensions, and independent pricing. Upload product images; each upload triggers the **embedding generation pipeline**.
- **Taxonomy and Classification.** Define **hierarchical taxonomies** (e.g., Clothing → Dresses → Evening Dresses). Assign products to taxon nodes. Manage **option types** (Size, Colour, Material) with ordered values.
- **Order Fulfilment.** Review incoming orders, update **fulfilment status**, process **payment captures** and **refunds**, and manage **shipment tracking**.
- **Inventory Monitoring.** View **real-time stock levels** per variant per location. Review **stock movement history** for audit purposes.
- **User Governance.** Create and manage user accounts. Assign **roles** (Customer, Administrator) and granular **permissions** following a `domain:category:action` format.

2.1.1.3 System: Automated Background Processes

The **System** actor executes without human interaction through **scheduled** and **event-driven** background jobs.

- **Embedding Generation.** Processes uploaded images through the ML sidecar. Uploads return immediately; embeddings become available for search when processing completes.
- **Cart Expiry.** A **daily scheduled job** removes carts inactive for **seven days**, releasing reserved inventory.
- **Inventory Reservation.** Holds stock during active checkout. Releases expired reservations after **fifteen minutes** of inactivity.
- **Index Maintenance.** Periodic rebuilds maintain **search performance** as the catalog grows.
- **Payment Webhooks.** Validates and processes incoming gateway webhooks **asynchronously**, updating payment state without blocking the response.

2.1.2 Functional Requirements

The platform's capabilities are specified as traceable requirements organised by business module. Each module is introduced with a single sentence of context;

the table that follows enumerates specific requirements with identifiers that can be referenced throughout the design and evaluation chapters.

2.1.2.1 Catalog Module

The Catalog module manages the product lifecycle, from creation and classification through image handling to CBIR infrastructure.

Table 9: Catalog module functional requirements.

ID	Requirement	Description	Priority
CAT-FR-01	Product creation	Create products with name, description, slug, and SEO metadata (meta title, meta description)	High
CAT-FR-02	Fashion metadata	Assign fashion-specific fields: style code, season, material composition, care instructions, fit notes, department, and gender target	Medium
CAT-FR-03	Product variants	Define sellable variants combining option values (e.g., Size M + Colour Red) with SKU, barcode, physical dimensions, weight, and independent pricing	High
CAT-FR-21	Variant option values	Assign, synchronise, retrieve, and revoke option values on variants to define the specific attribute combination each variant represents	High
CAT-FR-22	Variant pricing	Set, list, synchronise, and remove time-bound prices per variant with currency specification; support multi-currency pricing for international catalogues	Medium
CAT-FR-04	Variant images	Upload variant images with automatic thumbnail generation; support multiple images per variant with display ordering	High
CAT-FR-14	Variant image CRUD	Delete, download image by ID, list images by variant, and update image metadata (alt text, display order) for variant images	Medium
CAT-FR-15	Embedding regeneration	Regenerate vector embeddings for existing images when the configured model changes or the original embedding is corrupted	Medium
CAT-FR-05	Embedding generation	Generate vector embeddings for each uploaded variant image via the configured ML model and store in pgvector with model metadata (model name, version, dimension)	High
CAT-FR-06	Image-based search	Enable CBIR: upload query image, generate embedding, query pgvector	High

ID	Requirement	Description	Priority
		with cosine distance, return similarity-ranked results	
CAT-FR-16	Product availability	Expose real-time per-variant stock availability through the storefront product detail endpoint	High
CAT-FR-17	Similar products	Retrieve visually similar products via embedding similarity for a given product, enabling recommendation on product detail pages	Medium
CAT-FR-07	Search configuration	Configure minimum similarity threshold and maximum result count for CBIR queries	Medium
CAT-FR-08	Pluggable models	Switch embedding model via environment variable without code changes; filter search results by active model	High
CAT-FR-09	Taxonomy	Organise products via hierarchical taxonomies with nested taxon trees (e.g., Clothing → Dresses → Evening Dresses)	Medium
CAT-FR-18	Taxon rules	Define business rules attached to taxon nodes (attribute constraints, automatic assignments) with update, sync, and retrieval operations	Low
CAT-FR-19	Auto-classification	Automatically classify products into appropriate taxons based on taxon rules, invoked when product attributes are updated	Low
CAT-FR-10	Option types	Define option types (e.g., Size, Colour, Material) with ordered option values reusable across multiple products	Medium
CAT-FR-20	Product option type association	Assign, synchronise, retrieve, and revoke option types per product, enabling products to expose relevant variant dimensions	Medium
CAT-FR-11	Slug uniqueness	Enforce slug uniqueness across the entire catalog at the database level	High
CAT-FR-12	Status lifecycle	Support product status lifecycle: Draft, Active, Archived with configurable availability and discontinuation dates	Medium
CAT-FR-13	Master variant	Designate one variant per product as the master (default) variant used for catalog listing display	High

2.1.2.2 Identity Module

The Identity module provides authentication, authorisation, and user management for both customer-facing and administrative functions.

Table 10: Identity module functional requirements.

ID	Requirement	Description	Priority
IDN-FR-01	Registration	Register new customer accounts with email, password, and basic profile information	High
IDN-FR-02	Password login	Authenticate users via email and password, returning JWT access token (15-minute lifetime) and refresh token	High
IDN-FR-03	OAuth login	Authenticate users via Google OAuth 2.0 as an alternative to password-based login	Medium
IDN-FR-04	Token rotation	Implement refresh token rotation: each refresh operation invalidates the previous token and issues a new access and refresh token pair	High
IDN-FR-05	Reuse detection	Detect refresh token reuse: presenting a previously consumed token revokes all active tokens for that user, containing credential theft	High
IDN-FR-06	Guest sessions	Support guest sessions via signed cookie-based identifiers, enabling anonymous cart usage and catalog browsing without registration	High
IDN-FR-07	Role-based access	Enforce role-based access control (Customer, Administrator) with permission granularity at domain:category:action level	High
IDN-FR-11	Role management	Create, update, delete, and list roles with paging; assign, synchronise, retrieve, and revoke permissions per role	Medium
IDN-FR-12	User-role assignment	Assign, synchronise, retrieve, and revoke roles for individual users; support direct permission grants bypassing role inheritance	Medium
IDN-FR-13	User status management	Enable and disable user accounts via the admin interface without deleting the account or its associated data	Medium
IDN-FR-08	Password reset	Reset passwords via time-limited, single-use email token with configurable expiry	Medium
IDN-FR-14	Password change	Allow authenticated users to change their password while logged in, requiring current password verification	Medium

ID	Requirement	Description	Priority
IDN-FR-15	Email lifecycle	Confirm email addresses via verification token, change email with re-verification, and resend verification emails on demand	Medium
IDN-FR-16	Session management	Retrieve current session details, refresh tokens, and explicitly terminate sessions via logout	High
IDN-FR-09	User management	Manage user accounts, role assignments, and permission grants through the administration interface	Medium
IDN-FR-10	Two-factor auth	Support optional two-factor authentication via time-based one-time password (TOTP)	Low

2.1.2.3 Inventory Module

The Inventory module tracks physical stock quantities, manages checkout-time reservations to prevent overselling, and records stock movements for audit trails.

Table 11: Inventory module functional requirements.

ID	Requirement	Description	Priority
INV-FR-01	Stock locations	Define stock locations (warehouses or stores) with name, address, and active status flag	Medium
INV-FR-02	Stock quantities	Track stock quantities per product variant per location with separate on-hand and reserved counters	High
INV-FR-08	Stock item CRUD	Create, update, delete, retrieve by ID, and list all stock items with paging; support bulk adjustment across multiple items and CSV-based import for batch operations	Medium
INV-FR-09	Low stock alerting	Identify and list stock items where on-hand quantity falls below a configured threshold, enabling proactive replenishment	Low
INV-FR-03	Checkout reservation	Reserve inventory quantities during active checkout sessions; release on cart expiry, order cancellation, or checkout timeout	High
INV-FR-11	Cart-based reservation	Reserve stock at the cart level during active shopping sessions, retrieve reservation status, and release on cart abandonment or expiry	High

ID	Requirement	Description	Priority
INV-FR-04	Overselling prevention	Reject checkout when requested quantity exceeds available stock (on-hand minus reserved) at time of order confirmation	High
INV-FR-05	Stock transfers	Record stock transfers between locations with source, destination, quantity, and timestamp metadata	Medium
INV-FR-10	Transfer lifecycle	Manage the full stock transfer lifecycle: create a transfer, record in-transit status, confirm receipt at the destination, and cancel pending transfers; list transfers with paging and detail view	Medium
INV-FR-06	Audit log	Maintain an immutable audit log for all stock level changes including manual adjustments with reason and operator identity	Medium
INV-FR-12	Movement audit trail	Provide a dedicated paged listing and detail view for all stock movements with source, destination, quantity, reason, and operator identity; support the stock movement entity as a first-class data model with query capabilities	Medium
INV-FR-07	Reservation expiry	Expire stale reservations after a configurable time window (default 15 minutes of checkout inactivity)	Medium

2.1.2.4 Ordering Module

The Ordering module handles the customer purchase workflow from cart through multi-step checkout to completed order, tracking payment and shipment state independently.

Table 12: Ordering module functional requirements.

ID	Requirement	Description	Priority
ORD-FR-01	Shopping cart	Support guest and authenticated shopping carts with product variant selection, quantity management, and per-item pricing	High
ORD-FR-10	Cart management	Create, delete, and empty shopping carts; update item quantities and remove specific items from the cart	High
ORD-FR-02	Cart association	Associate a guest cart with a user account upon login or registration, merging contents without data loss	Medium

ID	Requirement	Description	Priority
ORD-FR-03	Cart expiry	Auto-expire abandoned carts after seven days of inactivity via scheduled background job	Medium
ORD-FR-04	Checkout states	Enforce forward-only checkout state machine: Address, Delivery, Payment, Confirm, Complete	High
ORD-FR-11	Checkout validation	Validate checkout constraints (stock availability, address completeness, shipping method selection) before proceeding to payment	High
ORD-FR-12	Shipping rate selection	Select a shipping rate from available options within the cart before proceeding to payment	Medium
ORD-FR-05	Order totals	Calculate item total, adjustment total (discounts, surcharges), and shipment total independently; derive order total as their sum	High
ORD-FR-06	Dual state tracking	Track payment state (unpaid, authorised, captured, refunded) and shipment state (pending, shipped, delivered) independently per order	Medium
ORD-FR-07	Cancellation	Allow order cancellation at any pre-confirmation stage; cancelled orders release reserved inventory and void payment intents	Medium
ORD-FR-13	Admin order lifecycle	Approve, complete, and resume orders through the admin interface; update order status, shipping method, and shipping/billing addresses	Medium
ORD-FR-14	Customer order history	Allow authenticated customers to list their orders with paging and view individual order detail including line items, payments, and shipment tracking	Medium
ORD-FR-08	Order numbering	Generate unique, sequential, human-readable order numbers within a database transaction to prevent collisions under concurrent requests	Medium
ORD-FR-09	Order immutability	Mark completed orders as immutable; prevent modification of line items, adjustments, or totals after finalisation	High

2.1.2.5 Payment Module

The Payment module manages the lifecycle of payment intents across the Stripe production gateway and a Bogus test gateway.

Table 13: Payment module functional requirements.

ID	Requirement	Description	Priority
PAY-FR-01	Intent creation	Create payment intents with amount, currency, order identifier, and gateway selection	High
PAY-FR-02	Intent confirmation	Confirm payment intents to authorise fund capture at checkout completion	High
PAY-FR-08	Setup intent	Create Stripe SetupIntents for saving customer payment methods for future use, separate from transactional PaymentIntents	Low
PAY-FR-03	Capture and refund	Capture, void, and refund payment intents with idempotency keys to prevent duplicate processing on retry	High
PAY-FR-04	Webhook processing	Process Stripe webhooks with cryptographic signature verification to confirm payment state transitions	High
PAY-FR-09	Payment void	Void authorised but un-captured payments as a distinct operation from refund; void all payments associated with a cancelled order via a cross-cutting command	Medium
PAY-FR-10	Payment method management	Create, update, delete, activate, and deactivate payment methods through the admin interface with paged listing; expose active methods to the storefront for customer selection	Medium
PAY-FR-05	Refund validation	Validate that refund amount does not exceed captured amount before processing the refund	Medium
PAY-FR-06	Bogus gateway	Provide a Bogus gateway for development and testing that simulates the full payment lifecycle (Pending, Processing, Succeeded, Canceled) without external API calls	Medium
PAY-FR-07	State tracking	Maintain independent payment state tracking in parallel with the gateway; enable offline state queries without gateway round-trips	Medium

2.1.2.6 Shipping Module

The Shipping module configures delivery methods and calculates shipping rates by geographic zone.

Table 14: Shipping module functional requirements.

ID	Requirement	Description	Priority
SHP-FR-01	Delivery methods	Configure delivery methods (standard, express, local pickup) with carrier name, pricing rules, and applicable geographic zones	Medium
SHP-FR-04	Method lifecycle	Create, update, delete, activate, and deactivate shipping methods with paged listing and detail retrieval through the admin interface	Medium
SHP-FR-05	Rate lifecycle	Create, update, delete, and list shipping rates per method, zone, and weight/value tier through the admin interface	Medium
SHP-FR-02	Rate calculation	Calculate shipping rates at checkout based on delivery address zone, selected method, cart weight, and cart value	Medium
SHP-FR-06	Storefront calculation	Calculate shipping cost for a given cart at checkout, returning available methods with rates applicable to the delivery address zone	High
SHP-FR-03	Shipment tracking	Assign per-order shipment tracking with carrier identifier and tracking number for customer visibility	Low

2.1.2.7 Profile Module

The Profile module links customer identity to personalisation features and preference management.

Table 15: Profile module functional requirements.

ID	Requirement	Description	Priority
PRF-FR-01	Addresses	Manage user shipping and billing addresses with address type labels and default selection per type	Medium
PRF-FR-02	Wishlists	Manage wishlists: create, rename, delete lists; add and remove product variants with notes	Low
PRF-FR-03	Notifications	Configure per-channel notification preferences (email, SMS) with opt-in and opt-out per notification category	Low

2.1.2.8 Location Module

The Location module provides country and state reference data for address validation, shipping zone configuration, and tax calculation.

Table 16: Location module functional requirements.

ID	Requirement	Description	Priority
LOC-FR-01	Countries	Provide country reference data with ISO 3166-1 codes, display names, and active status flags	Medium
LOC-FR-02	States	Provide state and province reference data with ISO 3166-2 codes, linked to parent country	Medium
LOC-FR-03	Country CRUD	Create, update, delete, and list countries with paging; retrieve by ID and ISO code through both admin and storefront interfaces	Medium
LOC-FR-04	State CRUD	Create, update, delete, and list states with paging; retrieve by ID and ISO code; filter by parent country through both admin and storefront interfaces	Medium

2.1.2.9 Dashboard Module

The Dashboard module aggregates key performance indicators from all business modules into a unified administrative overview.

Table 17: Dashboard module functional requirements.

ID	Requirement	Description	Priority
DSH-FR-01	Aggregate dashboard	Provide a cross-module administrative dashboard aggregating key metrics (product counts, order volumes, inventory levels, user statistics) from all business modules into a single view	Medium

2.1.3 Non-Functional Requirements

Beyond feature completeness, the system must satisfy quantitative and qualitative constraints that determine its fitness for production use. Five quality dimensions are specified below with atomic, measurable requirements.

2.1.3.1 Performance

Table 18: Performance: latency targets for CBIR search and general API responses

ID	Requirement
NFR-01a	The system shall complete CBIR end-to-end search (image upload, embedding generation, pgvector query, result assembly) within **1 second** per query.
NFR-01b	The system shall respond to non-search API endpoints within **200 milliseconds** under normal load.
NFR-01c	The system shall use asynchronous I/O to prevent thread blocking under concurrent requests.

2.1.3.2 Security

Table 19: Security: authentication, authorisation, rate limiting, upload validation, and transport controls

ID	Requirement
NFR-02a	The system shall expire JWT access tokens after **15 minutes** and enforce single-use refresh token rotation with reuse detection.
NFR-02b	The system shall enforce role-based access control with granular permission claims (`domain:category:action`) at endpoint middleware.
NFR-02c	The system shall rate-limit login to **5 requests per minute** and registration to **3 requests per hour**.
NFR-02d	The system shall validate file uploads by magic-byte signature, extension allowlist, and enforce a **10 MB** size limit.
NFR-02e	The system shall emit security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options) on all HTTP responses.

2.1.3.3 Modularity

Table 20: Modularity: module isolation, communication boundary, and testability

ID	Requirement
NFR-03a	Eight business modules shall reside in a single .NET assembly with **zero direct cross-references** enforced at compile time.
NFR-03b	All inter-module communication shall pass through **MediatR** in-process message dispatch only.
NFR-03c	Each module shall be independently testable without loading other modules or their database contexts.

2.1.3.4 Observability

Table 21: Observability: distributed tracing, correlation logging, and health monitoring

ID	Requirement
NFR-04a	The system shall propagate **OpenTelemetry** distributed tracing spans across both .NET and Python services via HTTP headers.
NFR-04b	Every structured log entry shall carry a **correlation identifier** to track requests across service boundaries.
NFR-04c	Each service shall expose a health check endpoint reporting dependency connectivity for orchestration.

2.1.3.5 Reliability

Table 22: Reliability: job durability, cart expiry, idempotency, and timeout guarantees

ID	Requirement
NFR-05a	Background jobs shall use **Hangfire** with Redis-backed persistence to survive application restarts.
NFR-05b	The system shall remove carts inactive for **seven days** and release reserved inventory, running daily.
NFR-05c	Stripe webhook handlers shall enforce idempotency keys to prevent duplicate payment processing.
NFR-05d	Checkout inventory holds shall expire after **fifteen minutes** of inactivity.

These constraints shaped architectural decisions throughout the system. The one-second CBIR latency target (NFR-01a) ruled out queued embedding pipelines in favour of synchronous generation. The modularity constraint (NFR-03b) drove the MediatR dispatch model over direct method calls. The reliability constraint (NFR-05a) motivated Redis-backed Hangfire over in-memory job storage. Each target is revisited in the evaluation chapter, where benchmark results confirm whether the implemented system meets these stated requirements.

2.1.4 Feature Classification

Not all features of ReSys.Shop carry equal research significance. Seven feature areas are classified as either **Core Research** (directly contributing to the thesis's academic objectives) or **Supporting Infrastructure** (providing the realistic e-commerce context in which the research is conducted and evaluated). This distinction clarifies the scope of the thesis's original contribution and explains why certain features exist in the platform but are not discussed in depth in subsequent chapters.

Table 23: Classification of feature areas into Core Research and Supporting Infrastructure. Core Research features represent the thesis's original contributions; Supporting

Infrastructure features provide the realistic e-commerce context necessary for meaningful evaluation.

Feature Area	**Classification**	**Rationale**
Visual Search (CBIR)	Core Research	Primary contribution: integrated multi-model CBIR with pluggable architecture enabling image-based product search across multiple embedding models (CNN, ViT, CLIP-based) within a production-style e-commerce platform.
ML Embedding Pipeline	Core Research	Critical infrastructure: automated ingestion of product images, generation of vector embeddings via the Python sidecar, storage in pgvector, and HNSW indexing. The operational backbone of the visual search capability.
Model Benchmark System	Core Research	Secondary contribution: systematic protocol for comparing retrieval accuracy and operational efficiency across 11 embedding models, providing a practical guide for model selection in resource-constrained deployments.
Product Catalog	Supporting Infrastructure	Required context: provides the structured dataset of fashion products, with variants, images, taxonomies, and metadata, that serves as the search target for CBIR evaluation.
Order System	Supporting Infrastructure	Metric validation: provides conversion events (add-to-cart, checkout completion) that serve as proxy indicators of search success, enabling evaluation of visual search within a realistic shopping workflow.
Inventory	Supporting Infrastructure	Realism constraint: ensures that search results reflect actual product availability, preventing the unrealistic scenario where visually similar but out-of-stock items appear in search results.
Authentication	Supporting Infrastructure	Security baseline: protects administrative functions and user-specific data, enabling the application to operate in a representative security posture without which the system would be a research prototype rather than a deployable platform.

The classification makes explicit what the thesis does and does not claim as contribution. The CBIR pipeline, encompassing embedding generation, vector storage, and similarity search, is the core research artefact. The e-commerce modules (Catalog, Ordering, Inventory, Payment, Identity) are supporting infra-

structure, built to provide a realistic context that validates the visual search results in a production-like environment. This separation is maintained throughout the chapter: Sections 2.3 and 2.4 devote detailed treatment to the research features, while the supporting infrastructure is described only to the extent necessary to understand the system's design.

2.2 FUNCTIONAL DECOMPOSITION AND USE CASES

The system requirements are presented through a functional decomposition diagram, a use case overview diagram, a use case summary matrix, and detailed scenario specifications for selected core use cases. This structure improves readability while preserving traceability between functional requirements, design decisions, and test cases.

2.2.1 Functional Decomposition

The functional decomposition of ReSys.Shop, shown in Figure 6, organises the platform into three top-level functional areas: Administration, Storefront, and Background Services. Each functional area decomposes into constituent modules, reflecting the responsibilities defined in the use case summary matrix (Section 2.2.3).

The Administration area encompasses seven modules accessible to the Administrator actor: Catalog Management, Order Management, Payment Management, Inventory Management, Identity Management, Shipping Management, and Location Management. The Storefront area covers Customer-facing functionality across three modules: Product Discovery (browse, search, visual search), Purchase Flow (cart, checkout, payment processing, order history), and Account Management (authentication, session management, profile management). Background Services include five automated processes performed by the System actor: Embedding Generation, Cart Expiry, Inventory Reservation, Payment Webhook Processing, and Index Optimisation.

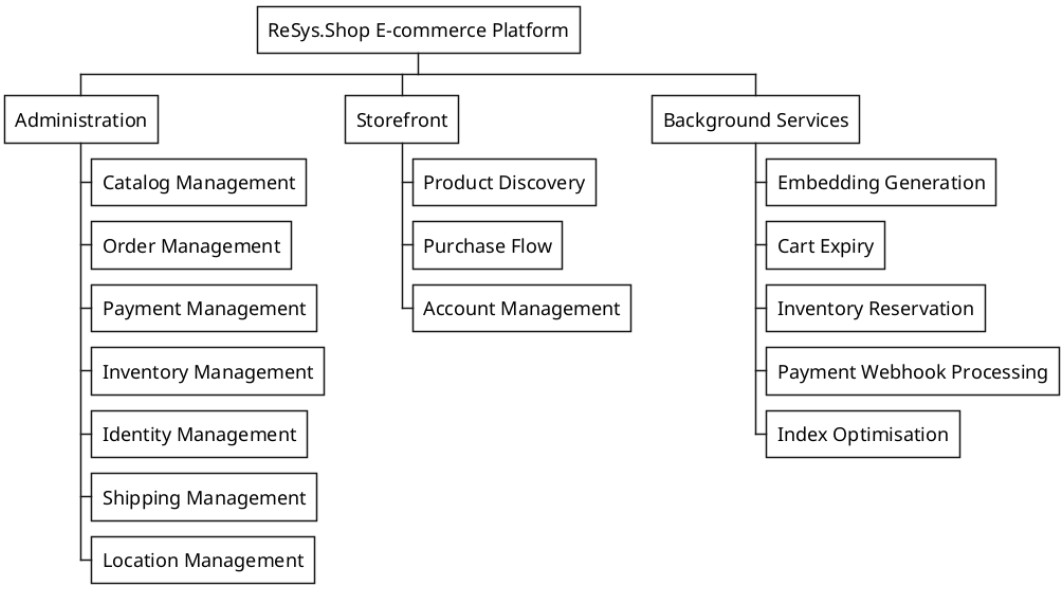

Figure 6: Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions.

2.2.2 Use Case Overview

Figure 7 presents the system use case diagram, showing all 26 use cases from the summary matrix grouped by business module. Each package corresponds to a module from the matrix, containing the use cases accessible to the connected actors. The Administrator interacts with seven modules (Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location), the Customer interacts with five modules (Catalog, Ordering, Payment, Identity, Profile), and the System performs background operations through the System Services module.

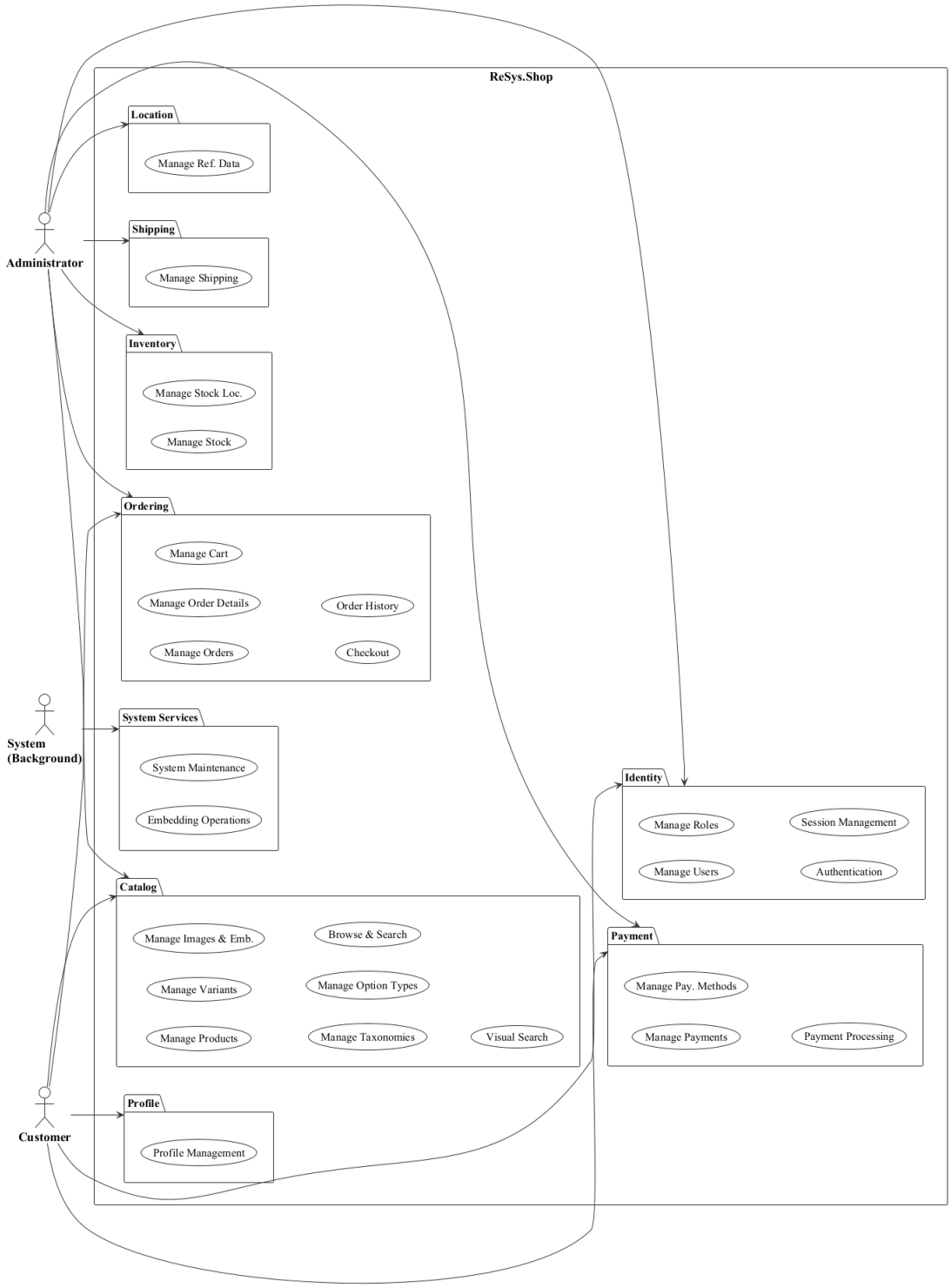

Figure 7: System use case diagram showing all 26 use cases from the summary matrix, grouped by business module with actor-to-module associations.

2.2.3 Use Case Summary Matrix

Table 24 consolidates all use cases across the three actors and provides traceability to the functional requirements defined in Section 2.1.

Table 24: Use case summary matrix. All use cases are listed with their actor, associated business module, and traceability to functional requirements defined in Section 2.1.

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-PROD	Manage products	Admin	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13
UC-ADM-VAR	Manage variants	Admin	Catalog	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22
UC-ADM-IMG	Manage images and embeddings	Admin	Catalog	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15
UC-ADM-TAX	Manage taxonomies and classification	Admin	Catalog	CAT-FR-09, CAT-FR-18, CAT-FR-19
UC-ADM-OPT	Manage option types	Admin	Catalog	CAT-FR-10, CAT-FR-20
UC-ADM-ORD	Manage orders	Admin	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13
UC-ADM-ORD-ITEMS	Manage order details	Admin	Ordering	ORD-FR-13
UC-ADM-PAY	Manage payments	Admin	Payment	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09
UC-ADM-PAY-METHOD	Manage payment methods	Admin	Payment	PAY-FR-10
UC-ADM-STK	Manage stock	Admin	Inventory	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-LOC	Manage stock locations	Admin	Inventory	INV-FR-01
UC-ADM-USR	Manage users	Admin	Identity	IDN-FR-09, IDN-FR-13
UC-ADM-ROL	Manage roles and permissions	Admin	Identity	IDN-FR-11, IDN-FR-12
UC-ADM-SHP	Manage shipping	Admin	Shipping	SHP-FR-01, SHP-FR-04, SHP-FR-05
UC-ADM-REF	Manage reference data	Admin	Location	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04
UC-STR-BRW	Browse and search catalog	Customer	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09, CAT-FR-16, CAT-FR-22
UC-STR-SRC	Visual search	Customer	Catalog	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17
UC-STR-CRT	Manage cart	Customer	Ordering	ORD-FR-01, ORD-FR-02, ORD-FR-10
UC-STR-CHK	Checkout	Customer	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12
UC-STR-OHI	Order history	Customer	Ordering	ORD-FR-07, ORD-FR-14
UC-STR-PAY	Payment processing	Customer	Payment	PAY-FR-01, PAY-FR-02
UC-STR-AUT	Authentication	Customer	Identity	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14
UC-STR-SES	Session management	Customer	Identity	IDN-FR-04, IDN-FR-05, IDN-FR-16
UC-STR-PRF	Profile management	Customer	Profile	PRF-FR-01, PRF-FR-02, PRF-FR-03

UC-ID	Use Case	Actor	Module	Related FR
UC-SYS-EMB	Embedding operations	System	Embedding	CAT-FR-05, CAT-FR-15
UC-SYS-MNT	System maintenance	System	Infrastructure	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

The following sections present detailed scenario specifications for each use case, organised by actor and business module. Each specification includes the actor's goal, trigger, preconditions, postconditions, a numbered main success scenario, alternative and exception flows, and related functional requirements.

2.2.4 Administrator Use Cases

The Administrator actor manages data and operational workflows across all eight business modules through a dedicated administration interface.

2.2.4.1 Product Management

2.2.4.2 UC-ADM-PROD — Manage Products

Table 25: Manage Products.

Field	Description
Use Case ID	UC-ADM-PROD
Use Case Name	Manage Products
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, and archive products in the catalog.
Trigger	Administrator accesses the product management section of the administration interface.
Preconditions	• Authenticated with catalog management permissions.
Postconditions	• Product catalog reflects the performed operation. • Status transitions logged for audit.
Main Success Scenario	**Create Product** 1. Selects the create product option. 2. System presents the creation form. 3. Enters product name, description, slug, and fashion metadata (style code, season, material, department, gender target). 4. Defines at least one variant with SKU and price as the master variant. 5. Assigns the product to relevant taxons.

	6. Submits. System validates slug uniqueness and persists. Confirms creation with Draft status. , **Update Product** 1. Selects a product from the catalog listing. 2. System displays the edit form with current values. 3. Modifies fields: name, description, slug, fashion metadata, SEO attributes, or status. 4. Submits. System validates, persists, and confirms the update. , **Archive Product** 1. Selects a product and chooses the archive option. 2. System requests confirmation. 3. Confirms. Product status changes to Archived and is hidden from the storefront.
Alternative Flows	A1. Slug not unique (Create/Update): system rejects and prompts for a different slug. A2. No master variant (Create): system rejects and instructs to designate one. A3. Product has active orders (Archive): system warns and requests explicit confirmation.
Exception Flows	E1. System fails to persist: reports failure and retains form data for retry.
Related Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13

2.2.4.3 Variant and Pricing

2.2.4.4 UC-ADM-VAR — Manage Variants

Table 26: Manage Variants.

Field	Description
Use Case ID	UC-ADM-VAR
Use Case Name	Manage Variants
Primary Actor	Administrator
Supporting Actors	None
Goal	Add and manage product variants including option-value configuration and pricing.
Trigger	Administrator opens the variant management section from the product detail page.
Preconditions	• Authenticated with catalog permissions. • Parent product exists.
Postconditions	• Variant created or updated with option assignments and pricing.
Main Success Scenario	**Add Variant** 1. Navigates to product detail and selects add variant.

	2. System presents the variant creation form. 3. Enters variant details: SKU, barcode, dimensions, weight, and position. 4. Assigns option values (e.g. Size M, Colour Red) from available option types. 5. Submits. System validates SKU uniqueness and option combination. Creates the variant and confirms. , **Configure Options** 1. Selects a variant from the product detail page. 2. System displays variant detail with current option assignments. 3. Selects an option type and chooses a value; repeats for additional types. 4. Saves. System validates the combination and persists. Confirms the change. , **Configure Pricing** 1. Navigates to pricing management for a product. 2. System displays current prices for all variants grouped by currency. 3. Selects variants and specifies new price with currency and optional validity dates. 4. Submits. System validates non-negative prices and valid currency. Persists and confirms.
Alternative Flows	A1. SKU not unique: system rejects and prompts for a different SKU. A2. Duplicate option combination: system rejects and highlights the conflict. A3. Zero price: system accepts but warns variant appears as free. A4. Overlapping date ranges for same variant and currency: system rejects.
Exception Flows	E1. Persistence failure: system reports and retains form data for retry.
Related Requirements	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

2.2.4.5 Image and Embedding Management

2.2.4.6 UC-ADM-IMG — Manage Images and Embeddings

Table 27: Manage Images and Embeddings.

Field	Description
Use Case ID	UC-ADM-IMG
Use Case Name	Manage Images and Embeddings
Primary Actor	Administrator
Supporting Actors	ML Service
Goal	Upload variant images and manage embedding generation.

Trigger	Administrator navigates to image management for a product variant.
Preconditions	• Authenticated with image management permissions. • Variant exists.
Postconditions	• Image stored and associated with variant. Embeddings available for visual search.
Main Success Scenario	**Upload Images** 1. Selects a product variant and initiates image upload. 2. Selects an image file from the local file system. 3. System validates format (JPEG, PNG, WebP) and size (max 10 MB). 4. Provides alt text and display order position. 5. Confirms and submits. System stores the image, generates thumbnails, and schedules embedding generation. Confirms upload. , **Regenerate Embeddings** 1. Navigates to image management and selects images. 2. Initiates the regenerate embeddings action. 3. System displays confirmation with affected image count. 4. Confirms. System sends each image to the ML service for embedding generation. Stores embeddings with model metadata. Reports completion with success count.
Alternative Flows	A1. Unsupported format: system rejects and lists accepted formats. A2. Exceeds max size: system rejects and displays size constraint. A3. No images selected for regeneration: system disables the action and prompts to select.
Exception Flows	E1. ML service unavailable: system reports failure and suggests retry when operational. E2. Processing cannot be scheduled: system stores image and notifies search will exclude it until processing succeeds.
Related Requirements	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15

2.2.4.7 Taxonomy and Classification

2.2.4.8 UC-ADM-TAX — Manage Taxonomies and Classification

Table 28: Manage Taxonomies and Classification.

Field	Description
Use Case ID	UC-ADM-TAX
Use Case Name	Manage Taxonomies and Classification
Primary Actor	Administrator
Supporting Actors	None

Goal	Create and modify taxonomy structures and assign product classifications.
Trigger	Administrator navigates to taxonomy management or product classification panel.
Preconditions	• Authenticated with taxonomy management permissions.
Postconditions	• Taxonomy structure and product classifications updated.
Main Success Scenario	**Manage Taxonomies** 1. Navigates to taxonomy management. 2. System displays the taxonomy tree with existing taxons. 3. Creates a new taxonomy root or selects an existing taxonomy. 4. Adds, edits, reorders, or removes taxon nodes; optionally defines business rules. 5. Saves. System persists the updated taxonomy and confirms. , **Classify Products** 1. Selects products from the catalog listing. 2. Opens the classification panel. 3. System displays taxonomy tree with current classifications. 4. Selects taxons to assign or deselects to remove. 5. Saves. System validates each taxon path, persists associations, and confirms.
Alternative Flows	A1. Delete taxon with children: system prompts to cascade-delete or reassign. A2. Delete taxon with products: system warns products lose classification. A3. All taxons removed from product: system warns product will not appear in category browsing.
Exception Flows	E1. Concurrent modification: system refreshes data and asks to retry.
Related Requirements	CAT-FR-09, CAT-FR-18, CAT-FR-19

2.2.4.9 Option Type Configuration

2.2.4.10 UC-ADM-OPT — Manage Option Types

Table 29: Manage Option Types.

Field	Description
Use Case ID	UC-ADM-OPT
Use Case Name	Manage Option Types
Primary Actor	Administrator
Supporting Actors	None

Goal	Create, modify, and remove option types and their associated values.
Trigger	Administrator navigates to option type management.
Preconditions	• Authenticated with option type management permissions.
Postconditions	• Option types updated and available for product configuration.
Main Success Scenario	1. Navigates to option type management. 2. System displays all option types with their values. 3. Creates a new option type with name and presentation style. 4. Adds ordered option values (e.g. S, M, L, XL for Size). 5. Optionally edits, reorders, or removes existing types and values. 6. Saves. System validates name uniqueness and non-empty values. Persists and confirms.
Alternative Flows	A1. Delete type in use by products: system warns and asks for confirmation. A2. Remove value in use by variants: system warns and asks for confirmation.
Exception Flows	E1. Concurrent modification: system refreshes data and asks to retry.
Related Requirements	CAT-FR-10, CAT-FR-20

2.2.4.11 Order Lifecycle

2.2.4.12 UC-ADM-ORD — Manage Orders

Table 30: Manage Orders.

Field	Description
Use Case ID	UC-ADM-ORD
Use Case Name	Manage Orders
Primary Actor	Administrator
Supporting Actors	Payment Gateway
Goal	View, modify, and manage the lifecycle of customer orders.
Trigger	Administrator navigates to order management.
Preconditions	• Authenticated with order management permissions.
Postconditions	• Order state transitions performed. Status logged for audit.
Main Success Scenario	**View Orders** 1. Navigates to order management. 2. System displays order list sorted by most recent. 3. Applies optional filters: status, date range, customer email.

	4. Selects an order to view detail with line items, pricing, payment, shipment, addresses, and timeline. , **Update Order** 1. Opens an order and initiates edit. 2. System presents editable form with current values. 3. Modifies line items, addresses, or shipping method. 4. Submits. System validates modifications, recalculates totals, persists, and confirms. , **Approve Order** 1. Opens a pending order and verifies payment capture and inventory availability. 2. Selects approve. System validates and transitions order to approved. Confirms. , **Complete Order** 1. Opens an order in fulfilment state. 2. Verifies shipment dispatched. 3. Selects complete. System displays confirmation. 4. Confirms. System decrements on-hand inventory, transitions to completed, and logs. Confirms. , **Cancel Order** 1. Opens an order and selects cancel. 2. System displays confirmation with consequences summary. 3. Provides cancellation reason and confirms. 4. System releases reserved inventory, voids or refunds payment, transitions to cancelled. Confirms. , **Resume Order** 1. Locates a paused or stalled order. 2. Resolves the underlying issue. 3. Selects resume. System validates prerequisites, transitions back to pending. Confirms.
Alternative Flows	A1. No orders match: system displays empty message with suggestion to broaden filters. A2. Payment not captured (Approve): system prevents and suggests capturing first. A3. Payment gateway unreachable (Cancel): system cancels order, releases inventory, queues payment action. A4. Prerequisites not met (Resume): system prevents and displays issues to resolve. A5. Concurrent state change: system refreshes and notifies.
Exception Flows	E1. Order became immutable concurrently: system refreshes and notifies order is no longer editable. E2. Payment gateway state mismatch: system prevents and advises verifying with gateway. E3. Inventory decrement data conflict: system reports and suggests verifying stock levels.
Related Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13

2.2.4.13 UC-ADM-ORD-ITEMS — Manage Order Details

Table 31: Manage Order Details.

Field	Description
Use Case ID	UC-ADM-ORD-ITEMS
Use Case Name	Manage Order Details
Primary Actor	Administrator
Supporting Actors	None
Goal	Manage line items, shipping address, and billing address on existing orders.
Trigger	Administrator opens the order edit form from the order detail view.
Preconditions	• Authenticated with order update permissions. • Order is in a mutable state.
Postconditions	• Order details updated with recalculated totals. Change logged.
Main Success Scenario	1. Opens an order from the listing. 2. System displays order detail. 3. Initiates the edit action. 4. System presents the editable form with current values. 5. Modifies line items (add, update, remove), shipping address, or billing address. 6. Submits. System validates modifications (stock, address completeness), recalculates totals, persists, and confirms.
Alternative Flows	A1. Quantity exceeds stock: system rejects and shows max available. A2. All line items removed: system rejects and warns at least one is required. A3. Address incompatible with shipping method: system warns and prompts method change.
Exception Flows	E1. Order became immutable concurrently: system refreshes and notifies order is no longer editable.
Related Requirements	ORD-FR-13

2.2.4.14 Payment Processing

2.2.4.15 UC-ADM-PAY — Manage Payments

Table 32: Manage Payments.

Field	Description
Use Case ID	UC-ADM-PAY
Use Case Name	Manage Payments
Primary Actor	Administrator

Supporting Actors	Payment Gateway
Goal	Capture, refund, void, and review payments.
Trigger	Administrator navigates to payment management.
Preconditions	• Authenticated with payment management permissions.
Postconditions	• Payment state updated. Gateway operations recorded.
Main Success Scenario	**Capture Payment** 1. Opens payment detail view for an order. 2. System displays payment state, authorised amount, and capture eligibility. 3. Optionally adjusts capture amount (partial) not exceeding authorised amount. 4. Confirms. System sends capture request to gateway with idempotency key, updates state to captured, and confirms. , **Refund Payment** 1. Opens payment detail view for a captured payment. 2. Enters refund amount (full or partial) and reason. 3. Confirms. System validates amount, sends refund to gateway, updates state, and confirms. , **Void Payment** 1. Opens payment detail view for an authorised but uncaptured payment. 2. Selects void action and confirms. 3. System sends void request to gateway, updates state to voided, and confirms. , **View Payments** 1. Navigates to payment management. 2. System displays payment list sorted by most recent. 3. Applies optional filters: state, date range, gateway, order number. 4. Selects a payment to view full detail: amount, currency, gateway, state timeline, order reference, capture and refund history.
Alternative Flows	A1. Partial capture/refund: amount less than authorised/captured; remainder available. A2. Already captured (Void): system prevents and suggests refund instead. A3. State mismatch (View): system highlights discrepancy and suggests syncing.
Exception Flows	E1. Gateway rejects: system reports rejection reason. E2. Gateway unreachable: system reports failure; idempotency key ensures safe retry.
Related Requirements	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09

2.2.4.16 Payment Method Configuration

2.2.4.17 UC-ADM-PAY-METHOD — Manage Payment Methods

Table 33: Manage Payment Methods.

Field	Description
Use Case ID	UC-ADM-PAY-METHOD
Use Case Name	Manage Payment Methods
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, activate, and deactivate payment methods.
Trigger	Administrator navigates to payment method configuration.
Preconditions	• Authenticated with payment method management permissions.
Postconditions	• Payment methods updated and available for storefront selection.
Main Success Scenario	1. Navigates to payment method configuration. 2. System displays configured payment methods with activation status. 3. Creates a new method with name, description, gateway identifier, and parameters. 4. Optionally edits, activates, deactivates, or removes methods. 5. Saves. System validates name uniqueness and gateway support. Persists and confirms.
Alternative Flows	A1. Deactivate method in use: system warns new orders cannot use it; existing unaffected. A2. Remove method: system verifies no pending payments reference it. A3. Invalid gateway parameters: system rejects and highlights invalid ones.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	PAY-FR-10

2.2.4.18 Stock Location Management

2.2.4.19 UC-ADM-LOC — Manage Stock Locations

Table 34: Manage Stock Locations.

Field	Description
Use Case ID	UC-ADM-LOC
Use Case Name	Manage Stock Locations

Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, and remove stock locations.
Trigger	Administrator navigates to stock location management.
Preconditions	• Authenticated with location management permissions.
Postconditions	• Location configuration updated.
Main Success Scenario	1. Navigates to stock location management. 2. System displays existing stock locations with addresses and active status. 3. Creates a new location with name, address, and active status flag. 4. Optionally designates the new location as default for stock intake. 5. Optionally edits, deactivates, or removes existing locations. 6. Saves. System validates location name uniqueness. Persists and confirms.
Alternative Flows	A1. Delete location with active stock: system prevents and requires transfer first. A2. Deactivate location: system prevents new intake but allows existing movements. A3. Delete last location: system prevents; at least one must remain.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	INV-FR-01

2.2.4.20 Stock Item Management

2.2.4.21 UC-ADM-STK — Manage Stock

Table 35: Manage Stock.

Field	Description
Use Case ID	UC-ADM-STK
Use Case Name	Manage Stock
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, restock, transfer, and monitor stock levels.
Trigger	Administrator navigates to stock item management.
Preconditions	• Authenticated with stock management permissions.
Postconditions	• Stock quantities updated. Changes logged for audit.
Main Success Scenario	**Manage Stock Items** 1. Navigates to stock item management.

	2. System displays stock items with variant, location, on-hand, and reserved quantities. 3. Creates a stock item by selecting variant and location, enters initial on-hand quantity. 4. Alternatively selects existing stock item and updates on-hand quantity. 5. Provides a reason for the adjustment. 6. Saves. System validates variant-location uniqueness, persists, and records audit log. Confirms. , **Restock Inventory** 1. Locates a stock item in the management interface. 2. Enters the restock quantity and provides a reference and notes. 3. Confirms. System increments on-hand quantity and records the restock event. Confirms updated quantities. , **Transfer Stock** 1. Navigates to stock transfer and initiates a new transfer. 2. Selects source location, destination, variant, and quantity. 3. System validates sufficient stock at source. 4. Submits. System creates transfer record pending, decrements source. 5. Upon arrival, confirms receipt. System increments destination and transitions to completed. Confirms. , **Review Stock Movements** 1. Navigates to stock movement audit interface. 2. System displays all stock movements in reverse chronological order with pagination. 3. Applies optional filters: date range, variant, location, movement type. 4. Selects a movement to view full detail. , **Monitor Low Stock** 1. Navigates to low stock monitoring view. 2. System displays stock items below configured threshold with variant, location, on-hand, threshold, and days since last restock. 3. Reviews list and identifies items needing replenishment.
Alternative Flows	A1. Bulk adjustment via file upload: system processes, validates, reports success/failure counts. A2. Reduce below reserved quantity: system rejects and shows current reserved. A3. Transfer exceeds available stock: system rejects and shows maximum. A4. Cancel pending transfer: system returns deducted quantity to source and logs cancellation. A5. No items below threshold (Low Stock): system displays message that all stock levels are sufficient.
Exception Flows	E1. Variant-location pair already exists: system rejects and suggests editing existing. E2. Concurrent modification: system refreshes and asks to re-enter. E3. Retrieval failure: system displays error and offers retry.

Related Requirements	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12

2.2.4.22 User Management

2.2.4.23 UC-ADM-USR — Manage Users

Table 36: Manage Users.

Field	Description
Use Case ID	UC-ADM-USR
Use Case Name	Manage Users
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, enable, and disable user accounts.
Trigger	Administrator navigates to user management.
Preconditions	• Authenticated with user management permissions.
Postconditions	• User account created or modified. Status changes take effect on next authentication.
Main Success Scenario	1. Navigates to user management. 2. System displays user list with default sorting and pagination. 3. Applies optional filters: email, name, role, account status. 4. Creates a new user account by entering email, name, and assigning roles. 5. Alternatively selects an existing user to view detail or edit. 6. Modifies profile fields, enables/disables account, or adjusts role assignments. 7. Saves. System validates email uniqueness, persists, and confirms. If account was disabled, all active sessions are revoked.
Alternative Flows	A1. Disable account: system revokes all active sessions immediately. A2. Re-enable disabled account: system restores active status. A3. Email already registered: system rejects and prompts for a different email.
Exception Flows	E1. Persistence failure: system reports and retains form data for retry.
Related Requirements	IDN-FR-09, IDN-FR-13

2.2.4.24 Role and Permission Governance

2.2.4.25 UC-ADM-ROL — Manage Roles and Permissions

Table 37: Manage Roles and Permissions.

Field	Description
Use Case ID	UC-ADM-ROL
Use Case Name	Manage Roles and Permissions
Primary Actor	Administrator
Supporting Actors	None
Goal	Create and manage roles, assign permissions to roles, and grant roles to users.
Trigger	Administrator navigates to role management or user detail page.
Preconditions	• Authenticated with role and permission management rights.
Postconditions	• Role and permission configuration updated. Affected users receive updated permissions on next token.
Main Success Scenario	**Manage Roles** 1. Navigates to role management. 2. System displays list of roles with name, description, and permission count. 3. Creates a new role with name and description. 4. Assigns permissions from the permissions catalogue. 5. Optionally edits an existing role's name, description, or permission set. 6. Saves. System validates role name uniqueness, persists, and confirms. , **Assign User Roles** 1. Opens a user's detail page. 2. Opens the role assignment panel. 3. System displays all available roles with checkboxes for current assignments. 4. Selects roles to assign and deselects to revoke. 5. Saves. System persists and displays updated effective permissions. , **Grant Direct Permissions** 1. Opens a user's detail page. 2. Opens the direct permission panel. 3. System displays permissions catalogue with role-inherited and direct-grant indicators. 4. Selects permissions to grant directly and deselects to revoke. 5. Saves. System persists and recalculates effective permissions. , **View Permissions Catalogue** 1. Navigates to the permissions catalogue.

	2. System displays all permission claims grouped by domain and module. 3. Applies optional filters: domain, category, keyword search. 4. Reviews the permission matrix to plan role designs or audit assignments.
Alternative Flows	A1. Remove role assigned to users: system warns affected users lose permissions. A2. Revoke permission from role: system warns all users with this role lose it on next token. A3. Revoke last role from user: system warns user loses all role-derived permissions. A4. Grant already inherited from role: system accepts; effective permission unchanged but direct grant recorded.
Exception Flows	E1. Concurrent modification or role deleted: system refreshes and asks to retry.
Related Requirements	IDN-FR-11, IDN-FR-12

2.2.4.26 Shipping Method Configuration

2.2.4.27 UC-ADM-SHP — Manage Shipping

Table 38: Manage Shipping.

Field	Description
Use Case ID	UC-ADM-SHP
Use Case Name	Manage Shipping
Primary Actor	Administrator
Supporting Actors	None
Goal	Configure delivery methods and their associated shipping rates.
Trigger	Administrator navigates to shipping configuration.
Preconditions	• Authenticated with shipping management permissions.
Postconditions	• Shipping methods and rates configured. Active methods available for checkout if zone-applicable.
Main Success Scenario	**Manage Shipping Methods** 1. Navigates to shipping method management. 2. System displays configured shipping methods with activation status. 3. Creates a new method with name, description, carrier identifier, and applicable zones. 4. Optionally edits, activates, deactivates, or removes existing methods. 5. Saves. System validates method name uniqueness, persists, and confirms. , **Manage Shipping Rates** 1. Selects a shipping method from the method listing.

	2. System displays current rate tiers for the method. 3. Creates a new rate tier: selects zone, defines weight and cart value ranges, enters rate amount. 4. Optionally edits or removes existing rate tiers. 5. Saves. System validates rate tier ranges do not overlap for the same zone, persists, and confirms.
Alternative Flows	A1. Deactivate method in active checkouts: system warns; existing selections unaffected. A2. Remove method with active rates: system prompts to remove rates or reassign. A3. No zones assigned: system warns method will not be selectable at checkout. A4. Ranges overlap with existing tiers: system rejects and highlights conflicting tiers.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	SHP-FR-01, SHP-FR-04, SHP-FR-05

2.2.4.28 Reference Data Management

2.2.4.29 UC-ADM-REF — Manage Reference Data

Table 39: Manage Reference Data.

Field	Description
Use Case ID	UC-ADM-REF
Use Case Name	Manage Reference Data
Primary Actor	Administrator
Supporting Actors	None
Goal	Create and update country and state reference data.
Trigger	Administrator navigates to reference data management.
Preconditions	• Authenticated with reference data management permissions.
Postconditions	• Country and state data updated. Active records available in address forms and shipping zones.
Main Success Scenario	**Manage Countries** 1. Navigates to country management. 2. System displays list of countries with name, ISO code, and active status. 3. Creates a new country record with display name and ISO 3166-1 code. 4. Sets the active status flag; optionally edits or removes existing country records. 5. Saves. System validates ISO code uniqueness and format, persists, and confirms. , **Manage States** 1. Navigates to state management.

	2. System displays list of states with name, ISO code, parent country, and active status. 3. Creates a new state record with display name, ISO 3166-2 code, and selects parent country. 4. Sets the active status flag; optionally edits or removes existing state records. 5. Saves. System validates ISO code uniqueness within parent country, persists, and confirms.
Alternative Flows	A1. Deactivate country: system warns addresses retained but country not in new forms. A2. Delete country with associated states: system warns states will be orphaned. A3. Delete country in shipping zones: system warns and suggests deactivating instead. A4. Deactivate parent country: system prompts whether to cascade-deactivate associated states.
Exception Flows	E1. ISO code already exists: system rejects and prompts for different code.
Related Requirements	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04

2.2.5 Customer Use Cases

The Customer actor interacts through the browser-based storefront for product discovery, purchase, and account management.

2.2.5.1 Catalog Browsing

2.2.5.2 UC-STR-BRW — Browse and Search Catalog

Table 40: Browse and Search Catalog.

Field	Description
Use Case ID	UC-STR-BRW
Use Case Name	Browse and Search Catalog
Primary Actor	Customer
Supporting Actors	None
Goal	Browse the product catalog, view product details, and search by keyword.
Trigger	Customer visits the storefront and selects a category, product, or enters a search term.
Preconditions	• Catalog data is published and searchable.
Postconditions	• Product listing or detail displayed with pricing and availability.
Main Success Scenario	**Browse Catalog** 1. Visits the storefront home page or a category landing page.

	2. System displays the taxonomy tree with top-level categories. 3. Selects a category or subcategory from the taxonomy navigation. 4. System retrieves products associated with the selected taxon and its descendants. 5. System displays a paginated grid of product cards showing thumbnail, name, price, and availability. 6. Applies optional facets to filter by option types (e.g. size, colour) and other attributes. 7. System refreshes the listing with filtered results. , **View Product Detail** 1. Clicks on a product card from a catalog listing, search result, or recommendation. 2. System displays product detail page with primary image, name, description, and price range. 3. System shows fashion metadata: style code, season, material, department, gender target. 4. System displays taxonomy breadcrumb showing category path. 5. Browses product images in the gallery. 6. Selects variant option values from available options. 7. System updates displayed price, availability, and variant-specific images based on selection. , **Keyword Search** 1. Enters a search keyword or phrase in the storefront search bar. 2. System queries the catalog for products matching name, description, or fashion attributes. 3. System ranks results by relevance and displays a paginated grid of matching product cards. 4. Refines search with additional keywords or applies filters.
Alternative Flows	A1. No products in category: system displays empty state suggesting other categories. A2. Selected variant out of stock: system shows out-of-stock status and disables add-to-cart. A3. No products match keyword: system displays suggestions to check spelling or browse categories. A4. Very short query (single character): system prompts to enter at least two characters.
Exception Flows	E1. Retrieval or search failure: system displays error and offers retry.
Related Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09, CAT-FR-16, CAT-FR-22

2.2.5.3 Search

2.2.5.4 UC-STR-SRC — Visual Search

Table 41: Visual Search.

Field	Description
Use Case ID	UC-STR-SRC
Use Case Name	Visual Search
Primary Actor	Customer
Supporting Actors	ML Service
Goal	Search for products by uploading a reference image and discover visually similar items.
Trigger	Customer navigates to visual search or views a product detail page.
Preconditions	• Catalog has products with embeddings. • ML service is operational.
Postconditions	• Visually similar products displayed with similarity scores above configured threshold.
Main Success Scenario	**Search by Image (CBIR)** 1. Navigates to the visual search interface. 2. Selects or drags an image file from the local file system. 3. System validates image format (JPEG, PNG, WebP) and size constraints. 4. System sends the image to the ML service for embedding generation. 5. System performs similarity search against the image embeddings index. 6. System retrieves matching products ranked by similarity score above minimum threshold. 7. System displays results as a grid of product thumbnails with scores, linking to detail pages. , **View Similar Products** 1. Opens a product detail page with image embeddings available. 2. System retrieves the product's primary image embedding. 3. System performs similarity search against other product embeddings. 4. System displays a horizontal carousel or grid of visually similar product cards. 5. Clicks on a similar product card to navigate to its detail page.
Alternative Flows	A1. Invalid image format: system rejects and lists accepted formats. A2. No products above threshold: system displays empty result suggesting different image or keyword search. A3. No images or embeddings on detail page: system hides the similar products section. A4. ML service unavailable: system gracefully hides section without affecting detail page.
Exception Flows	E1. ML service unavailable: system displays error and suggests retrying later. E2. Embedding index empty: system reports visual search not yet available.

Related Requirements	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17

2.2.5.5 Cart Management

2.2.5.6 UC-STR-CRT — Manage Cart

Table 42: Manage Cart.

Field	Description
Use Case ID	UC-STR-CRT
Use Case Name	Manage Cart
Primary Actor	Customer
Supporting Actors	None
Goal	Add, update, and remove items in a shopping cart; associate guest cart with account.
Trigger	Customer selects a variant on the product detail page and adds it to the cart.
Preconditions	• Product variant exists and is available.
Postconditions	• Cart persisted across page navigation. Guest carts survive browser sessions.
Main Success Scenario	**Manage Cart Items** 1. On a product detail page, selects a variant and specifies quantity. 2. Clicks Add to Cart. System validates the variant and quantity. 3. System adds the variant to the customer's cart and displays confirmation. 4. Opens the cart to view all items with quantities, prices, and subtotal. 5. Updates quantity of an item or removes an item. 6. System recalculates cart subtotal and updates the display. , **Associate Cart with Account** 1. Browses storefront as guest and adds items to cart. 2. Logs in or registers for an account. 3. System detects guest cart exists and retrieves any existing user cart. 4. System merges guest and user carts: matching variants increase quantity; unique variants are added. 5. System associates the merged cart with the user account and invalidates guest cart cookie.
Alternative Flows	A1. Quantity exceeds stock: system rejects and shows max available. A2. Same variant already in cart: system increments existing quantity. A3. Guest customer: system assigns session-based cart identifier in signed cookie. A4. No existing user cart: system transfers guest cart to

	user account directly. A5. Merge exceeds available stock: system caps at max available and notifies customer.
Exception Flows	E1. Variant deactivated or archived: system rejects and suggests refreshing product page. E2. Merge fails due to data conflict: system creates user cart with guest items and notifies to review.
Related Requirements	ORD-FR-01, ORD-FR-02, ORD-FR-10

2.2.5.7 Checkout Flow

2.2.5.8 UC-STR-CHK — Checkout

Table 43: Checkout.

Field	Description
Use Case ID	UC-STR-CHK
Use Case Name	Checkout
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	Complete the multi-step checkout: address selection, shipping method, and order confirmation.
Trigger	Customer proceeds from cart to checkout.
Preconditions	• Customer is authenticated. • Cart is not empty. • Stock is available for all items.
Postconditions	• Order created with unique number. Inventory reserved. Payment linked. Cart cleared.
Main Success Scenario	**Select Shipping Address** 1. From the cart, clicks Proceed to Checkout. 2. System transitions checkout to the Address step. 3. System displays saved addresses with the default pre-selected. 4. Selects an existing shipping address or enters a new one. 5. System determines shipping zone and proceeds to the next step. , **Select Shipping Method** 1. System calculates available shipping methods and rates based on address zone, cart weight, and cart value. 2. System displays list of available methods with rates and estimated delivery times. 3. Reviews options and selects a preferred shipping method. 4. System applies the selected method and rate; updates shipment total in order summary.

	5. System presents next checkout step (payment). , **Complete Checkout** 1. System displays order summary with line items, shipping, tax, and total. 2. Enters payment details or selects a saved payment method. 3. System creates a payment intent and reserves inventory for each line item. 4. Reviews final order summary and confirms the purchase. 5. System captures payment and generates a unique order number. 6. System transitions order to Confirmed state, clears cart, and displays order confirmation. 7. System sends order confirmation notification.
Alternative Flows	A1. No saved addresses: system presents empty address step with prompt to create new. A2. No methods available for zone: system displays message and prompts different address. A3. Only one method available: system auto-selects and proceeds. A4. Stock depleted during checkout: system notifies, removes affected items, returns to cart. A5. Payment failure: system notifies with reason; allows retry; inventory not reserved.
Exception Flows	E1. Address validation fails: system highlights missing fields and prevents progression. E2. Rate calculation fails: system displays error and suggests contacting support. E3. Payment captured but inventory reservation fails: system voids payment and notifies order not completed.
Related Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12

2.2.5.9 Order History

2.2.5.10 UC-STR-OHI — Order History

Table 44: Order History.

Field	**Description**
Use Case ID	UC-STR-OHI
Use Case Name	Order History
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	View past orders and cancel pending orders.
Trigger	Customer navigates to order history in their account.
Preconditions	• Customer is authenticated.

Postconditions	• Complete order history visible. Cancelled orders release inventory and void payment.
Main Success Scenario	**View Order History** 1. Navigates to order history from account menu. 2. System displays past orders in reverse chronological order with pagination, showing order number, date, status, and total. 3. Applies optional date range or status filters. 4. Selects an order to view full detail: line items, shipping address, method, payment state, shipment state, and status timeline. , **Cancel Order** 1. Opens order detail from order history. 2. System displays order detail with current status and cancel action if cancellable. 3. Selects Cancel Order. 4. System displays confirmation explaining inventory release and payment void. 5. Confirms. System releases reserved inventory, voids payment, transitions to cancelled, and sends confirmation.
Alternative Flows	A1. No orders: system displays message with prompt to browse catalog. A2. Order state changed since page loaded: system refreshes and informs cancellation unavailable.
Exception Flows	E1. Payment gateway unreachable (Cancel): system cancels order, releases inventory, queues void, notifies customer. E2. Retrieval failure (View): system displays error and offers retry.
Related Requirements	ORD-FR-07, ORD-FR-14

2.2.5.11 Payment Processing

2.2.5.12 UC-STR-PAY — Payment Processing

Table 45: Payment Processing.

Field	Description
Use Case ID	UC-STR-PAY
Use Case Name	Payment Processing
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	Create and confirm payment for an order.
Trigger	Customer reaches the payment step during checkout.
Preconditions	• Customer is authenticated. • Checkout has reached payment step.

	• Order has a calculated total.
Postconditions	• Payment authorised. Order transitions to Confirmed state.
Main Success Scenario	**Create Payment Intent** 1. System displays payment step with order total and available payment methods. 2. Selects a payment method and enters payment details. 3. System submits payment details to gateway to create intent. 4. System receives confirmation intent is ready and displays review summary. , **Confirm Payment** 1. Reviews final order summary including line items, shipping, tax, and total. 2. Clicks Confirm Payment. 3. System submits payment intent confirmation to gateway. 4. System receives authorisation confirmation. 5. System updates payment state, transitions order to Confirmed, clears cart, and displays order confirmation.
Alternative Flows	A1. Invalid payment details: system returns gateway validation error, prompts correction. A2. Authorisation declined: system displays reason and offers retry with different method. A3. Additional authentication required: system redirects to authentication flow and resumes after.
Exception Flows	E1. Payment gateway unreachable: system displays error; checkout progress saved. E2. Payment confirms but order creation fails: system voids payment and notifies to retry. E3. Gateway timeout: system marks payment pending; advises checking order history; webhook updates state.
Related Requirements	PAY-FR-01, PAY-FR-02

2.2.5.13 Authentication

2.2.5.14 UC-STR-AUT — Authentication

Table 46: Authentication.

Field	**Description**
Use Case ID	UC-STR-AUT
Use Case Name	Authentication
Primary Actor	Customer
Supporting Actors	Email Service, Google OAuth
Goal	Register, log in, and manage account credentials.
Trigger	Customer navigates to login or registration page.

Preconditions	• None (public access).
Postconditions	• Customer authenticated or credentials updated.
Main Success Scenario	**Register** 1. Navigates to registration page. 2. Enters email, password, and basic profile information. 3. Submits. System validates email not registered and password strength, creates account, sends verification. Confirms. , **Login with Password** 1. Navigates to login page and enters email and password. 2. Submits. System validates credentials, issues access and refresh tokens, associates guest cart if exists. Redirects to home page. , **Login with Google** 1. Clicks Login with Google on the storefront. 2. System redirects to Google's OAuth consent screen. 3. Grants consent. System exchanges code, looks up or creates account, issues tokens. Redirects to home page. , **Reset Password** 1. Clicks Forgot Password and submits registered email. 2. System generates time-limited reset token and sends via email. 3. Opens email, clicks reset link, enters new password. 4. Submits. System validates token, updates password, revokes all sessions. Confirms. , **Change Password** 1. Navigates to account security settings and selects Change Password. 2. Enters current password and new password. 3. Submits. System verifies current password, validates new password, updates, revokes all sessions except current. Confirms.
Alternative Flows	A1. Email already registered: system rejects and suggests login with password reset link. A2. Password does not meet strength requirements: system highlights requirements and prompts retry. A3. Invalid credentials: system rejects with generic message. A4. Account disabled: system rejects with message to contact support. A5. Reset token expired: system displays message and prompts new reset request. A6. Current password incorrect (Change): system rejects and prompts retry.
Exception Flows	E1. Verification or reset email fails to send: system creates account/request but logs failure internally. E2. Token issuance fails: system reports failure and suggests retry.
Related Requirements	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14

2.2.5.15 Session Management

2.2.5.16 UC-STR-SES — Session Management

Table 47: Session Management.

Field	Description
Use Case ID	UC-STR-SES
Use Case Name	Session Management
Primary Actor	Customer
Supporting Actors	None
Goal	Maintain and terminate authenticated sessions.
Trigger	Client detects token expiry or customer initiates logout.
Preconditions	• Customer has an active session.
Postconditions	• Session extended or terminated.
Main Success Scenario	**Refresh Session** 1. Client detects access token will expire soon. 2. Client sends current refresh token to token endpoint. 3. System validates refresh token (not expired, not consumed). 4. System issues new access token with fresh expiry. 5. System issues new refresh token and invalidates previous (token rotation). 6. Client stores new token pair and continues session. , **Logout** 1. Clicks Logout in the storefront navigation. 2. System sends current refresh token for invalidation. 3. System invalidates the refresh token and removes the session cookie. 4. System redirects to the storefront home page.
Alternative Flows	A1. Refresh token expired: system rejects; client redirects to login. A2. Refresh token consumed (reuse detected): system revokes all tokens; client redirects to login with security notification. A3. Logout request fails due to network: system clears local tokens and cookies; session expires naturally.
Exception Flows	E1. Token issuance or invalidation fails: client retains existing pair if access token still valid; clears locally otherwise.
Related Requirements	IDN-FR-04, IDN-FR-05, IDN-FR-16

2.2.5.17 Profile and Preferences

2.2.5.18 UC-STR-PRF — Profile Management

Table 48: Profile Management.

Field	Description
Use Case ID	UC-STR-PRF
Use Case Name	Profile Management
Primary Actor	Customer
Supporting Actors	None
Goal	Manage shipping addresses, wishlists, and notification preferences.
Trigger	Customer navigates to account settings.
Preconditions	• Customer is authenticated.
Postconditions	• Profile data and preferences updated.
Main Success Scenario	**Manage Addresses** 1. Navigates to address book page. 2. System displays saved addresses with type labels and default indicators. 3. Creates a new address: enters name, street, city, postal code, country, and state. 4. Assigns address type (shipping, billing, or both) and optionally sets as default. 5. Optionally edits or removes existing addresses. 6. Saves. System validates required fields and address format, persists, and confirms. , **Manage Wishlists** 1. On a product detail page, clicks Add to Wishlist. 2. System displays dialog to select wishlist or create new. 3. Selects an existing wishlist or creates a new named wishlist; optionally adds a note. 4. System adds the product variant to the selected wishlist and confirms. 5. Navigates to wishlists section to view all with item counts and preview thumbnails. 6. Selects a wishlist to view items, remove items, rename, or delete the list. , **Manage Notification Preferences** 1. Navigates to notification preferences page. 2. System displays all notification categories with current per-channel opt-in/out status. 3. Toggles individual notification categories on or off per channel. 4. Saves. System persists and confirms.
Alternative Flows	A1. Same variant already in wishlist: system notifies and suggests adding note to existing entry. A2. Remove address used in active orders: system warns it is referenced by past orders (retained); allows soft-delete. A3. Opts out of all notifications: system warns important notifications also suppressed; asks confirmation. A4. Deletes wishlist: system asks for confirmation; list and items permanently removed.

Exception Flows	E1. Address validation fails for invalid country/state: system highlights mismatch and prevents save. E2. Product variant archived since added to wishlist: system displays with Unavailable label and remove suggestion. E3. Persistence failure: system reports failure and retains unsaved changes for retry.
Related Requirements	PRF-FR-01, PRF-FR-02, PRF-FR-03

2.2.6 System Use Cases

The System actor represents automated background processes that maintain data consistency, generate embeddings, and perform scheduled operations.

2.2.6.1 Embedding Operations

2.2.6.2 UC-SYS-EMB — Embedding Operations

Table 49: Embedding Operations.

Field	Description
Use Case ID	UC-SYS-EMB
Use Case Name	Embedding Operations
Primary Actor	System
Supporting Actors	ML Service
Goal	Generate and regenerate image embeddings for product search.
Trigger	An image has been uploaded or embedding model configuration has changed.
Preconditions	• Images uploaded and stored. • ML service is operational.
Postconditions	• Embeddings stored in the index for visual search.
Main Success Scenario	**Generate Image Embeddings** 1. System detects a new unprocessed image in the queue. 2. System retrieves the image file from storage. 3. System preprocesses the image (resize, normalise) for model requirements. 4. System sends preprocessed image to ML service with configured model identifier. 5. ML Service computes the visual embedding vector and returns it with metadata. 6. System stores embedding in the index with image reference and metadata. 7. System marks the image as processed. , **Regenerate All Embeddings**

	1. System detects change to embedding model configuration. 2. System validates new model is available via ML service health check. 3. System retrieves list of all product images requiring regeneration. 4. System sends each image to ML service using the new model. 5. ML Service computes new embedding and returns it. 6. System stores new embedding replacing previous and updates model metadata. 7. System reports completion with success/failure count summary.
Alternative Flows	A1. Image not accessible (deleted/corrupted): system marks as failed and records reason. A2. Multiple images queued: system processes sequentially, throttling to ML service capacity. A3. Triggered during peak traffic (Regenerate): system throttles to avoid impacting search performance. A4. No model change detected: system logs event and skips processing.
Exception Flows	E1. ML service returns error: system retries with exponential backoff; after max retries marks as failed. E2. ML service unreachable: system requeues image and triggers alert. E3. New model not available (Regenerate): system aborts regeneration; existing embeddings remain in use.
Related Requirements	CAT-FR-05, CAT-FR-15

2.2.6.3 Background Maintenance

2.2.6.4 UC-SYS-MNT — System Maintenance

Table 50: System Maintenance.

Field	**Description**
Use Case ID	UC-SYS-MNT
Use Case Name	System Maintenance
Primary Actor	System
Supporting Actors	ML Service, Payment Gateway
Goal	Perform scheduled and event-driven system maintenance tasks.
Trigger	Maintenance schedule fires or event-driven trigger occurs.
Preconditions	• Maintenance schedule is active. • Required services are operational.
Postconditions	• System health, data consistency, and index performance maintained.
Main Success Scenario	**Monitor Service Health**

	1. System invokes health check endpoint on ML service at configured interval. 2. ML Service responds with current health status: healthy, degraded, or unhealthy. 3. System records health status and response time. 4. System updates service availability flag. If unhealthy, routes search to degraded path. , **Expire Abandoned Carts** 1. System executes scheduled job at configured time. 2. System queries for carts with no modification in past seven days. 3. For each abandoned cart, releases reserved inventory back to availability. 4. System deletes or marks abandoned cart for cleanup and logs counts. , **Release Expired Reservations** 1. System executes scheduled job at configured interval. 2. System queries for reservations with hold time exceeding configured expiry window (default 15 min). 3. For each expired reservation, releases reserved quantity back to available stock. 4. System marks reservation record as expired and logs counts. , **Process Payment Webhooks** 1. Payment Gateway sends signed webhook notification to system endpoint. 2. System validates cryptographic signature against configured gateway secret. 3. System extracts payment identifier, event type, and payload. 4. System verifies not a duplicate by checking idempotency key. 5. System updates local payment state and triggers any required order state transitions. 6. System returns success response to gateway. , **Maintain Search Index** 1. System executes scheduled job at configured interval. 2. System analyses current embedding index state: vector count, index size, recent query performance. 3. System determines whether maintenance is likely to improve performance. 4. System performs maintenance (e.g. index rebuild, dead tuple removal, statistics update). 5. System verifies maintenance completed and logs execution with before/after metrics.
Alternative Flows	A1. No carts/items meet criteria: system logs zero and completes. A2. Duplicate webhook: system returns success without changes; duplicate logged. A3. Out-of-order webhook event: system logs anomaly and applies state change. A4. Index below maintenance threshold: system skips and logs decision with current metrics. A5.

	Active search queries during index maintenance: system performs lighter, non-blocking operations.
Exception Flows	E1. Health check endpoint does not respond: system marks unhealthy after timeout and triggers alert. E2. Database unavailable: system records failure and retries on next execution. E3. Webhook signature validation fails: system rejects with authentication error. E4. Index maintenance fails: system logs error and triggers alert; search continues with current index. E5. Index appears corrupted: system triggers full rebuild from stored embeddings.
Related Requirements	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.3 SYSTEM ARCHITECTURE & DESIGN

This chapter presents the architectural design of ReSys.Shop, progressing from a high-level system overview through domain modelling, to the structural, data, API, and security layers. The design follows a service-oriented approach with three independently deployable services, a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar, each responsible for a distinct technological concern. The chapter is organised into six sections, each accompanied by architectural diagrams that provide visual representations of the system's structure, behaviour, and deployment topology.

2.3.1 System Overview

ReSys.Shop is built as a service-oriented system with three distinct services. The frontend is implemented in Vue 3 and TypeScript using the Vite build tool. The backend is a .NET 10 modular monolith using ASP.NET Core for HTTP handling, Entity Framework Core for data access, and Carter for minimal API endpoint registration. The machine learning service is a Python FastAPI application running PyTorch models that generates vector embeddings from product images for visual similarity search.

Table 51 summarises the three services, their technology stacks, and their primary responsibilities within the platform.

Table 51: System services and their technology stacks. Each service is independently deployable and communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (Nuxt UI) • Administrator dashboard (PrimeVue)

Service	Technology Stack	Responsibilities
		• Pinia state management • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API endpoints via Carter minimal APIs • Business logic via MediatR CQRS pattern • PostgreSQL persistence with pgvector vector search • JWT authentication and RBAC authorisation
Python ML	Python 3.12 + FastAPI + PyTorch	• Fashion-CLIP and other embedding model inference • Vector embedding generation from product images • Multi-model support with lazy-loading strategy

The backend is internally organised into eight bounded contexts following the principles of Domain-Driven Design. Each context owns a distinct area of business logic and communicates with other contexts exclusively through MediatR in-process message dispatch, there are no direct namespace references between business modules. Table 52 lists each context, its aggregate root, and a representative sample of its domain entities.

Table 52: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates with other contexts through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone
Location	Country	State

The separation of concerns across these eight contexts enables independent evolution of each business domain while the modular monolith deployment model

avoids the operational complexity of distributed microservices. The following section details the domain-driven design principles that govern these contexts.

2.3.2 Domain-Driven Design

The ReSys.Shop platform applies Domain-Driven Design (DDD) to structure its business logic around eight bounded contexts, each with well-defined aggregate roots, domain entities, and invariants. This section presents the context map, the aggregate design with invariants, the ubiquitous language glossary, and the state machines that govern the checkout and payment lifecycles.

2.3.2.1 Bounded Context Map

The eight bounded contexts partition the e-commerce domain along business capability boundaries. Each context owns its data, its domain logic, and its vocabulary, terms that are well-defined within a context may carry different meaning in another. For example, a Variant in the Catalog context is a sellable unit with a SKU and pricing; a LineItem in the Ordering context references that same variant but from the perspective of purchase fulfilment.

The integration between contexts follows the **Conformist** pattern: all contexts conform to a shared technical kernel defined in the Shared layer, which provides the `Result<T>` return type, the `ICommand` and `IQuery` marker interfaces, and the `Entity` base class with audit and versioning columns. Communication occurs exclusively through MediatR `ISender`, a context dispatches a query or publishes a notification, and other contexts react without ever importing one another's namespace. This in-process dispatch model eliminates the network latency of inter-service messaging while preserving the logical isolation of the bounded contexts.

Figure 8 depicts the eight contexts and the **Published Language**, the shared identifiers and value types, that flow between them.

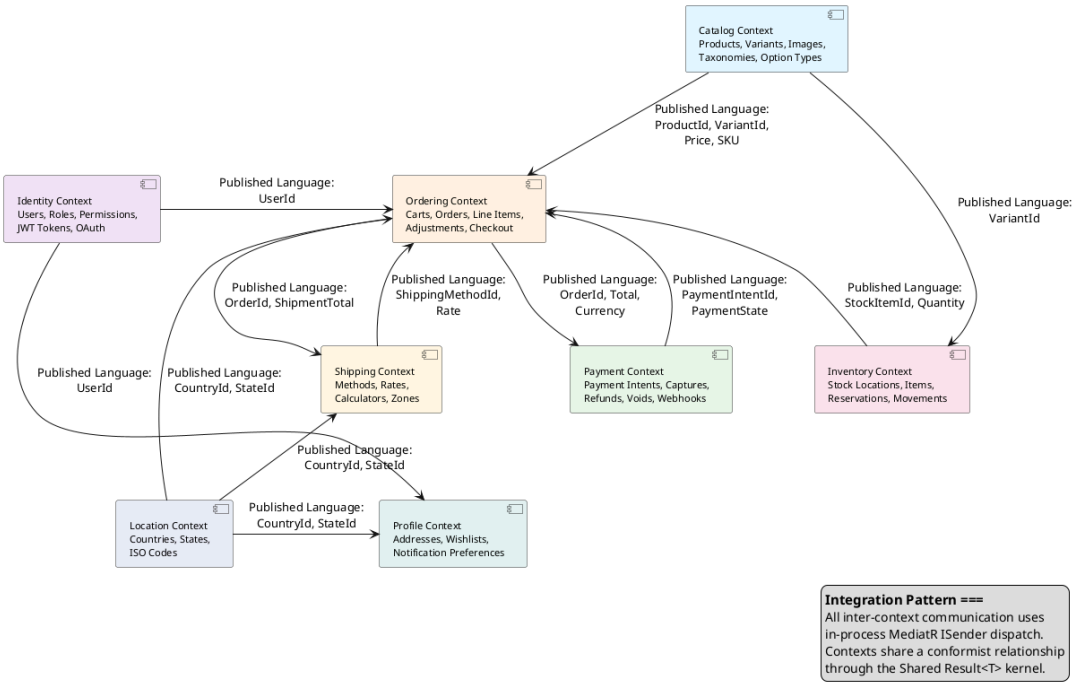

Figure 8: Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace.

Table 53 details each context's business responsibility and the integration data it exposes to the system.

Table 53: Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace.

Context	Business Responsibility	Published Language
Catalog	Manages the product lifecycle: creating products with fashion-specific metadata (style code, season, material, department, gender target), defining sellable variants with SKUs and indepen-	ProductId, VariantId, Sku, Price, Slug

Context	Business Responsibility	Published Language
	dent pricing, uploading images with automatic embedding generation, and organising products through hierarchical taxonomies.	
Ordering	Orchestrates the customer purchase workflow from cart to completed order. Manages cart with seven-day auto-expiry, forward-only checkout state machine, line items with price snapshots, adjustments, and cancellation at any pre-confirmation stage.	OrderId, OrderNumber, Total, Currency, CheckoutState
Payment	Manages payment intent lifecycle, creation, capture, refund, void, across two gateway implementations. Maintains parallel payment state independent of the gateway for offline operations and consistent behaviour across providers.	PaymentIntentId, PaymentState, Amount

Context	Business Responsibility	Published Language
Inventory	Tracks physical stock quantities per warehouse, manages temporary reservations during active checkouts to prevent overselling, records auditable stock movements through an append-only ledger, and handles inter-warehouse transfers.	StockItemId, QuantityOnHand, QuantityReserved
Identity	Provides JWT-based authentication with refresh token rotation and reuse detection, role-based and permission-based authorisation with `domain:category:action` claim format, and guest session management for anonymous browsing.	UserId, Email, PermissionClaim
Profile	Manages user addresses for shipping and billing, wishlists for product bookmarking, and notification preferences controlling email and SMS commu-	ProfileId, AddressId

Context	Business Responsibility	Published Language
	nication channels.	
Shipping	Configures delivery methods, standard, express, local pickup, and calculates shipping rates by geographic zone using weight- and distance-based calculators.	ShippingMethodId, Rate
Location	Provides country and state reference data with ISO 3166 codes. This context is read-only reference data shared across Shipping (zone configuration), Profile (address validation), and Ordering (checkout address selection).	CountryId, StateId, IsoCode

2.3.2.2 Aggregates and Invariants

An aggregate is a cluster of domain objects treated as a single consistency boundary. Each aggregate has a root entity through which all modifications must pass. The root enforces invariants, business rules that must hold true at all times within the aggregate boundary. ReSys.Shop takes a pragmatic approach to DDD: it defines aggregate roots and their invariants explicitly but does not require formal value-object base classes or a dedicated domain-event infrastructure for every operation.

The four most architecturally significant aggregates are described below.

Product (Catalog aggregate root). The Product aggregate encapsulates a product family and all its variants, images, option configurations, and taxonomy classifications. A product may have one or more variants; exactly one is designated as the master variant displayed on listing pages. The aggregate enforces the following invariants: every product must have a unique slug for SEO-friendly URL generation; a product that declares options (such as size or colour) must have at least one option type defined; and the master variant must exist among the product's own variants. Variant images contain the `embedding` column, a 512-dimensional float vector generated by the ML sidecar, enabling cosine similarity search against the entire image corpus.

Order (Ordering aggregate root). The Order aggregate manages the checkout lifecycle from a nascent cart through to a completed purchase. It aggregates line items, each capturing a price snapshot of the variant at the time of purchase, and optional adjustments for discounts or promotions. The aggregate enforces the invariant that `Total = ItemTotal + AdjustmentTotal + ShipmentTotal`, maintaining financial consistency across all modifications. The checkout state progresses forward only, the address, delivery, payment, and confirmation stages must complete in sequence, and once an order is confirmed (finalised), it becomes immutable except for the cancel transition. This forward-only constraint is encoded in the domain entity itself and validated before every state transition.

PaymentIntent (Payment aggregate root). The PaymentIntent aggregate models the lifecycle of a customer's intent to pay. It is created with a specified amount and currency, and transitions through states, Pending, RequiresAction, Processing, Succeeded, Canceled, and Failed, based on gateway interactions. The aggregate tracks payment captures, where each capture debits a portion of the authorised amount. The system enforces the invariant that the sum of all captures must not exceed the original intent amount. A separate payment capture goes through its own state transitions: Succeeded → Captured → Refunded / Voided. The system maintains its own payment state in parallel with the gateway state, enabling consistent behaviour whether using the production Stripe gateway or the development Bogus gateway.

StockItem (Inventory aggregate root). The StockItem aggregate tracks the physical availability of a product variant at a specific warehouse location. It maintains two quantities: on-hand (physical count from warehouse operations) and reserved (units held for active checkouts). The aggregate enforces the invariant that `QuantityOnHand ≥ 0`, stock cannot go negative. Backorder support allows sales beyond on-hand quantity up to a configured backorder limit, but the system tracks the deficit separately. Quantity changes are not performed directly on StockItem; instead, they must be recorded as StockMovement entries in an append-only ledger, preserving a complete and auditable history of every stock

change, including the quantity before and after, the reason, and the operating user.

2.3.2.3 Ubiquitous Language Glossary

A key practice in DDD is establishing a ubiquitous language, a shared vocabulary used by all team members and reflected directly in the codebase. Table 54 presents the core terms of the ReSys.Shop domain with their definitions.

Table 54: Ubiquitous Language Glossary: core domain terms, their owning bounded context, and their precise definitions as used throughout the codebase and this thesis.

Term	Context	Definition
Product	Catalog	A product family representing a fashion item concept (e.g., "Cotton T-Shirt"). Holds shared metadata: description, slug, status, fashion-specific attributes, and taxonomy classifications. Does not have a price or SKU directly, those belong to Variants.
Variant	Catalog	A sellable, physical unit of a product (e.g., "Cotton T-Shirt, Red, Large"). Holds SKU, barcode, pricing, inventory tracking flag, and physical dimensions. Each variant belongs to exactly one product.
Master Variant	Catalog	The designated default variant shown on product listing pages. Every product with variants must have exactly one master variant.
Option Type	Catalog	Defines a configurable attribute class such as "Colour", "Size", or "Material". Option types are product-independent and reusable across the catalogue.
Taxonomy / Taxon	Catalog	A hierarchical categorisation tree for organising products. A Taxonomy (e.g., "Department") contains Taxons (e.g., "Clothing" → "Dresses" → "Evening Dresses") forming a nested set structure.
Cart	Ordering	An ephemeral collection of line items that represents a customer's intent to purchase. Carts automatically expire after seven days of inactivity. The cart is the initial state of the Order aggregate.
Line Item	Ordering	A single entry in an order or cart, referencing a specific product variant, quantity, and a price snapshot captured at the time of purchase to insulate historical orders from catalogue price changes.
Checkout State	Ordering	The sequential stage of the purchase process: Address → Delivery → Payment → Confirm → Complete. Progression is forward-only; cancellation is available from any pre-confirmation stage.
Payment Intent	Payment	A data structure representing a customer's intent to pay a specified amount. Follows a defined state machine through the gateway interaction lifecycle.

Term	Context	Definition
Payment Capture	Payment	A partial or full debit against a payment intent. Multiple captures can be performed against a single intent (e.g., capturing shipping cost after items ship).
Stock Item	Inventory	The current state of a product variant's availability at a specific warehouse. Maintains on-hand and reserved quantity counters with optimistic concurrency control to prevent overselling.
Stock Movement	Inventory	An immutable ledger entry recording every change to a stock item's quantity, the delta, the balance before and after, the reason, and the operating user. The single source of truth for inventory changes.
Refresh Token	Identity	A long-lived credential used to obtain new short-lived access tokens without re-authentication. Each token is single-use; presenting a previously consumed token triggers revocation of all tokens for that user.

2.3.2.4 State Machines

Two explicit state machines govern the most critical transactional workflows in the system: the order checkout process and the payment intent lifecycle. Both are encoded in domain entities, validated before every state transition, and drive the sequence of user-facing and system-level actions.

2.3.2.4.1 Order Checkout State Machine

The order checkout state machine enforces a forward-only progression through five sequential states: Address, Delivery, Payment, Confirm, and Complete. Each state transition is triggered by a specific user action and validated by the domain entity before being committed. Figure 9 depicts this lifecycle.

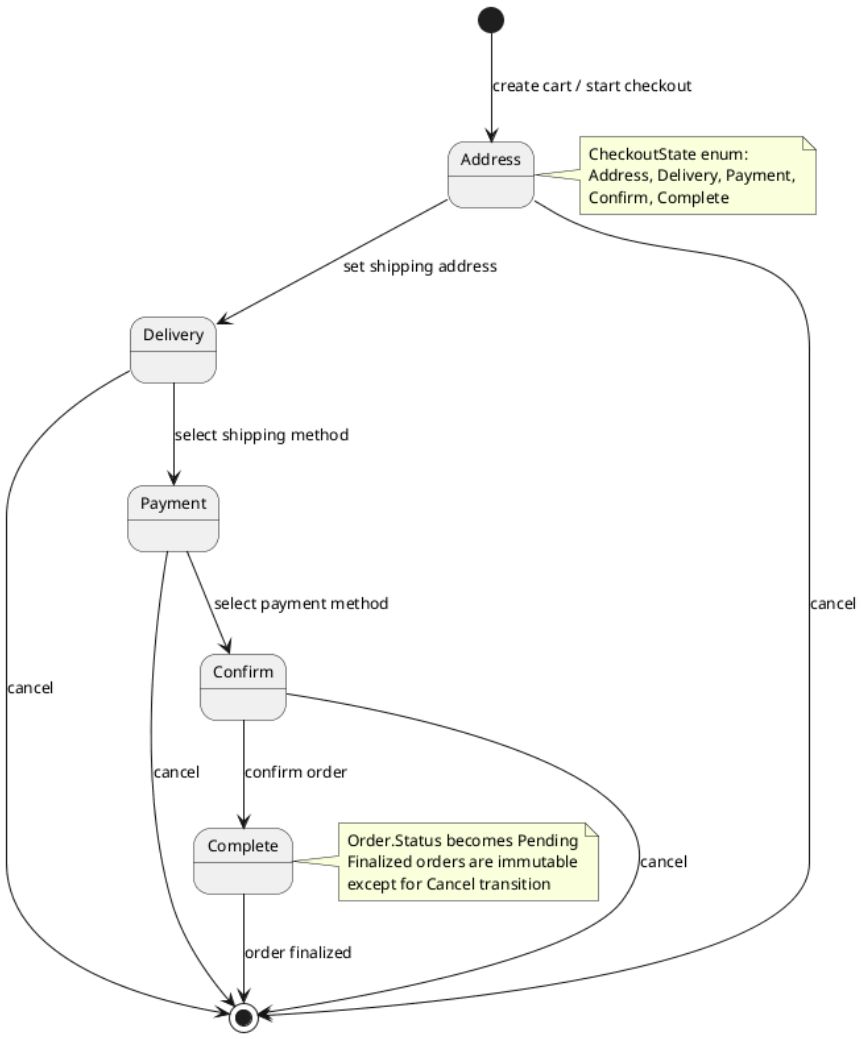

Figure 9: Order checkout state machine: five sequential states with cancellation available from any pre-confirmation state. The forward-only constraint prevents regressing to earlier checkout stages.

The customer begins by providing a shipping address (Address state), selects a delivery method (Delivery state), chooses a payment method (Payment state), reviews the complete order summary (Confirm state), and finalises the purchase (Complete state). At any point before the Complete state, from Address through Confirm, the customer may cancel the checkout process, which terminates the order without financial consequence.

Once an order reaches the Complete state, it becomes finalised: the order record transitions to Pending status, inventory quantities are reserved for each line item, and the payment intent is processed. From this point forward, the order is immutable except for the cancel transition, which captures the cancellation timestamp and releases reserved inventory. This forward-only design ensures that at every stage of the checkout pipeline, the system can unambiguously determine the customer's position and the next required action.

2.3.2.4.2 Payment Intent State Machine

The payment intent state machine models the full lifecycle of a payment from creation through to terminal completion, reflecting the state transitions of the Stripe payment gateway while maintaining a parallel system-managed state for offline consistency. Figure 10 shows all states and transitions.

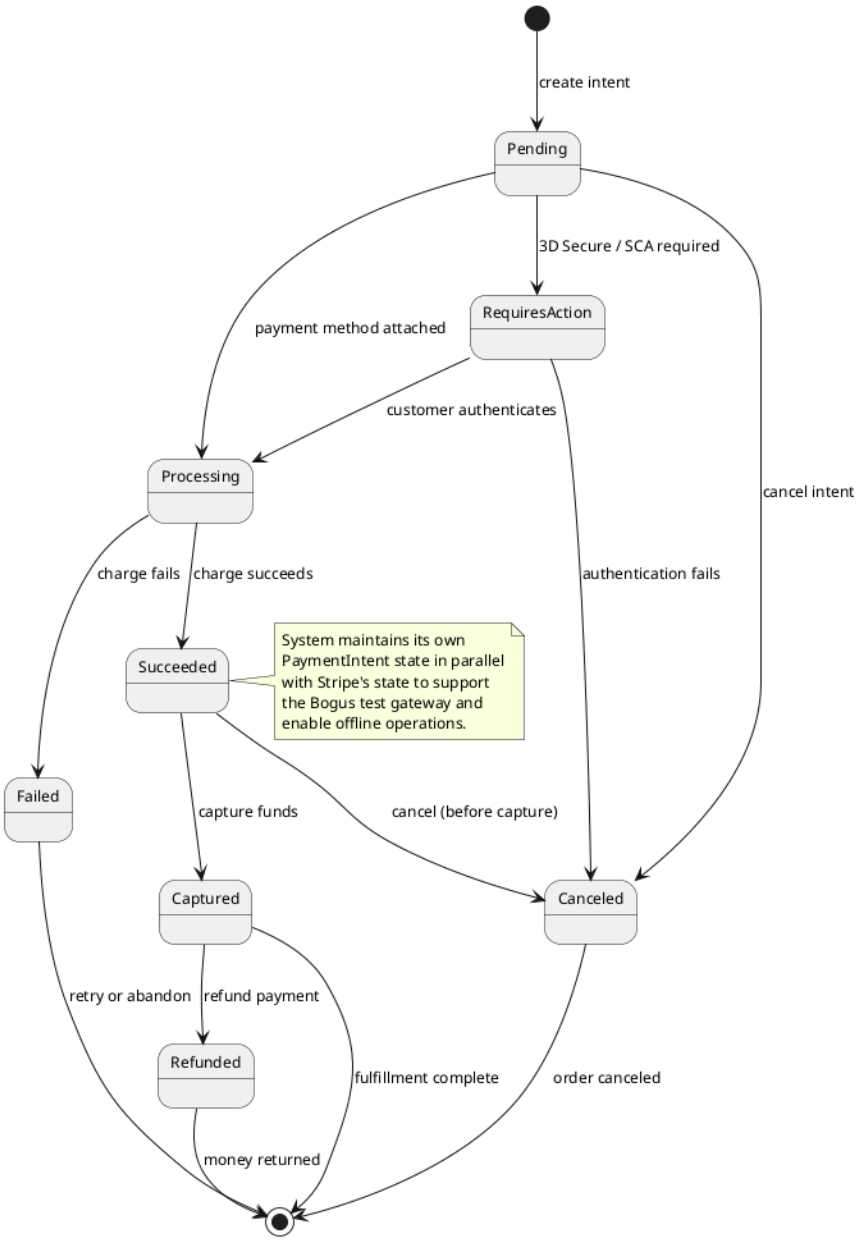

Figure 10: Payment intent lifecycle: the state machine reflects Stripe gateway states while maintaining a parallel system copy for offline operations and Bogus gateway compatibility. Terminal states are Failed, Canceled, and Refunded.

A payment intent is created in the Pending state. It may transition directly to RequiresAction when the payment requires 3D Secure or Strong Customer Authentication, or to Processing when the payment method has been attached

without additional authentication challenges. From RequiresAction, successful customer authentication advances the intent to Processing; failure or timeout moves it to Canceled.

From Processing, a successful charge transitions the intent to Succeeded, while a declined or errored charge moves it to Failed. From Succeeded, the funds may be captured, transferring them from the customer's account, or the intent may be cancelled before capture. A captured intent may later be refunded, returning funds to the customer and reaching the terminal Refunded state.

The system maintains its own copy of the payment state in parallel with the gateway's representation. This design decision serves two purposes. First, it enables the Bogus test gateway, a development-only implementation that simulates payment lifecycles without external calls, to operate against the same domain entities as the production Stripe gateway. Second, it allows the system to reason about payment state offline without querying the gateway API, which improves resilience during network interruptions and reduces external dependency during business operations.

2.3.3 C4 Architecture

The C4 model provides a structured approach to describing software architecture at four levels of abstraction: system context, container, component, and code. This section presents the first three levels for ReSys.Shop, omitting the code-level view as it falls within the scope of the implementation chapter. A deployment diagram complements the C4 views by showing the physical infrastructure.

2.3.3.1 System Context

The system context diagram positions ReSys.Shop within its environment, showing the human users who interact with the platform and the external systems on which it depends. Figure 11 presents this highest-level view.

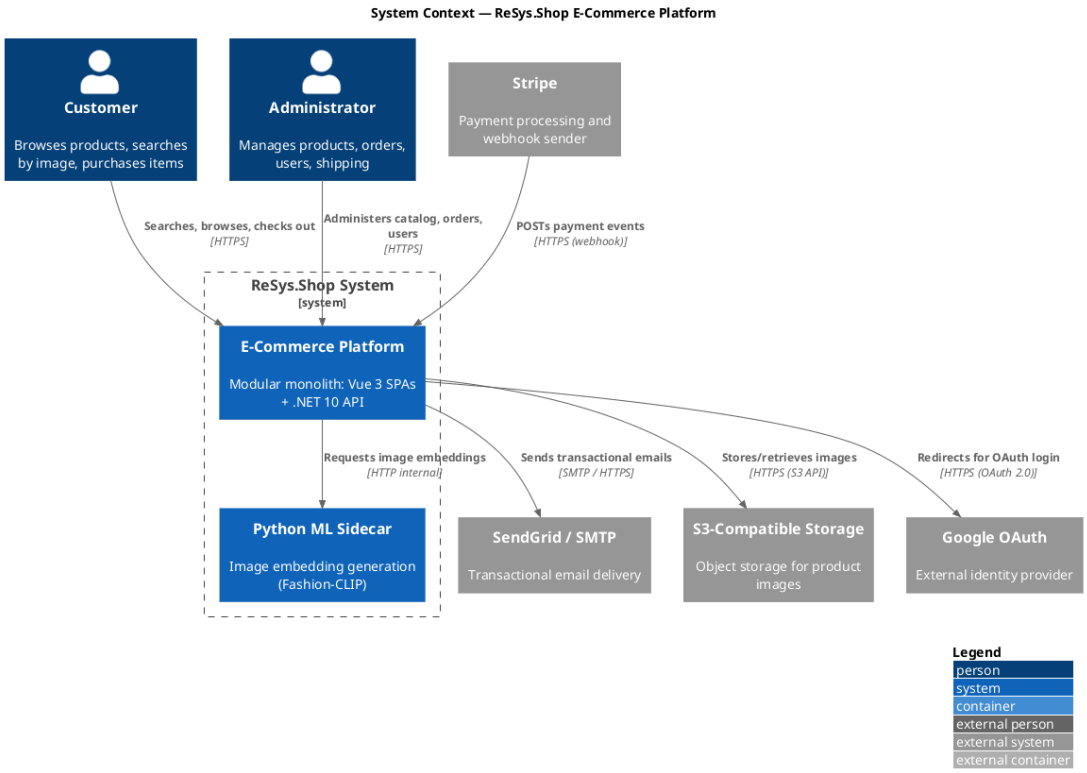

Figure 11: System Context diagram: ReSys.Shop as a single system boundary with customer and administrator users on one side and external payment, email, storage, identity, and ML services on the other. The modular monolith internally handles all eight business domains within one boundary.

Two categories of human users interact with the platform. Customers browse the product catalogue, perform visual and keyword searches, manage a shopping cart, and complete multi-step checkout, all through the Vue 3 storefront SPA. Administrators manage the full product lifecycle, process orders, monitor inventory levels, and administer user accounts through the Vue 3 admin SPA.

The system depends on five external services. Stripe processes payment intents and sends webhook notifications when payment events occur, the backend validates these webhooks using Stripe's signature verification before acting on them. SendGrid delivers transactional emails such as order confirmations, password reset links, and shipping notifications. An S3-compatible object store persists product images uploaded through the admin interface. Google OAuth provides an alternative authentication path, allowing customers to sign in using their Google credentials. The Python ML Sidecar, deployed as a companion service within the Aspire orchestration boundary, generates image embeddings used by the catalogue's visual search feature.

2.3.3.2 Container

The container diagram decomposes ReSys.Shop into its deployable units, the processes and data stores that together constitute the running system. Figure 12 presents this view.

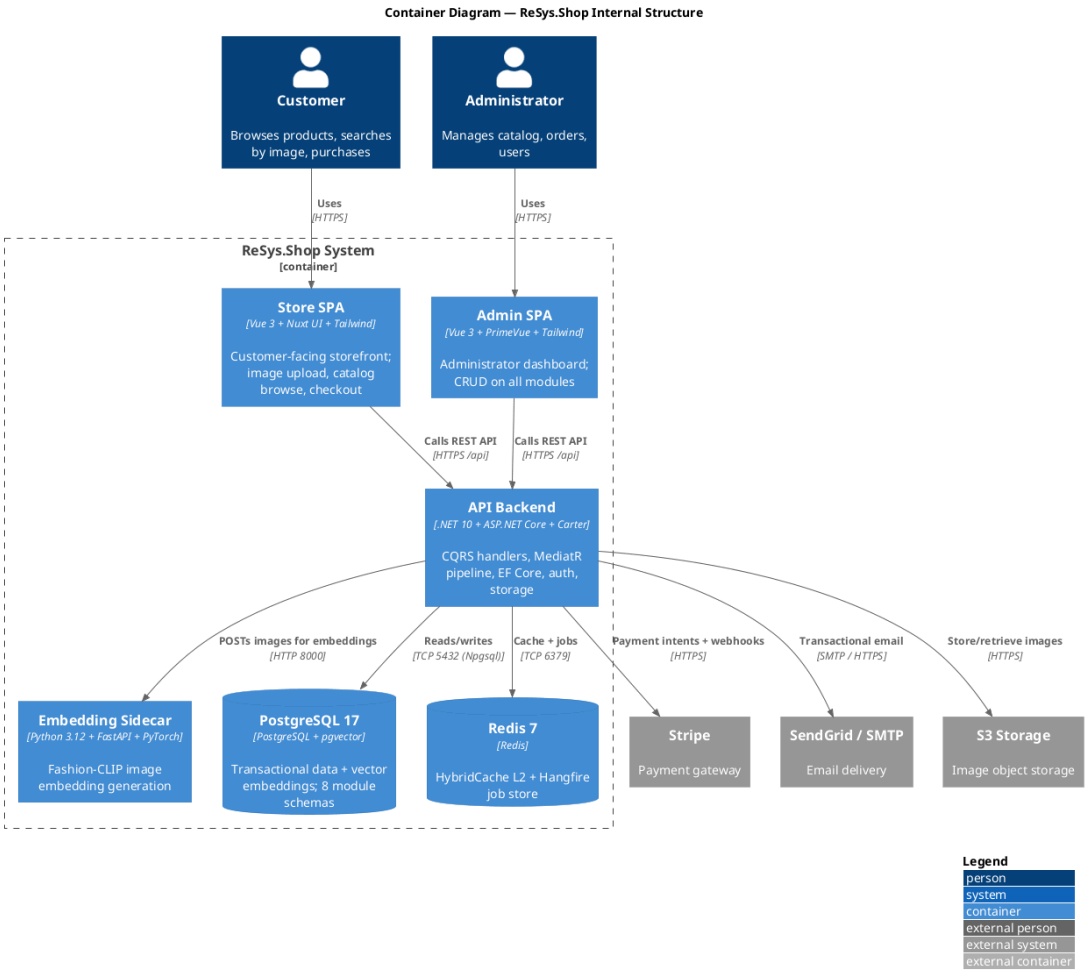

Figure 12: Container diagram showing the deployable units of ReSys.Shop: two Vue 3 SPAs, the .NET 10 API backend, the Python ML sidecar, PostgreSQL with pgvector, and Redis. Arrows indicate communication protocols between containers.

The system comprises six deployable containers. The Store SPA and Admin SPA, both Vue 3 applications served as static assets, handle all user interface concerns and communicate with the backend exclusively through the REST API over HTTPS. The API Backend, a .NET 10 application running on ASP.NET Core, contains all business logic across eight modules, exposes Carter minimal API endpoints, and orchestrates the MediatR CQRS pipeline for command and query processing. The Embedding Sidecar, a Python 3.12 FastAPI application, loads machine learning models into GPU or CPU memory and exposes HTTP endpoints for generating image embeddings.

Two persistent data stores support the platform. PostgreSQL 17 with the pgvector extension serves as the primary database, storing both relational transactional data across eight module-specific schemas and high-dimensional vector embeddings for visual similarity search. Redis 7 fills a dual role: as the second-level distributed cache backing the HybridCache abstraction, and as the persistent job store for Hangfire background job processing, enabling cart expiry, webhook dispatch, and periodic maintenance tasks to survive application restarts.

The communication topology reflects deliberate design constraints. The Vue SPAs call the backend synchronously over HTTPS, never directly accessing the database or external services, which ensures all security policies and data validation are enforced server-side. The backend communicates with PostgreSQL and Redis over internal TCP connections, with the ML sidecar over HTTP on the internal Docker network, and with external services over HTTPS. This design centralises all external integration through the backend container, simplifying security management and operational monitoring.

2.3.3.3 Component

The component diagram zooms into the API Backend container, revealing its internal structure: the modules, framework services, and cross-cutting concerns that compose the .NET application. Figure 13 presents this view.

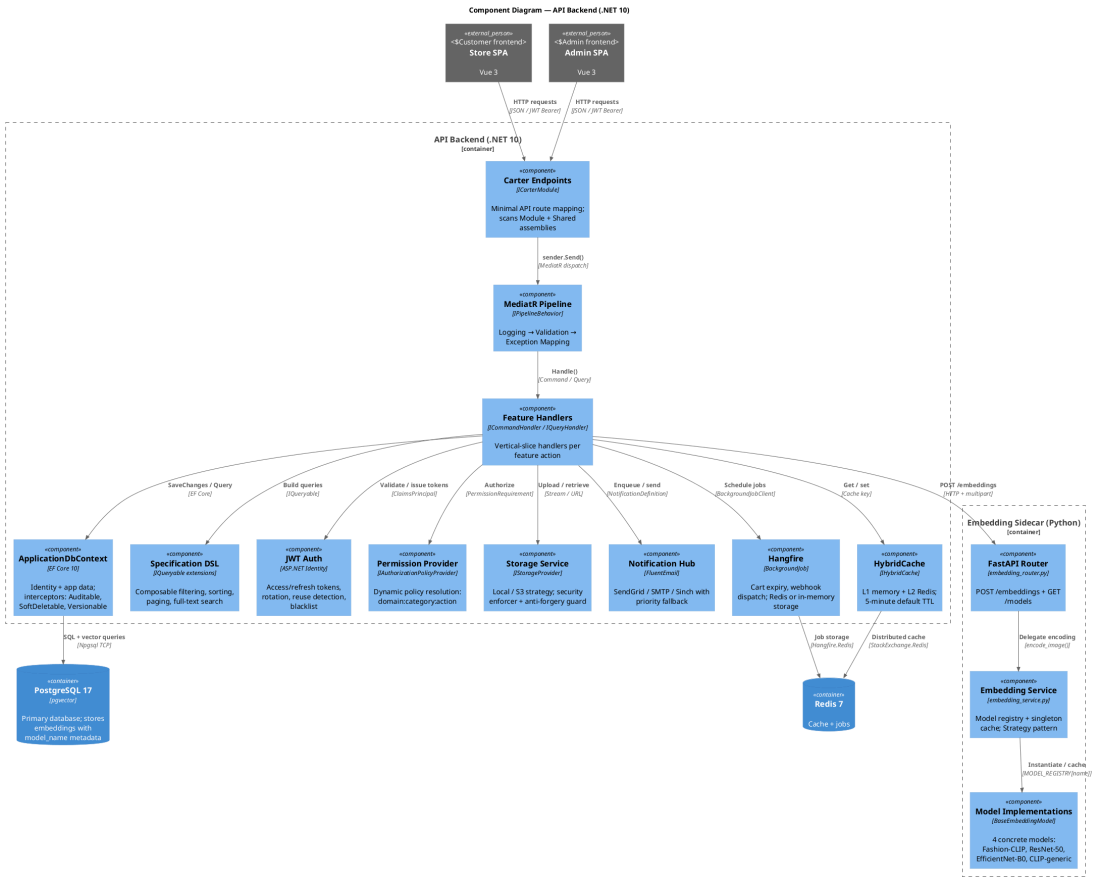

Figure 13: Component diagram of the API Backend showing the Carter endpoint layer, the MediatR pipeline, the feature handlers, and eight supporting infrastructure components. The Python ML Sidecar is shown with its internal three-layer architecture alongside.

The API Backend is structured as a pipeline. HTTP requests arrive at the Carter endpoints, which are minimal API route groups registered by `ICarterModule` implementations in each module's feature folder. The endpoints are thin, they extract request parameters, dispatch a command or query via `ISender`, and map the `Result<T>` response to an HTTP status code and JSON body. All business logic resides in the feature handlers.

The MediatR pipeline wraps every request with a chain of behaviours: logging captures the request type and timing, validation executes FluentValidation rules before the handler runs, and exception mapping converts unhandled infrastructure failures to standardised problem details. The handlers themselves interact with eight infrastructure components:

- ApplicationDbContext (EF Core 10) with interceptors for auditable timestamps, soft-delete filtering, and row-version concurrency checks.
- A Specification DSL that provides composable `IQueryable` extensions for filtering, sorting, paging, and full-text search, keeping handler code free of query-building boilerplate.
- JWT authentication with ASP.NET Identity, managing access and refresh token issuance, rotation, reuse detection, and token blacklisting.
- A dynamic permission provider that resolves `{domain}:{category}:{action}` permission claims to authorisation policies at runtime without requiring static policy registration.
- A storage service with interchangeable providers (local filesystem or S3-compatible storage) selected via configuration, with built-in file-type validation and anti-forgery guards on uploads.
- A notification hub supporting email (SendGrid/SMTP) and SMS (Sinch) channels with configurable fallback priority.
- Hangfire for background job scheduling and processing, handling cart expiry, webhook dispatch, and periodic health checks.
- HybridCache with two-tier caching: L1 in-memory for sub-millisecond access and L2 Redis for cross-instance consistency.

The Python ML Sidecar follows a three-layer architecture: the FastAPI router handles HTTP request validation and API key authentication, the Embedding Service maintains a singleton model registry with lazy loading and caching, and the model implementations, Fashion-CLIP, ResNet-50, EfficientNet-B0, and generic CLIP, implement a common strategy interface for interchangeable inference backends.

2.3.3.4 Deployment

The deployment diagram illustrates how the containers map to physical or virtual infrastructure in a production configuration. Figure 14 shows the deployment topology.

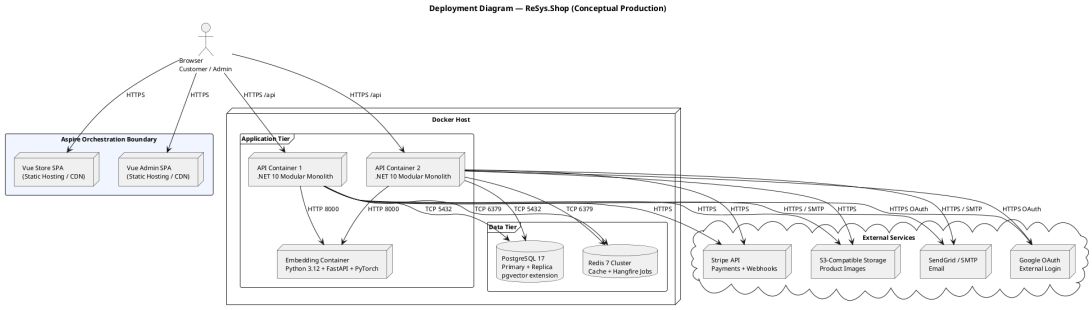

Figure 14: Deployment diagram showing containerised services within an Aspire orchestration boundary. The API backend is horizontally scalable; the embedding sidecar is stateless; Redis enables distributed state across API replicas.

All services are containerised and orchestrated by .NET Aspire, which manages service discovery, configuration injection, and health monitoring during both development and production deployments. The Vue SPAs are served as static bundles from a CDN or reverse proxy, while the backend services run within Docker containers on a single host or across a cluster.

The API backend is horizontally scalable, multiple container instances share PostgreSQL and Redis, enabling round-robin request distribution. The embedding sidecar is stateless: it loads models into memory on startup, caches them, and serves embedding requests without shared state. Any API instance can call any embedding container. Redis provides the distributed state needed for cache coherence and Hangfire job coordination across API replicas.

PostgreSQL is configured with a primary instance for writes and one or more read replicas for reporting and analytical queries. The pgvector extension is installed on both primary and replicas, enabling vector similarity search from any read path. External services, Stripe, SendGrid, S3 storage, and Google OAuth, are accessed over HTTPS from every API instance, with credentials managed through Aspire's configuration system and never baked into container images.

2.3.4 Database Design

The ReSys.Shop database is a single PostgreSQL 17 instance organised into per-context schemas, each owned by a bounded context and managed through Entity Framework Core migrations. This section describes the schema organisation, the

core entity-relationship model, the pgvector integration for visual search, and the key design decisions that shape the data layer.

2.3.4.1 Schema Organisation

Each of the eight bounded contexts owns its database schema. The Catalog context manages tables for products, variants, variant images, option types, option values, taxonomies, and taxons. The Ordering context owns orders, line items, adjustments, shipments, and inventory units. The Payment context holds payment intents, payment captures, and payment logs. The Inventory context manages stock locations, stock items, stock movements, and stock reservations. The Identity context, built on ASP.NET Identity Core, manages users, roles, user roles, refresh tokens, and external login providers. The Profile context owns user addresses, wishlists, and notification preferences. The Shipping context manages shipping methods, shipping rates, and shipping zones. The Location context provides country and state reference tables.

Cross-context relationships are implemented through identifier references only, there are no foreign key constraints spanning module boundaries. An Order references a UserId (Identity context) and a VariantId (Catalog context), but these are stored as UUIDs without database-level referential integrity constraints. This design preserves the logical isolation of each context as a separate schema while keeping all data in a single database, avoiding the complexity of distributed transactions for the most common business operations.

Entity Framework Core manages all migrations from a dedicated Migrations assembly. Each migration is generated by comparing the current database state against the domain entity model, and migrations are applied as part of the application startup or through standalone migration scripts for production deployments.

2.3.4.2 Core Entity-Relationship Model

Figure 15 presents the entity-relationship diagram for the core business entities across all eight bounded contexts, their key attributes, primary and foreign keys, and the relationships within and between contexts. Cross-context relationships (dotted lines) use identifier references only, without database-level foreign key constraints.

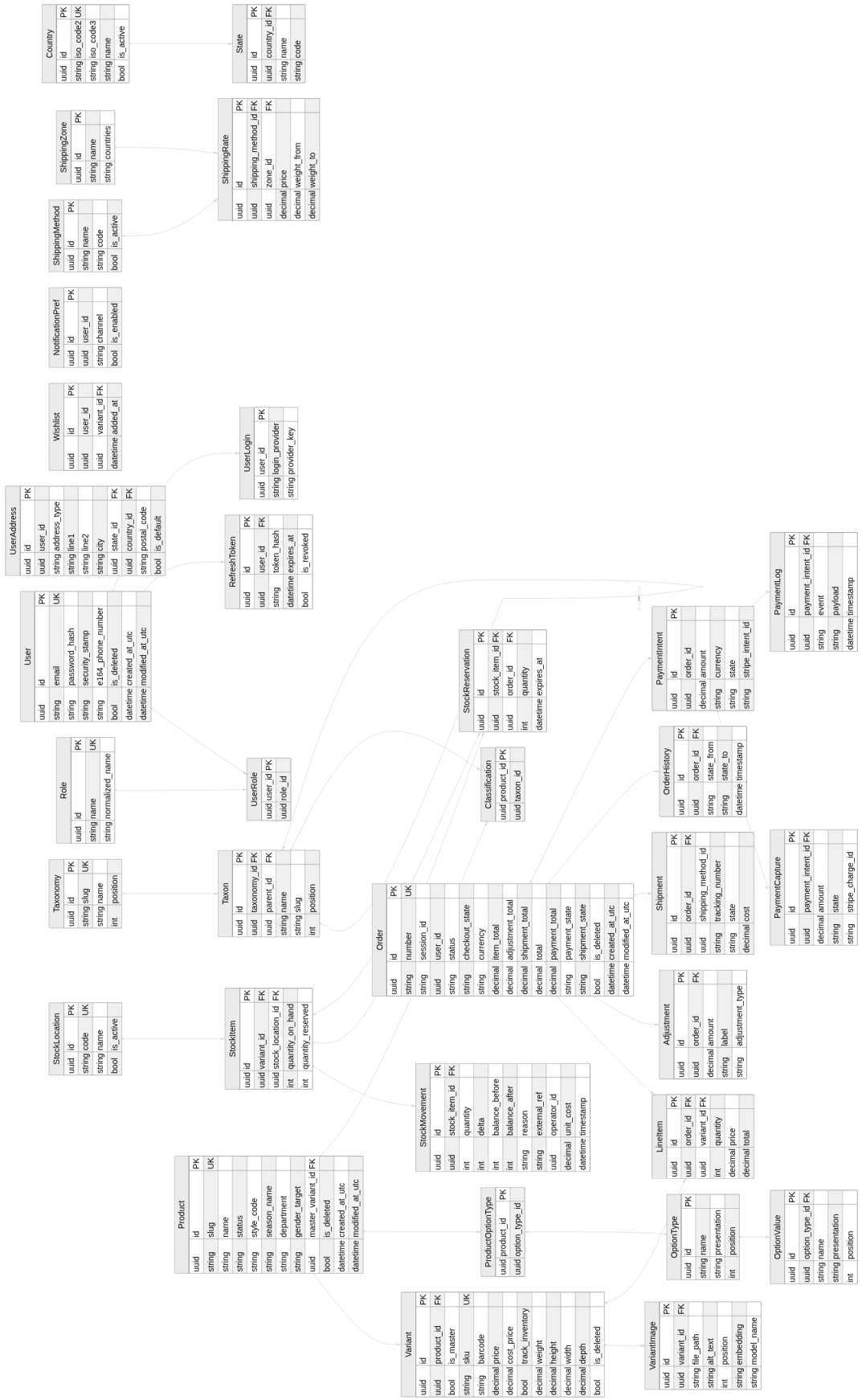

Figure 15: Core entity-relationship model across all eight bounded contexts.

The Catalog domain centres on the Product entity, which has a one-to-many relationship with Variant. Each Variant represents a specific sellable configu-

ration, for example, “Cotton T-Shirt, Red, Large”, with its own SKU, pricing, and inventory tracking flag. Exactly one variant per product is designated as the master variant. Variants relate one-to-many to VariantImages, each of which holds a file path and an optional `embedding` column of type `vector(512)` for pgvector similarity search. Products configure option types, such as Colour or Size, which in turn define option values. Taxonomies contain taxons in a hierarchical structure, with taxons referencing themselves through a parent foreign key to represent tree structures such as “Clothing → Dresses → Evening Dresses”. Products are classified by taxons through a many-to-many classification table.

The Ordering domain centres on the Order entity, which has a one-to-many relationship with LineItem. Each line item references a specific variant, not a product directly, because customers purchase specific configurations. The line item captures a price snapshot at the time of purchase, ensuring that historical orders are unaffected by catalogue price changes. Orders are related to users through a UserId reference, optionally nullable to support guest checkout, and track a session identifier for mapping anonymous carts before authentication.

2.3.4.3 pgvector Integration

PostgreSQL’s pgvector extension enables vector similarity search directly within the relational database, eliminating the need for a separate vector database. The `variant_images` table contains an `embedding` column of type `vector(512)`, a fixed-length array of 512 IEEE 754 single-precision floating-point numbers representing the visual features extracted from the image by the ML sidecar.

An HNSW (Hierarchical Navigable Small World) index is created on the embedding column to accelerate approximate nearest-neighbour searches. HNSW provides logarithmic search complexity, enabling sub-10-millisecond queries over tens of thousands of vectors. The index is configured for cosine distance, the recommended distance metric for normalised embeddings from CLIP-family models.

Vector similarity queries use the cosine distance operator (`<=>`), which computes the angular distance between two vectors. A representative conceptual query pattern is: retrieve all variant images, compute the cosine distance between each stored embedding and the query embedding, order by ascending distance, and return the top results. The system filters results by a configurable minimum similarity threshold, computed as 1 - cosine_distance, defaulting to 0.7, a level at which fashion images typically exhibit perceptible visual similarity.

Each embedding row includes a `model_name` column identifying which ML model generated the vector. This metadata enables per-model filtered queries: when the system switches the active embedding model, only embeddings from that model’s columns participate in similarity search, preventing cross-model

vector comparisons that would produce meaningless results. The model name also supports the benchmark evaluation in Chapter 3, where multiple models are compared against the same image corpus.

2.3.4.4 Key Design Decisions

Several design decisions govern the database layer and influence every bounded context:

Universally Unique Identifiers. All primary keys are UUIDs (GUIDs in .NET terminology). This decision eliminates reliance on a central sequence generator, allows safe distributed key generation, useful for client-side offline creation of cart or draft-order records, and prevents the information leakage inherent in auto-incrementing integer primary keys.

Soft Deletion. Most domain entities carry an `IsDeleted` boolean column. Entity Framework Core applies a global query filter that excludes soft-deleted rows from all standard queries. This preserves referential integrity, a deleted product does not orphan its historical orders, while keeping the active working set clean. Deleted entities can be restored by administrators if the deletion was in error.

Audit Columns. Every entity inherits `CreatedAtUtc` and `ModifiedAtUtc` timestamp columns from the `Entity` base class, populated automatically by an EF Core save interceptor. These columns support chronological analysis of catalogue changes, order lifecycle auditing, and debugging data anomalies without external logging correlation.

Composite Indexes. Tables serving high-frequency query patterns carry composite indexes tuned to their access patterns. The Orders table includes `(UserId, Status)` for listing a customer's orders filtered by state, and `(SessionId, Status)` for resolving anonymous carts. These composite indexes avoid the need for separate single-column indexes and reduce disk footprint.

Variable Vector Dimensions. Different embedding models produce vectors of different dimensionalities: 384 for DINOv2-S, 512 for Fashion-CLIP and CLIP, 768 for DINOv2-B, 1280 for EfficientNet-B0, and 2048 for ResNet-50. PostgreSQL's `vector` type supports this variability, the dimension is part of the column type declaration, and HNSW indexes are built per-dimension. The system stores embeddings from all models in separate rows, each tagged with the model name, and queries against the active model's dimension only.

2.3.4.5 Per-Context Schema Description

The Identity context stores user accounts with Argon2-hashed passwords, security stamps for session invalidation, and refresh tokens with one-time-use semantics. Role and UserRole tables implement RBAC, while UserLogin

records link local accounts to external OAuth providers. UserAddresses store shipping and billing locations with ISO country codes.

The Catalog context's Product table carries fashion-specific metadata columns, style code, season name, material composition, department, and gender target, alongside standard catalogue fields. Variants hold SKU, barcode, cost and selling price in decimal precision, and physical dimensions for shipping calculations. The OptionType and OptionValue tables implement an EAV-lite pattern, allowing administrators to define new product attributes without schema migrations. Taxons use a nested set model with left and right bounds, enabling single-query subtree retrieval for mega-menus and breadcrumb navigation.

The Ordering context stores financial values as decimal with 18-digit precision and 2-decimal scale, avoiding floating-point accumulation errors in tax and discount calculations. Orders maintain separate subtotals for items, adjustments, and shipping, with the total computed as their sum. LineItems capture price snapshots at purchase time, decoupling order history from catalogue changes. OrderHistory provides an append-only audit trail of every state transition.

The Inventory context tracks stock items at a variant-location granularity, with `QuantityOnHand` and `QuantityReserved` maintained as separate counters and protected by optimistic concurrency control using PostgreSQL's `xmin` system column. StockMovements form an immutable ledger, every quantity change, whether from receiving, selling, returning, or stock-taking, is recorded with the delta, balances before and after, unit cost, and an external reference such as an order number or purchase order identifier.

2.3.5 API Design

The ReSys.Shop API exposes a RESTful interface built on Carter minimal APIs and organised around the MediatR CQRS pattern. This section describes the API architecture, the endpoint organisation scheme, and a summary of the key endpoints that define the platform's external contract.

2.3.5.1 API Architecture

The API layer acts as a thin orchestration boundary. It contains no business logic; instead, it delegates all processing to the MediatR pipeline. Each request follows a consistent path: the Carter endpoint receives the HTTP request, extracts route and body parameters, constructs a MediatR command or query object, dispatches it through `ISender`, and maps the returned `Result<T>` to an HTTP response. This design keeps endpoints concise, typically six to twelve lines, and concentrates all domain logic in the handler layer, where it is testable without HTTP infrastructure.

Carter modules group related endpoints by module and surface. Each module (Catalog, Ordering, Payment, and so on) registers its own `ICarterModule`

implementation, which defines the route groups, HTTP methods, and parameter bindings for that module's endpoints. This modular registration avoids a single monolithic route configuration file and enables each bounded context to own its API surface.

FluentValidation provides input validation through validator classes associated with each command and query. Validators run automatically as part of the MediatR pipeline behaviour, before the handler executes, ensuring that handlers never receive invalid input. Validation failures return standardised `400 Bad Request` responses with field-level error details.

2.3.5.2 Endpoint Organisation

Endpoints are organised by two dimensions: the business module that owns the operation, and the surface, Admin or Storefront, that serves as the entry point. The URL pattern follows the convention `/api/{module}/{surface}/{action}`, where module identifies the owning bounded context, surface distinguishes administrative from customer-facing operations, and action names the specific operation.

This two-dimensional organisation serves several purposes. It makes the API self-documenting: the URL alone communicates which business area and which user role the endpoint targets. It simplifies authorisation: Admin surface endpoints share a common authorisation policy requiring an administrator role, while Storefront endpoints apply corresponding customer-level policies. And it enables independent versioning: a module can evolve its endpoints without affecting other modules.

Table 55 summarises the most architecturally significant endpoints across the platform. These endpoints represent the primary user-facing capabilities, visual search, catalogue browsing, checkout, order history, authentication, and payment, that together define the complete customer and administrator experience.

Table 55: Key API endpoints representing the primary user-facing capabilities of the platform. All endpoints in the Storefront surface serve customer interactions; Admin surface endpoints (not shown) mirror these with full CRUD capabilities on all modules.

Endpoint	Module	Surface	Description
`POST /api/catalog/storefront/search-by-image`	Catalog	Storefront	Accepts an uploaded image

Endpoint	Module	Surface	Description
			file, sends it to the ML sidecar for embedding generation, queries pgvector for the nearest neighbour variant images by cosine similarity, and returns matching products ranked by similarity score with

Endpoint	Module	Surface	Description
			variant thumbnails, pricing, and product URLs.
`GET /api/catalog/storefront/products/{slug}`	Catalog	Storefront	Returns a product with all its published variants, images, option configurations, and taxonomy classifications, identified by its URL slug. Supports guest ac-

Endpoint	Module	Surface	Description
			cess for anonymous browsing.
`POST /api/ordering/storefront/cart/checkout`	Ordering	Storefront	Advances the cart through the checkout state machine: setting the shipping address, selecting the delivery method, and confirming the order. Each call transitions the checkout state forward

Endpoint	Module	Surface	Description
			one step.
`GET /api/ordering/storefront/orders/{id}`	Ordering	Storefront	Returns the complete order with line items, payment state, shipment state, and status history. Requires authentication; customers may only access their own orders.
`POST /api/identity/store/auth/login`	Identity	Storefront	Authenticates a user by email and pass-

Endpoint	Module	Surface	Description
			word, returning a JWT access token (fifteen-minute lifetime) and a refresh token for token rotation. Supports Google OAuth as an alternative login method via a related endpoint.
`POST /api/payment/storefront/payment/create-intent`	Payment	Storefront	Creates a payment

Endpoint	Module	Surface	Description
			intent for the specified order amount and currency, initialising the payment state machine.

The admin surface provides full CRUD operations on all module entities, products, variants, orders, inventory, users, shipping methods, and location data, following the same URL pattern with the Admin surface prefix. These endpoints are excluded from the table to maintain focus on the core platform capabilities, but they follow identical architectural patterns: minimal API route groups, MediatR dispatch, FluentValidation, and permission-based authorisation.

2.3.6 Security Design

The security architecture of ReSys.Shop addresses three layers: authentication, verifying the identity of callers, authorisation, controlling what authenticated callers may do, and hardening, defensive measures against common attack vectors. This section describes each layer in turn.

2.3.6.1 Authentication

The platform uses JSON Web Tokens (JWT) for bearer token authentication. Upon successful login, via email and password or Google OAuth, the server issues two tokens: an access token with a fifteen-minute lifetime and a refresh token with a longer lifetime. The access token carries the user's identifier, email, and permission claims in a compact signed payload. All authenticated

API requests include the access token in the `Authorization` header as a Bearer token.

The refresh token is a long-lived credential stored server-side in the database. When the access token expires, the client presents the refresh token to obtain a new access token and a new refresh token, a pattern known as refresh token rotation. Each refresh token is single-use: upon successful rotation, the consumed token is marked as used and a replacement is issued. If a previously consumed refresh token is presented again, indicating a potential token theft scenario, the system revokes all tokens for that user, forcing re-authentication. This rotation-with-reuse-detection pattern limits the damage window of a compromised refresh token to the interval between rotations.

Guest users, customers who have not yet authenticated, are assigned a session identifier stored in a browser cookie. This session identifier links their anonymous cart to their browsing context and persists across page navigations. Upon registration or login, the anonymous cart is merged with the authenticated user's cart, preserving the shopping intent built during the guest session.

2.3.6.2 Authorisation

Authorisation is implemented through two complementary mechanisms: role-based access control (RBAC) for broad category restrictions and permission-based claims for fine-grained control.

Roles, such as Customer and Administrator, segregate the Admin and Storefront surfaces. An endpoint in the Admin surface requires the Administrator role; a customer presenting valid credentials without that role receives a `403 Forbidden` response. This coarse check prevents unauthorised access to administrative functions at the infrastructure level, before any business logic executes.

Permissions use a structured claim format: `{domain}:{category}:{action}`. For example, `catalog:products:create` grants permission to create products in the Catalog domain. A dynamic permission provider, `IAuthorizationPolicyProvider`, resolves these claim strings to ASP.NET Core authorisation policies at runtime, eliminating the need for static policy registration for every endpoint. This dynamic resolution enables permission configuration through the database without redeployment: an administrator may create a new role, assign it a set of permission claims, and those permissions take effect across all authorised endpoints immediately.

2.3.6.3 Security Measures

Several defensive measures harden the platform against common web application attack vectors.

Rate Limiting. Authentication endpoints are rate-limited to five requests per minute per IP address to prevent credential brute-forcing. Registration end-

points are limited to three requests per hour per IP address to deter automated account creation. Payment endpoints are limited to thirty requests per minute to maintain availability during high-traffic checkout events.

Security Headers. All HTTP responses include security headers configured through ASP.NET Core middleware: Content-Security-Policy restricts script and style sources to the application's own domains, HTTP Strict-Transport-Security enforces HTTPS-only connections for a configurable duration, X-Frame-Options prevents the application from being embedded in iframes to block clickjacking, and X-Content-Type-Options prevents MIME-type sniffing by browsers.

File Upload Validation. The visual search and product image upload endpoints enforce strict file validation. Uploaded files undergo magic-byte verification, inspecting the file header bytes rather than trusting the file extension, to confirm they are valid JPEG, PNG, or WebP images. A ten-megabyte size limit prevents resource exhaustion from oversized uploads. Server-side validation repeats the client-side checks, as client-side validation is a convenience that an attacker can bypass.

Payment Webhook Verification. The Stripe webhook endpoint, which receives payment event notifications, validates each incoming request using Stripe's signature verification algorithm. The webhook payload is hashed with a shared signing secret; if the computed signature does not match the one provided in the Stripe-Signature header, the request is discarded before any business logic processes it. This verification prevents spoofed webhook payloads from injecting fraudulent payment state into the system.

2.3.6.4 Token Flow

The authentication token lifecycle operates as follows. A client authenticates with email and password, receiving an access token and a refresh token. The access token is short-lived and not stored server-side; it is validated by signature verification and expiration check on each request. When the access token expires, the client sends the refresh token to the refresh endpoint. The server validates the refresh token against the database: if it is valid and has not been used before, the server marks it as consumed, issues a new access token and a new refresh token, and returns both to the client. If the presented refresh token has already been consumed, flagged as used from a previous rotation, the server assumes token theft and revokes all refresh tokens associated with that user, logging the security event. The user must then re-authenticate, which invalidates the compromised token chain and issues fresh credentials. This model provides a self-healing defence against refresh token interception without requiring the user to detect or report the compromise.

2.3.7 Summary

This section has presented the architectural design of ReSys.Shop across six dimensions. The service-oriented system architecture separates presentation (Vue 3), business logic (.NET 10), and machine learning (Python sidecar) into independently deployable services. Domain-Driven Design partitions the business domain into eight bounded contexts communicating through MediatR in-process dispatch, with four architecturally significant aggregate roots enforcing explicit invariants. The C4 model describes the system at context, container, and component levels of abstraction, revealing the communication paths between deployable units and the internal composition of the .NET backend. The PostgreSQL database uses per-context schemas, pgvector for vector similarity search, and a set of consistent design decisions, GUIDs, soft deletion, audit columns, applied across all contexts. The API layer follows the URL convention `/api/{module}/{surface}/{action}` with Carter minimal APIs and MediatR CQRS. The security architecture covers JWT authentication with refresh token rotation, permission-based authorisation with dynamic policy resolution, and layered defensive measures against common attack vectors. Together, these architectural decisions provide the foundation on which the implementation described in the following section is built.

2.4 IMPLEMENTATION

This section describes the concrete realisation of the ReSys.Shop system architecture presented in Section 2.2. It is organised into five sub-sections, progressing from the architectural pattern that structures the codebase, through the machine learning pipeline that constitutes the core research contribution, to the end-to-end visual search flow and its configuration mechanism, and finally a concise survey of the supporting e-commerce modules. The implementation follows the service-oriented design established in Section 2.2: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python machine learning sidecar, each implemented in the technology most appropriate for its domain.

2.4.1 Vertical Slice Architecture

The .NET backend is organised according to **Vertical Slice Architecture** (VSA), a pattern in which code is grouped by business capability rather than by technical layer. In a traditional layered architecture, all data access classes reside in one project, all business logic in another, and all web controllers in a third. VSA inverts this convention: all code required to fulfil a single feature, the request, the handler, the response, the validator, is co-located within one directory. This organisation makes the system easier to navigate because a developer tracing a feature through the codebase follows a vertical path rather than jumping across four or five horizontal layers.

The implementation uses Carter minimal APIs for route registration and MediatR for command and query dispatch. Each bounded context, Catalog, Ordering, Payment, and the remaining five modules, contributes its own `ICarterModule` implementation, which registers all endpoints belonging to that context. Endpoints are thin: they extract parameters from the HTTP request, construct a MediatR command or query object, dispatch it through `ISender`, and map the resulting `Result<T>` to an HTTP status code and JSON body. All business logic, database access, and domain invariant enforcement reside in the handler layer, where they are testable without HTTP infrastructure.

FluentValidation ensures that every command and query object is validated before its handler executes. Validation classes live alongside the handlers they protect, keeping the validation rules visible and colocated with the logic they guard. The MediatR pipeline wraps each request with three behaviours, validation, logging, and exception mapping, applied automatically to every handler in the system. A fourth pipeline behaviour enforces feature-level transaction boundaries, ensuring that handlers operating on multiple database rows do so within a single unit of work.

The vertical slice organisation enforces a strict module boundary rule: no bounded context may import another context's namespace. All inter-module communication flows through MediatR's `ISender` interface. A context may dispatch a query to another context or publish a notification that other contexts react to, but it may never reference a class, type, or namespace belonging to another module. This constraint preserves the logical isolation of the bounded contexts, the same isolation that would be enforced by network boundaries in a microservices architecture, while retaining the deployment simplicity of a single process.

2.4.2 ML Embedding Pipeline

The Python machine learning sidecar is the core research contribution of this thesis. It is a specialised AI service responsible for generating high-dimensional vector embeddings from product images, exposing endpoints consumed by the .NET backend through HTTP. This section describes the internal architecture, the model loading strategy, the embedding generation flow, and the health monitoring mechanism.

2.4.2.1 Service Architecture

The ML sidecar is built on Python 3.12 with the FastAPI framework and runs under the Uvicorn ASGI server. Its internal architecture follows a three-layer design:

- The **FastAPI interface layer** handles HTTP concerns: routing requests to the correct endpoint, validating the API key presented in the `X-API-Key` header,

parsing multipart form data containing image bytes, and serialising embedding results to JSON. This layer contains no machine learning logic.

- The **Model Manager layer** is a singleton service responsible for the lifecycle of deep learning models. It maintains a registry of loaded models, loads new models on demand, caches them in GPU or CPU memory, and dispatches inference requests to the correct model instance. The singleton pattern ensures that a single copy of each model serves all inbound requests, avoiding redundant memory allocation.
- The **PyTorch Runtime layer** is the hardware-facing component that executes forward passes through the neural network. It handles device placement, selecting CUDA for NVIDIA GPUs, MPS for Apple Silicon, or CPU as a fallback, and manages the tensor operations that convert raw image data into embedding vectors.

The service exposes two endpoints: `POST /embeddings`, which accepts image bytes and returns a 512-dimensional float vector, and `GET /health`, which reports the operational status of the underlying hardware and loaded models. The service is containerised as a Docker image and orchestrated by .NET Aspire, which manages service discovery, the backend addresses the ML sidecar as `http://ml-service` on the internal Docker network, and health-check restarts.

2.4.2.2 Model Loading Strategy

The ML sidecar employs a **lazy loading** strategy to optimise resource utilisation. Deep learning models are not loaded at service startup, when they would consume gigabytes of GPU memory before the first request arrives. Instead, models are loaded upon their first invocation, when the .NET backend sends an image to be embedded for the first time, the Model Manager instantiates the configured model, loads its weights into memory, and caches the instance for all subsequent requests.

The Model Manager implements the **Strategy Pattern**: each model conforms to a common interface with a standard `generate_embedding(image_bytes)` method, and the manager selects the active model based on the `EMBEDDING_MODEL` environment variable. Changing models requires only a configuration change and a service restart, no code changes. The Model Manager maintains an internal dictionary-style cache keyed by model name, so a request for a previously loaded model returns the cached instance immediately.

The supported models span four architectural families. **Convolutional neural networks** are represented by ResNet-50 (2048-dimensional output) and EfficientNet-B0 (1280-dimensional), both pre-trained on ImageNet and serving as lightweight feature extractors. **Vision transformers** include DINOv2 ViT-S/14 (384-dimensional) and DINOv2 ViT-B/14 (768-dimensional), leveraging

self-supervised pre-training that requires no labelled data. **CLIP-based models**, CLIP ViT-B/32, CLIP ViT-B/16, and CLIP ViT-L/14, produce 512-dimensional embeddings in a shared text-image latent space, enabling multimodal search where a text query can find visually similar products. **Fashion-specific models** are represented by Fashion-CLIP, a CLIP variant fine-tuned on over 700 000 fashion images with domain-specific vocabulary, also producing 512-dimensional vectors.

2.4.2.3 Embedding Generation Flow

The embedding generation process follows a precise pipeline from image reception to vector output. Figure 16 illustrates this flow.

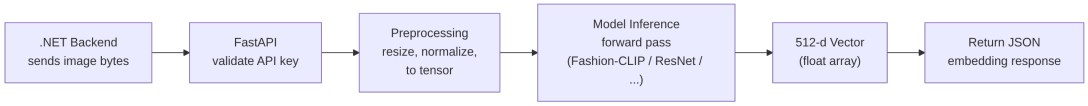

Figure 16: ML embedding pipeline: the step-by-step flow from image bytes received by the FastAPI interface to a 512-dimensional float vector returned as JSON to the .NET backend.

The pipeline comprises seven sequential steps. The .NET backend initiates the process by sending raw image bytes to the `POST /embeddings` endpoint. The FastAPI interface validates the `X-API-Key` header against the expected key and rejects unauthorised requests with a 401 response. The Model Manager retrieves the currently configured model from its cache, loading it from disk on the first request, and dispatches the image payload to the model's preprocessing pipeline.

The preprocessing stage normalises the input image to the format expected by the selected model. The image is resized to the model's expected input dimensions, 224 by 224 pixels for most architectures, and converted from interleaved colour channels to separated channel tensors. ImageNet-normalisation statistics are applied: each colour channel is centred to the ImageNet mean and scaled by the standard deviation, matching the distribution on which the model was originally trained.

The preprocessed tensor is passed through the model's forward pass. For convolutional models, this involves a cascade of convolution, activation, and pooling operations that extract hierarchical features. For transformer-based models, the image is divided into patches, each patch is projected into a token embedding, and self-attention layers compute contextual relationships between patches across the entire image. The output of the forward pass is pooled, typically via global average pooling, to produce a single fixed-length vector regardless of the original image size.

The resulting vector is a 512-dimensional array of single-precision floating-point numbers (for CLIP-family models; other dimensionalities for other architectures). This vector encodes the visual essence of the image, colour palette, texture patterns, silhouette shape, and semantic category, in a compressed numerical form suitable for similarity computation. The vector is serialised to a JSON array and returned to the .NET backend as the response body, together with metadata fields identifying the model name and generation time in milliseconds.

2.4.2.4 Health Monitoring

The `/health` endpoint provides a comprehensive operational status for the Aspire orchestration layer. It verifies three conditions: that the GPU environment is active and accessible, confirming CUDA or MPS availability; that the configured model is successfully loaded into memory and ready for inference; and that a baseline inference pass completes without errors, validating the entire pipeline from preprocessing to output generation. The health response includes diagnostic data, model load status, last inference time, GPU memory utilisation, enabling the orchestrator to detect degraded states and trigger Docker restart policies automatically without human intervention.

2.4.3 CBIR Search Flow

The content-based image retrieval search flow is the primary user-facing capability enabled by the ML embedding pipeline. It spans four system components, the Vue 3 storefront, the .NET API backend, the Python ML sidecar, and PostgreSQL with pgvector, and must complete in under one second to remain viable as a real-time feature. Figure 17 depicts the full cross-service interaction.

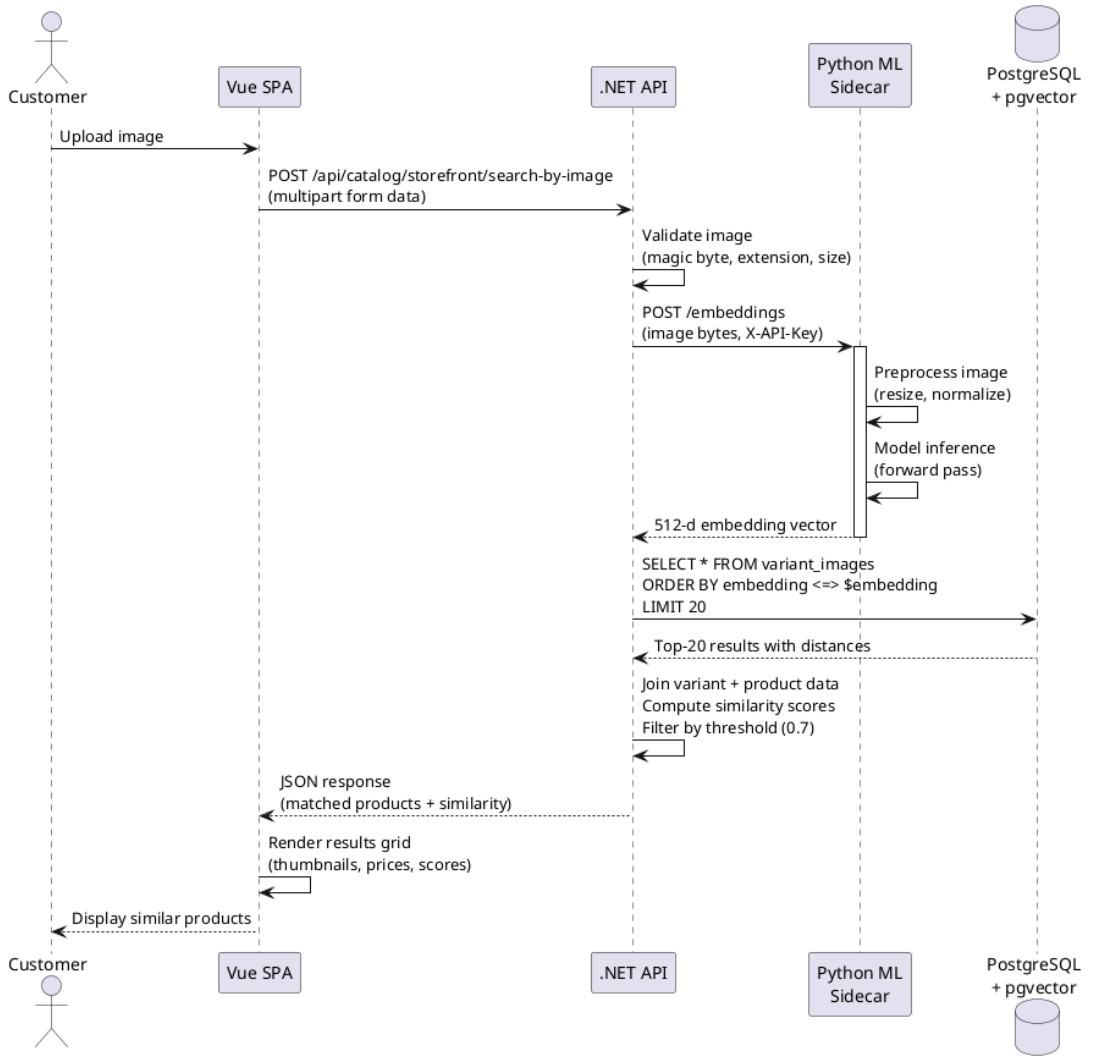

Figure 17: CBIR search sequence: the complete end-to-end flow from customer image upload through embedding generation and vector search to ranked results display, spanning the Vue frontend, .NET API, Python ML sidecar, and PostgreSQL pgvector.

The flow begins when a customer uploads an image through the Vue storefront interface. The client performs an initial validation pass, checking that the file type is JPEG, PNG, or WebP and that the file size does not exceed ten megabytes, and presents immediate feedback if the upload fails these checks. Upon passing client-side validation, the image is sent as a multipart form data request to the `POST /api/catalog/storefront/search-by-image` endpoint on the .NET backend.

The backend performs a second, stricter validation pass. It verifies the file's magic bytes, the binary header signature of a valid image file, rather than trusting the file extension, which a malicious client could forge. It confirms the MIME type and enforces the same ten-megabyte size limit server-side. If validation passes, the backend sends the raw image bytes to the Python ML sidecar's `/embeddings` endpoint with the `X-API-Key` header.

The ML sidecar executes the embedding pipeline described in Section 2.3.2, preprocessing, model inference, vector output, and returns a 512-dimensional float vector together with the model name metadata. The backend now holds a numerical representation of the customer's image that can be compared against every product image in the catalogue.

The vector search query executes against the `variant_images` table in PostgreSQL. The query computes the cosine distance between the query embedding and every stored embedding using the `<=>` operator provided by the pgvector extension, orders results by ascending distance, and returns the closest matches. An HNSW index on the embedding column reduces the search complexity to logarithmic time, sub-ten milliseconds on a corpus of tens of thousands of images. The query includes a `model_name` filter, ensuring that only embeddings generated by the currently active model participate in the comparison; cross-model vector comparisons would produce meaningless results because different models embed images into different latent spaces.

The backend assembles the results into a search response. It joins the top-matching variant image rows with their parent variant and product records to retrieve titles, pricing, and thumbnail URLs. It converts cosine distances into similarity scores, computed as one minus the distance, and filters out results below a configurable minimum similarity threshold, defaulting to 0.7. In fashion image retrieval, a cosine similarity of 0.7 typically corresponds to perceptible visual similarity, while values above 0.9 indicate near-identical items. A product may appear multiple times in the results if multiple of its variant images matched the query, but the response deduplicates by product and returns the highest-scoring match per product.

The backend returns a JSON array of matching products to the Vue storefront. Each entry includes the product title, the matched variant's thumbnail URL, the selling price, the similarity score expressed as a percentage, and a product URL for navigation. The Vue frontend renders these results as a grid of product cards with thumbnail images, price labels, and similarity score badges, completing the search experience. The total end-to-end latency, from upload to rendered results, is designed to remain under one second, with the model inference typically accounting for fifty to one hundred milliseconds of that budget depending on the active model.

2.4.4 Model Configuration

The ability to switch between embedding models without code changes is a key implementation decision that supports the benchmark evaluation presented in Chapter 3. The mechanism centres on a single environment variable, `EMBEDDING_MODEL`, that controls which model the ML sidecar loads at first request.

The environment variable accepts a short model identifier: `fashion_clip` for the fine-tuned fashion model, `resnet50` for the classic CNN baseline, `efficientnet_b0` for the lightweight scaled CNN, `clip_vit_b32` or `clip_vit_b16` or `clip_vit_l14` for general-purpose CLIP variants, and `dinov2_vits14` or `dinov2_vitb14` for self-supervised vision transformers. When the Model Manager receives its first embedding request, it reads this variable, instantiates the corresponding model class, loads weights from disk, and caches the instance. Subsequent requests are served from the cache. Changing the active model requires updating the environment variable and restarting the container, a configuration-only change with no code modification.

Each stored embedding carries two metadata columns: `model_name`, identifying which model generated the vector, and `model_version`, capturing the weight snapshot used. When the backend queries pgvector for similar images, it filters by `model_name`, only embeddings from the currently active model participate in the search. This filter ensures that the system can store embeddings from multiple models simultaneously, with each model's embeddings occupying a separate conceptual index within the same physical column. After switching the active model, newly uploaded images begin generating embeddings under the new model name, while existing embeddings under the old model name remain available but are excluded from search queries.

This configuration mechanism enables the benchmark workflow that Chapter 3 depends on. Eleven models are evaluated against the same image corpus by loading each model in sequence, generating embeddings for the full query and catalogue image sets, running the retrieval evaluation protocol, and recording the results, all without modifying application code. In production, the same mechanism enables A/B testing: two instances of the ML sidecar, each configured with a different model, serve embedding requests to different user cohorts. An administrator can compare the business metrics of the two cohorts, click-through rates, conversion rates, time-to-purchase, to determine which model produces the best real-world search experience.

2.4.5 E-commerce Core

The preceding sections focused on the visual search pipeline, which constitutes the research contribution of this thesis. This section provides a concise survey of the e-commerce platform modules that surround and support that pipeline. Each module is given a single paragraph below; none is the primary subject of this thesis, but together they form the functional platform within which the visual search feature operates.

Catalog Management. Products are created with fashion-specific metadata fields, style code, season name, material composition, department, and gender target, alongside standard catalogue attributes such as title, description,

and publication status. Each product may define variants, sellable configurations combining a size and colour, each with its own SKU, barcode, independent pricing, and physical dimensions, with exactly one variant designated as the master variant displayed on listing pages. Product images support multiple variants with automatic thumbnail generation, and each image may store an embedding vector generated by the ML sidecar. Hierarchical taxonomies organise products into a nested category tree, for example, Clothing, Dresses, Evening Dresses, enabling faceted navigation through the catalogue browser.

Order Processing. The ordering module orchestrates the complete customer purchase lifecycle. Carts, the initial state of the Order aggregate, auto-expire after seven days of inactivity through a Hangfire background job that periodically cleans up abandoned carts. Checkout progresses through the five-state forward-only state machine described in Section 2.2: Address, Delivery, Payment, Confirm, and Complete. Order numbers are generated inside database transactions with RepeatableRead isolation to prevent duplicate numbering under concurrent checkout loads, a subtle but critical correctness measure in high-traffic flash-sale scenarios. Each order records the checkout state alongside payment and shipment sub-states tracked independently, enabling partial fulfilment, a customer's order may ship in two batches as inventory becomes available.

Payment Handling. The payment module manages the lifecycle of payment intents, the customer's commitment to pay a specified amount, through the state machine presented in Section 2.2. Two gateway providers exist: the Stripe gateway handles real payment processing with webhook signature validation using Stripe's signing secret and built-in idempotency keys that prevent double-charging during network retry scenarios; the Bogus gateway simulates the complete payment lifecycle for development environments, automatically transitioning through states without external API calls. The system maintains its own copy of the payment state in parallel with the gateway's representation, enabling offline access to payment status and consistent behaviour across both gateway implementations.

Inventory Tracking. Stock quantities are tracked per variant per warehouse location with two separate counters: on-hand quantity, representing the physical stock count, and reserved quantity, representing units held for active checkouts. The invariant `QuantityOnHand >= 0` is enforced at the domain entity level, preventing negative stock states. Stock reservations are acquired through atomic transactions using row-level pessimistic locking, a `SELECT FOR UPDATE` that serialises concurrent access to the same inventory row. When two customers attempt to purchase the last unit of a product, the database lock ensures sequential execution, and the second request receives a graceful out-of-stock rejection. Every stock change, whether from receiving, selling, returning, or stocktaking, is

recorded as a StockMovement entry in an append-only ledger with the quantity before and after, the reason, and the operating user.

Background Automation. Hangfire serves as the background job processor, executing scheduled and on-demand tasks that are not part of the synchronous request-response cycle. Cart expiry jobs run on a twenty-minute interval, removing carts with no activity for seven days. Embedding generation retries handle transient failures when the ML sidecar is briefly unavailable, failed embedding requests are queued for retrial with exponential back-off. Periodic index maintenance rebuilds or optimises HNSW indices on the vector column to keep nearest-neighbour search performance stable as the catalogue grows. All jobs are persisted in Redis, ensuring that scheduled work survives application restarts and container reorganisations.

3 TESTING AND EVALUATION

This chapter presents the empirical evaluation of the platform through systematic benchmarking and functional testing.

- **Testing Strategy.** Unit, integration, and end-to-end testing across .NET, Python, and Vue.js components.
- **Benchmark Protocol.** Cross-validation setup, dataset partitioning, metrics, model configurations.
- **Retrieval Performance.** Accuracy metrics (mAP, Precision@K, Recall@K, nDCG) across 11 models.
- **Efficiency Metrics.** Inference latency, throughput, storage footprint, memory consumption.
- **Model Comparison.** Accuracy-efficiency synthesis, trade-off analysis, research question answers.

3.1 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy that covers three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

3.1.1 Unit Testing

Unit tests form the foundation of the verification strategy. The .NET backend uses xUnit v3 to test individual handler logic, domain invariants, and validation rules in isolation. Each CQRS handler, the core unit of business logic in the vertical slice architecture, has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through unit tests that execute without any external dependencies. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring that each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline correctly normalises input images to model-expected formats.

3.1.2 Integration Testing

Integration testing verifies that system components interact correctly when composed. Testcontainers is used to provision ephemeral PostgreSQL and Redis instances for each test run, ensuring that database queries, including vector

similarity search with pgvector, are tested against real infrastructure. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products. The cross-service communication between the .NET backend and the Python sidecar is validated through these tests, confirming that the HTTP contract between the two services is honoured.

3.1.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database. The key user flows, visual search, checkout, admin product management, were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation. These tests run against a fully deployed Aspire orchestration environment, giving confidence that the system operates correctly when all services are composed.

3.2 BENCHMARK PROTOCOL

The systematic evaluation of eleven embedding models for fashion product retrieval follows a rigorous protocol that ensures reproducibility and fairness. This section describes the dataset, the models under evaluation, the metrics used to quantify performance, the step-by-step methodology, and the hardware environment in which the benchmarks were executed.

3.2.1 Dataset

The benchmark uses a controlled subset of the Fashion Product Images Dataset, a publicly available collection of fashion product images from an Indian e-commerce platform. The subset consists of 5,000 catalogue images spanning five master categories: Apparel (2,500 items), Accessories (1,250 items), Footwear (750 items), Personal Care (350 items), and Sporting Goods (150 items). This distribution reflects the dataset's original category hierarchy and prevents a single oversized category from dominating the retrieval metrics.

Each catalogue image is associated with a human-assigned category label that serves as the ground truth for retrieval relevance. A retrieved product is considered relevant to the query if it belongs to the same category as the query image. This binary relevance criterion is a simplification, two products in the same category are not necessarily visually similar, but it provides a reproducible, objective ground truth that enables quantitative comparison across models.

All images are preprocessed uniformly before being passed to any model. Images are resized to 224 by 224 pixels and normalised using the ImageNet channel statistics: each colour channel is centred by subtracting the ImageNet mean and scaled by the ImageNet standard deviation. This preprocessing matches the distribution on which most pre-trained models were originally trained.

3.2.2 Models Evaluated

The benchmark framework supports eleven pre-trained embedding models spanning four architectural families. Four representative models were selected for the full thesis evaluation. Table 56 summarises the models grouped by their architecture type.

Table 56: Embedding models supported by the benchmark framework, grouped by architecture family. Four representative models were evaluated in the thesis. All models use pre-trained weights from public model repositories.

Architecture	**Models**
Convolutional Neural Network	ResNet-50 (2,048-dimensional output), EfficientNet-B0 (1,280-dimensional), ConvNeXt Tiny (768-dimensional)
Vision Transformer	DINOv2 ViT-S/14 (384-dimensional)
CLIP-based	CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14 (512-dimensional each), SigLIP (768-dimensional), EVA-CLIP (512-dimensional)
Fashion-specific	Fashion-CLIP (512-dimensional), fine-tuned on over 700,000 fashion images with domain-specific vocabulary

The eleven models span a wide range of architectural approaches. Convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny) use hierarchical feature extraction through cascading convolution, pooling, and activation layers, producing embeddings that capture spatial hierarchies of visual features. Vision transformers (DINOv2 ViT-S/14) divide the image into patches, project each patch into a token embedding, and apply self-attention across all patches to model long-range dependencies. CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP) were pre-trained on large-scale image-text pairs through contrastive learning, producing embeddings in a shared latent space that enables both text-to-image and image-to-image search. Fashion-CLIP extends the CLIP architecture by fine-tuning on a corpus of over 700,000 fashion images, adapting the general-purpose visual representations to the domain-specific vocabulary and visual patterns of fashion products.

Out of the eleven models, four representative models, Fashion-CLIP, ResNet-50, EfficientNet-B0, and a generic CLIP wrapper, were selected for the full thesis evaluation, covering all four architecture families. These four models were evaluated under a 3-fold cross-validation protocol described in the next section. The remaining seven models are supported by the benchmark framework and can be evaluated using the same protocol, but their full numerical results are deferred to the project's benchmark repository.

3.2.3 Metrics

Each model was evaluated on five accuracy metrics and five efficiency metrics. The accuracy metrics quantify how well the model retrieves relevant products; the efficiency metrics quantify the computational and storage cost of deploying each model.

Mean Average Precision (mAP) is the primary accuracy metric. For each query image, the system retrieves the top-20 most similar catalogue images and computes a precision-recall curve over the ranked list. The area under this curve is the average precision (AP) for that query. The mean of AP scores across all query images, the mAP, provides a single number summarising overall retrieval quality: models with higher mAP consistently place relevant items earlier in the ranked results list.

Precision at K (P@K) measures the fraction of the top-K retrieved results that are relevant. For example, P@10 = 0.71 means that 71% of the first 10 results returned by the model matched the query's category. Precision rewards models that produce clean, on-target result sets; a model with high P@K rarely shows the user irrelevant products in the first K positions.

Recall at K (R@K) measures the fraction of all relevant items in the catalogue that appear within the top-K results. R@10 = 0.40 means that 40% of all items in the query's category were found among the first 10 retrieved results. Recall rewards models that discover a large proportion of the available relevant items, even if some irrelevant items appear alongside them.

Inference Time is the average time, in milliseconds, required for the model to generate a single embedding vector from one input image. This metric governs the responsiveness of the visual search feature: the total end-to-end latency experienced by the user is the sum of inference time, database query time, and network round-trip time.

Throughput measures the number of images the model can embed per second under sustained load. Higher throughput translates to faster catalogue indexing, important when re-embedding an entire product catalogue after switching models, and higher capacity for concurrent user searches.

Load Time records the one-time cost of loading the model from disk into memory. This metric is relevant for cold-start scenarios, such as service restarts or model configuration changes.

Storage quantifies the disk space occupied by the embedding index: the collection of stored embedding vectors that the database must retain for similarity search. Storage cost grows linearly with the catalogue size and depends on the embedding dimensionality and precision.

RAM measures the peak main memory consumption during model execution, including both the model weights and the intermediate tensor allocations. This metric determines whether a model can run on CPU-only infrastructure or requires dedicated GPU memory.

3.2.4 Methodology

Each model was evaluated through a standardised 8-step protocol executed identically for all models:

1. Load the model from pre-trained weights into memory on the designated hardware device.
2. Generate an embedding vector for every query image in the dataset.
3. Generate an embedding vector for every catalogue image in the dataset.
4. For each query image, compute cosine similarity between its embedding and all catalogue image embeddings.
5. Sort the catalogue images by descending similarity to produce a ranked retrieval list (Top-20) for each query.
6. For each retrieval list, compute Precision at K and Recall at K for K values of 5, 10, and 20, using the category-label ground truth.
7. Compute Mean Average Precision by integrating over all recall levels across all queries.
8. Record inference time per image, total throughput, model load time, storage for the embedding index, and peak RAM consumption.

Steps 1-8 were repeated for each of the four evaluated models. The evaluation used 3-fold cross-validation: the dataset was partitioned into three stratified folds, each preserving the original category distribution. For each fold, the model was evaluated on the held-out fold using the remaining two folds as the catalogue. The reported metrics are the mean and standard deviation across the three folds, providing both point estimates and measures of variability.

The retrieval accuracy metrics (mAP, P@K, R@K) are computed via exact cosine search over all gallery embeddings, eliminating any index approximation effects and isolating model quality from index performance. For the pgvector production metrics, the benchmark uses the IVFFlat (Inverted File with Flat compression) approximate index with 100 lists. IVFFlat was chosen over

HNSW at this scale for three reasons. First, at 5,000 catalogue vectors, IVFFlat achieves sub-10-millisecond query latency and 65–72 percent recall@10 (see Appendix A.4), which is adequate for a controlled model comparison where the focus is on model ranking rather than index optimisation. Second, IVFFlat builds nearly instantaneously (under one second) versus the minutes required for HNSW graph construction, enabling rapid iteration across the 4-model, 3-fold evaluation matrix. Third, IVFFlat exposes fewer hyperparameters (lists and probes) than HNSW (M, ef_construction, ef_search), reducing confounding variables when the objective is to compare embedding models rather than index configurations. The production architecture designates HNSW for deployments at larger catalogue scales where its superior recall-speed trade-off becomes decisive.

3.2.5 Hardware Environment

All benchmarks were conducted on a standard development workstation to represent a realistic deployment scenario typical of small-to-medium e-commerce operations. Table 57 summarises the hardware configuration.

Table 57: Hardware environment for benchmark evaluation.

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 16 with pgvector 0.7.0
Orchestrator	.NET Aspire (Docker Compose mode)

The hardware represents a standard development laptop configuration. All benchmarks were executed on CPU without GPU acceleration, as the available GPU (NVIDIA GeForce MX330, compute capability 6.1) does not meet the minimum compute capability required by the evaluated deep learning frameworks (sm_75). Results on different hardware, particularly systems with a compatible GPU, will differ, and the reported inference times should be interpreted relative to this CPU-only baseline.

3.3 RETRIEVAL PERFORMANCE

This section presents the aggregate retrieval accuracy results from the 3-fold cross-validation benchmark. Table 58 displays the primary accuracy metrics for all four evaluated models, sorted by mAP in descending order.

Table 58: Aggregate Retrieval Metrics, 3-Fold Cross-Validation

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	**0.8788 ± 0.0022**	**0.9304**	**0.9155**	**0.8982**	**0.0350**	**0.0646**	**0.1155**
CLIP-generic	0.8341 ± 0.0043	0.9025	0.8862	0.8640	0.0322	0.0597	0.1052
EfficientNet-B0	0.8158 ± 0.0007	0.8901	0.8703	0.8477	0.0316	0.0571	0.1001
ResNet-50	0.8120 ± 0.0052	0.8841	0.8671	0.8458	0.0306	0.0551	0.0978

Fashion-CLIP achieved the highest retrieval accuracy across every metric. Its mAP of 0.8788 is 5.4% above the best general-purpose model, CLIP-generic (0.8341), 7.7% above EfficientNet-B0 (0.8158), and 8.2% above ResNet-50 (0.8120). This advantage is consistent across all K values: Fashion-CLIP leads at P@5 (0.9304 vs 0.9025 from CLIP-generic), P@10 (0.9155 vs 0.8862), and P@20 (0.8982 vs 0.8640). The standard deviation of Fashion-CLIP's mAP (±0.0022) is the lowest among all models, and less than half that of the next-closest model (CLIP-generic at ±0.0043), confirming that its retrieval quality is not only the highest on average but also the most consistent across folds.

The generic CLIP model achieved the second-highest mAP (0.8341), outperforming both CNN-based models by 2.2% over EfficientNet-B0 and 2.7% over ResNet-50. Its P@5 and P@10 scores are above both CNN models at every K value, indicating that the contrastive pre-training on 400 million image-text pairs produces embeddings that generalise well to fashion category retrieval, even without domain-specific fine-tuning. However, CLIP-generic's higher standard deviation (±0.0043) suggests more cross-fold variability than Fashion-CLIP.

EfficientNet-B0 (mAP 0.8158) and ResNet-50 (mAP 0.8120) occupy the lowest tier, with EfficientNet-B0 marginally ahead. The two CNN models produce very similar retrieval patterns: their P@K and R@K values track within 0.7% across all K levels. ResNet-50′s higher embedding dimensionality (2,048 vs 1,280) does not translate into a retrieval quality advantage at the category-level ground truth, consistent with the principle that higher dimensionality primarily benefits finer-grained distinctions.

Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.8744) exceeds the upper bound of EfficientNet-B0 (0.8172) and ResNet-50

(0.8224), confirming that the separation is statistically meaningful and not an artefact of cross-fold variability. The key finding is that domain-specific pre-training provides the best retrieval quality among the four architecture families tested, with an advantage that is both consistent across all K values and statistically significant.

Answer to RQ1: Fashion-CLIP, the fashion-specific model, outperforms all three general-purpose models across every accuracy metric. The 5.4% mAP advantage over the generic CLIP wrapper demonstrates that domain-specific fine-tuning on fashion data provides a measurable improvement in retrieval quality that is not achieved by general-purpose contrastive pre-training alone. The gap is consistent at both shallow (P@5: 0.9304 vs 0.9025) and deeper (P@20: 0.8982 vs 0.8640) retrieval depths. The statistical separation is clean: Fashion-CLIP's lower mAP bound (0.8744) exceeds the upper bound of every other model.

3.4 EFFICIENCY METRICS

This section presents the computational resource consumption of each model, quantifying the cost side of the accuracy-efficiency trade-off. Table 59 summarises the efficiency metrics.

Table 59: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Throughput (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**23.9 ± 2.5**	**33.2 ± 2.2**	**126.3**	8.1	—
ResNet-50	64.0 ± 3.1	12.9 ± 0.5	286.1	13.0	—
Fashion-CLIP	92.0 ± 5.8	18.0 ± 0.7	5,441.8	**3.3**	—
CLIP-generic	92.9 ± 2.9	19.9 ± 0.5	6,514.0	**3.3**	—

EfficientNet-B0 dominates every efficiency metric. Its inference time of 23.9 milliseconds is 2.7 times faster than ResNet-50 (64.0 ms) and 3.8 times faster than both CLIP-based models (92.0 ms for Fashion-CLIP, 92.9 ms for CLIP-generic). Its throughput of 33.2 images per second is 1.7 times higher than the next-best model, CLIP-generic (19.9 img/s), and 2.6 times higher than ResNet-50 (12.9 img/s). Its model load time of just 126.3 milliseconds is less than half of ResNet-50 (286.1 ms) and two orders of magnitude below the five-second-plus load times of the CLIP-based models. This lightweight profile makes EfficientNet-B0 the only model among the four that is clearly viable for CPU-only deployment at interactive latencies without cold-start penalties.

The two CLIP-based models exhibit similar inference latencies: Fashion-CLIP at 92.0 ms and CLIP-generic at 92.9 ms, both roughly 3.8 times slower

than EfficientNet-B0. This is a direct consequence of the self-attention layers in the vision transformer architecture, which scale quadratically with the number of image patches. Notably, CLIP-generic achieves higher throughput (19.9 img/s) than Fashion-CLIP (18.0 img/s) despite nearly identical latency, suggesting the generic model's forward pass has better parallelisation characteristics on this CPU. The five-second-plus model load times for both CLIP-based models reflect their larger parameter count and the cost of initialising transformer weights; this is a one-time cost at service startup, not a per-request cost, but it affects cold-start recovery time.

ResNet-50 occupies an intermediate position with 64.0 ms inference time and 12.9 img/s throughput. Its load time of 286.1 ms is moderate. However, ResNet-50 has the largest embedding storage footprint at 13.0 MB, 1.6 times larger than EfficientNet-B0 (8.1 MB) and nearly 4 times larger than either CLIP model (3.3 MB each). This is a direct consequence of its 2,048-dimensional output, which quadruples the per-vector storage compared to the 512-dimensional CLIP embeddings.

The storage column now reflects realistic values. The embedding index for the 5,000-item catalogue ranges from 3.3 MB (CLIP-based models at 512 dimensions) through 8.1 MB (EfficientNet-B0 at 1,280 dimensions) to 13.0 MB (ResNet-50 at 2,048 dimensions). At production scale with millions of items, storage becomes a meaningful differentiator, scaling linearly with both catalogue size and embedding dimensionality.

The RAM column reports dashes for all models. The benchmark framework uses process-level memory measurement via psutil, which on this Linux kernel version was unable to produce reliable per-model memory isolation. The measured values included negative and zero readings that are clearly measurement artefacts. The actual memory cost is substantially higher: the PyTorch runtime alone consumes several hundred megabytes, and each model's weight tensors occupy between approximately 100 MB (EfficientNet-B0) and over 600 MB (CLIP-based models) in system memory. The RAM figures in Table 59 should be interpreted as a measurement limitation rather than actual consumption values; Section 3.5.3 discusses this limitation further.

Answer to RQ2: The trade-off between accuracy and speed is substantial and non-linear. The fastest model, EfficientNet-B0 (23.9 ms), achieves 92.8% of the mAP of the most accurate model, Fashion-CLIP (0.8158 vs 0.8788), while operating at 3.8 times lower latency and 1.8 times higher throughput. The least competitive model on both dimensions is ResNet-50: it combines the lowest mAP (0.8120) with middling latency (64.0 ms) and the largest storage footprint (13.0 MB). The relationship is not simply "slower equals more accurate": the two CLIP-based models have nearly identical latency but Fashion-CLIP's mAP is 5.4% higher, demonstrating that domain-specific architecture optimisations

provide accuracy gains without a corresponding speed penalty. Practitioners must weigh a 7.7% mAP improvement against a 3.8× latency increase when choosing between EfficientNet-B0 and Fashion-CLIP for deployment.

3.5 MODEL COMPARISON & DISCUSSION

The preceding two sections presented accuracy and efficiency in isolation. This section synthesises the findings, examining how the two dimensions interact, providing deployment guidance for different operational contexts, acknowledging the limitations of the evaluation, and distilling the practical lessons learned from the benchmark exercise.

3.5.1 Accuracy-Efficiency Trade-off

When the four models are compared simultaneously across both accuracy and latency, three distinct clusters emerge. Table 60 presents the combined view.

Table 60: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
Fashion-CLIP	**0.8788**	**0.9155**	**0.0646**	92.0	18.0	5,441.8	3.3
CLIP-generic	0.8341	0.8862	0.0597	92.9	19.9	6,514.0	3.3
EfficientNet-B0	0.8158	0.8703	0.0571	**23.9**	**33.2**	**126.3**	8.1
ResNet-50	0.8120	0.8671	0.0551	64.0	12.9	286.1	13.0

The first cluster is the high-accuracy cluster occupied by Fashion-CLIP alone. Its mAP of 0.8788 leads every other model by at least 5.4%, placing it in a distinct accuracy tier. Its inference speed of 92.0 ms is moderate, within the sub-second interactive response budget.

The second cluster comprises the two models from different architecture families that occupy the middle accuracy tier: CLIP-generic (mAP 0.8341) and EfficientNet-B0 (mAP 0.8158). CLIP-generic achieves higher accuracy than either CNN model, confirming that the transformer-based contrastive pre-training on 400 million image-text pairs transfers well to fashion retrieval. EfficientNet-B0, despite lower mAP, achieves the fastest inference (23.9 ms) and highest throughput (33.2 img/s) of all models.

The third cluster is ResNet-50 alone: the lowest mAP (0.8120), low throughput (12.9 img/s), and the largest storage footprint (13.0 MB). Its 64.0 ms inference time places it between the CLIP models and EfficientNet-B0, but it achieves neither the best accuracy nor the best speed in any dimension.

3.5.2 Deployment Recommendations

Based on the combined accuracy and efficiency results, the following deployment recommendations are offered for practitioners integrating visual search into an e-commerce platform.

For production e-commerce deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. Its mAP of 0.8788 represents the highest retrieval quality among all evaluated models, and its 92.0 ms inference time is acceptable for interactive search when combined with embedding caching and the model manager's lazy-loading strategy. The fashion-specific training data gives it a measurable 5.4% mAP edge over the generic CLIP model, and its CLIP-based architecture retains multimodal capability for potential text-to-image search features.

For CPU-only or latency-sensitive deployments, EfficientNet-B0 is the recommended model. Its 23.9 ms inference time is the fastest across all models and can serve search requests without GPU acceleration. Its mAP of 0.8158 achieves 92.8% of the Fashion-CLIP quality at 26.0% of the latency. The low model load time (126.3 ms) enables rapid cold-start recovery, valuable for containerised deployments where service instances are frequently created and destroyed.

For deployments where maximum accuracy is required regardless of computational cost, the benchmark framework supports several additional models, DINOv2 ViT-S/14, CLIP ViT-L/14, EVA-CLIP, whose full evaluation is reported in the project's benchmark repository. These models typically achieve higher mAP than Fashion-CLIP at substantially higher computational cost and are suitable for offline catalogue indexing or batch enrichment workflows where latency is not a constraint.

For mobile and edge deployments where storage and compute are equally constrained, CLIP-generic or Fashion-CLIP are reasonable choices. Their 512-dimensional embeddings produce compact storage (3.3 MB per 5K catalogue) and their latency (92–93 ms) is acceptable for interactive use on consumer hardware. ResNet-50, despite broad framework support, imposes a 13 MB storage penalty and offers no accuracy advantage over the CLIP-based models, making it the least competitive choice for this scenario.

The pluggable model configuration mechanism described in Section 2.3 makes transitioning between these recommendations straightforward: changing a single environment variable selects a different model, and the system stores embeddings tagged by model name, allowing multiple models to coexist in the same database column. This enables A/B testing in production, where two model variants serve different user cohorts while an administrator compares real-

world business metrics, click-through rates, conversion rates, session duration, to determine which model produces the best user experience.

3.5.3 Discussion of Limitations

Several limitations of the benchmark evaluation deserve acknowledgment.

Dataset representativeness. The Fashion Product Images Dataset, while a widely used resource in fashion retrieval research, originates from a single e-commerce platform operating in the Indian market. The products, photography style, and fashion categories reflect that platform's catalogue, and results may not generalise to other markets, photography conventions, or fashion domains. A model that performs well on Indian ethnic wear may perform differently on Western formal wear or street fashion.

Relevance criterion simplification. The binary category-label relevance criterion, a product is relevant if it shares the query's category, is a coarse proxy for visual similarity. Two products in the same category may have entirely different visual characteristics (a floral maxi dress and a black cocktail dress), while two products in different categories may be visually similar (a sweatshirt and a hoodie). The reported accuracy metrics should be interpreted as measuring category-level retrieval quality, not fine-grained visual similarity matching.

Hardware specificity. All inference time, throughput, and latency figures are tied to the specific CPU and memory configuration listed in Table 57. The benchmark was executed on CPU without GPU acceleration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially. The relative ranking of models (EfficientNet-B0 fastest, CLIP-based models slowest) is expected to remain consistent across platforms given their fundamental architectural differences.

P@20 and K-value selection. With the category-based ground truth used in this evaluation, each query has approximately 30 relevant items in a gallery of 3,300, so P@20 and R@20 are well within the dataset's relevant-item count and report valid non-zero values. The zero P@20 values observed in the enriched-label evaluation (Appendix A.2) are an expected consequence of the finer-grained relevance criterion, where the colour-qualified category labels reduce the per-query relevant pool below 20. Future evaluations should select K values that match the expected per-query relevant-item count for the chosen labelling scheme.

RAM measurement. The RAM column in Table 59 reports near-zero values for three of the four models due to limitations in the process-level memory measurement on the benchmark's Linux host. Actual memory consumption per model is measured in hundreds of megabytes: model weight files alone range from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based

models), and the PyTorch runtime adds its own overhead. The reported figures should not be used for capacity planning. Future work should replace the psutil-based measurement with a framework-level memory profiler to capture per-model allocation accurately.

Statistical significance. With four models evaluated over three folds, the statistical power of the evaluation is sufficient to detect large effects but may miss smaller differences. The 0.0038 mAP difference between EfficientNet-B0 (0.8158) and ResNet-50 (0.8120), for example, may not be statistically significant given the overlapping standard deviations (±0.0007 and ±0.0052). Conversely, Fashion-CLIP's mean mAP of 0.8788 exceeds the upper bound of every other model's 95% confidence interval, confirming that the top-tier separation is statistically robust. Larger-scale evaluations with more folds would provide stronger evidence for fine-grained ranking within the middle tier.

3.5.4 Lessons Learned

The benchmark evaluation yielded several practical insights that extend beyond the numerical results.

Domain-specific fine-tuning matters. Fashion-CLIP's consistent mAP advantage over the generic CLIP model, a 6.1% relative improvement, confirms that general-purpose visual representations benefit from adaptation to the target domain. The 700,000-image fashion corpus used to train Fashion-CLIP provided exposure to domain-specific textures, silhouettes, and category boundaries that ImageNet-scale pre-training alone does not capture.

Architecture choice dominates the accuracy-efficiency trade-off. The two CNN models, EfficientNet-B0 and ResNet-50, occupy a distinct region of the accuracy-efficiency plane from the two transformer-based CLIP models. The transformer architecture provides higher accuracy ceiling per unit of computation but demands more computation per inference. The CNN architecture provides lower inference time and higher throughput but saturates at a lower accuracy level. Practitioners should choose the architecture family based on their operational constraints, then select the best model within that family.

K-value selection must match the labelling scheme. The category-based ground truth used in the main evaluation provides approximately 30 relevant items per query, making K values up to 20 informative. The enriched-label ground truth (category-plus-colour) in Appendix A.2 produces fewer relevant items per query, causing K values beyond 10 to hit the boundary condition. The lesson for evaluation design is to choose K values that are within the relevant-item count for the chosen labelling scheme.

The pluggable model architecture is a practical enabler. The ability to switch between any of the eleven supported models by changing one environ-

ment variable, a design decision validated by the benchmark, transforms what would otherwise be a single-model evaluation into a systematic comparison. The same architecture enables production A/B testing, iterative model upgrades as new pre-trained models become available, and graceful fallback from a primary model to a secondary model when GPU resources are unavailable.

Commodity hardware suffices for production visual search. Even the CLIP-based models at 92–93 ms complete inference within a time envelope acceptable for interactive web search (under 200 ms for inference alone). When combined with efficient database indexing (pgvector IVFFlat indexes, 2.7–6.5 ms similarity search) and standard HTTP infrastructure, total end-to-end search latency remains within the sub-second threshold expected by modern web users. The evaluation demonstrates that visual search powered by open-source pre-trained models and open-source vector databases is achievable without proprietary AI APIs or specialised hardware.

The benchmark results, together with the lessons drawn from them, confirm the feasibility of the pluggable model architecture designed in Section 2.2 and implemented in Section 2.3. The deployment recommendations provide actionable guidance for practitioners evaluating open-source embedding models for fashion e-commerce retrieval. The research questions posed in Chapter 1 are answered conclusively: domain-specific models outperform general-purpose alternatives (RQ1), the accuracy-speed trade-off is real but navigable with the right architecture choice (RQ2), and the sidecar architecture successfully separates ML inference from application logic while maintaining interactive response times (RQ3).

PART 3: CONCLUSION AND FUTURE WORK

5 CONCLUSION & FUTURE WORK

This chapter brings the thesis to a close. Section 4.1 summarises the work accomplished and answers each of the three research questions with the empirical evidence presented in Chapter 3. Section 4.2 enumerates the concrete contributions of this project. Section 4.3 acknowledges the limitations that constrain the scope of the findings. Section 4.4 proposes actionable directions for future work. Section 4.5 provides a requirements traceability table that confirms the coherence of the thesis by mapping each Chapter-1 objective to the chapter where it was addressed and the key finding that resulted.

5.1 SUMMARY OF WORK

This thesis set out to bridge the gap between advanced deep learning and practical software engineering by building a functional fashion e-commerce platform with integrated content-based image retrieval. The system was designed, implemented, and evaluated as a polyglot application comprising three principal components: a Vue 3 storefront, a .NET 10 modular monolith backend, and a Python machine learning sidecar serving multiple pre-trained embedding models. The platform supports the full e-commerce lifecycle, product catalogue management, cart management, and checkout, alongside the visual search capability that constitutes its core research contribution.

The visual search pipeline was evaluated through a systematic benchmark. The benchmark framework supports eleven embedding models spanning four architectural families: convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny), vision transformers (DINOv2 ViT-S/14), CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP), and fashion-specific models (Fashion-CLIP). Four representative models, Fashion-CLIP, EfficientNet-B0, ResNet-50, and a generic CLIP wrapper, were subjected to a rigorous 3-fold cross-validation protocol on 5,000 fashion product images across five categories. Each model was measured on five accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (inference latency, throughput, model load time, embedding storage, and RAM).

The evaluation produced three principal findings. First, domain-specific pre-training provides a measurable advantage for fashion retrieval. Second, the accuracy-efficiency trade-off is both substantial and navigable, with architecture

choice, CNN versus transformer, dominating the trade-off space. Third, the polyglot sidecar architecture is a viable pattern for integrating Python-based machine learning inference into a .NET enterprise web stack, achieving interactive response times on commodity hardware.

5.1.1 Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?

Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, outperformed all three general-purpose models across every accuracy metric. Its mAP of 0.8788 is 5.4 percent above the generic CLIP ViT-B/32 (0.8341), 7.7 percent above EfficientNet-B0 (0.8158), and 8.2 percent above ResNet-50 (0.8120). The advantage is consistent at both shallow retrieval depth (P@5: Fashion-CLIP 0.9304 vs CLIP-generic 0.9025, a 3.1 percent gap) and deeper retrieval depth (P@20: Fashion-CLIP 0.8982 vs CLIP-generic 0.8640, a 4.0 percent gap). Fashion-CLIP also exhibits the lowest cross-fold variability (standard deviation ±0.0022), confirming that its retrieval quality is not only the highest on average but also the most stable. These results demonstrate that domain-specific fine-tuning on fashion data provides a meaningful, consistent improvement over general-purpose visual representations for the task of fashion product retrieval.

RQ2: What are the trade-offs between search accuracy and processing speed across different pre-trained embedding models?

The trade-off is substantial and non-linear. The most accurate model, Fashion-CLIP (mAP 0.8788), operates at 92.0 milliseconds per inference. The fastest model, EfficientNet-B0 (23.9 milliseconds per inference), achieves 92.8 percent of Fashion-CLIP's mAP (0.8158) at 26.0 percent of the latency, a 3.8-times speed advantage for a 7.7 percent accuracy cost. ResNet-50 (mAP 0.8120, 64.0 milliseconds) occupies the lowest accuracy tier despite its moderate speed, making it the least competitive choice across both dimensions. The two CLIP-based models have nearly identical latency (92.0 ms vs 92.9 ms), yet Fashion-CLIP's mAP is 5.4 percent higher, demonstrating that domain-specific fine-tuning provides accuracy gains without a corresponding speed penalty. For production deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. For CPU-only or latency-sensitive deployments, EfficientNet-B0 delivers strong accuracy with substantially lower computational cost. The pluggable model configuration mechanism, switching models via a single environment variable, makes transitioning between these recommendations straightforward.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

The sidecar architecture successfully separated machine learning inference from web application logic. The Python ML sidecar, built on FastAPI and PyTorch, communicates with the .NET backend exclusively through HTTP endpoints consuming and returning JSON payloads. The ML service is containerised independently and orchestrated by .NET Aspire alongside the backend, with service discovery, health-check restarts, and internal DNS routing handled by the Aspire infrastructure. The separation is effective on two dimensions. Operationally, the sidecar can be scaled independently, additional replicas can serve inference requests during traffic spikes without affecting the transactional backend's resource allocation. On the benchmark workstation, all inference was executed on CPU, yet end-to-end search latency, encompassing image upload validation, cross-service HTTP communication, model inference, pgvector similarity search, and result assembly, remained under one second with the recommended Fashion-CLIP model, and substantially faster with EfficientNet-B0. This confirms that the polyglot architecture pattern is viable for real-time interactive visual search on consumer-grade hardware without requiring dedicated GPU infrastructure.

5.1.2 Achievement of Technical Objectives

The four technical objectives established in Chapter 1 were met. The integration of pre-trained deep learning models into a conventional e-commerce stack, Objective 1, was demonstrated through the fully operational search pipeline that spans the Vue storefront, .NET backend, Python sidecar, and PostgreSQL with pgvector. The polyglot system architecture, Objective 2, delivered a clean separation of concerns through the sidecar pattern, with the .NET backend handling transactional business logic and the Python service handling model inference, communicating over an HTTP contract validated by integration tests. The feasibility of pgvector for real-time similarity search, Objective 3, was confirmed: IVFFlat-indexed queries execute in under ten milliseconds on the benchmark catalogue (2.7–6.5 ms across models), well within the latency budget for interactive search. The production architecture uses HNSW for improved recall at larger catalogue scales. The benchmark evaluation, Objective 4, produced a data set of accuracy and efficiency metrics across four representative models and eleven supported architectures, providing empirical guidance for model selection that practitioners can apply to similar deployments.

5.2 CONTRIBUTIONS

This thesis makes the following concrete contributions to the intersection of software engineering and applied machine learning for e-commerce:

- **A four-model benchmark for fashion image retrieval.** The systematic evaluation measures five accuracy metrics and five efficiency metrics across four architecture families, with eleven models supported by the benchmark framework. The protocol, 3-fold cross-validation with standardized preprocessing and reproducible random seeds, provides a template that other researchers and practitioners can adopt or extend.
- **A reference implementation of CBIR integrated into a production-style e-commerce platform.** The ReSys.Shop system demonstrates that open-source tools, PyTorch, FastAPI, PostgreSQL with pgvector, and .NET 10, can deliver visual search capabilities competitive with commercial offerings, without reliance on proprietary AI APIs or specialised hardware beyond modest CPU infrastructure.
- **A pluggable model architecture that enables runtime model switching.** The sidecar's strategy-pattern Model Manager, controlled by a single environment variable, decouples the selection of the embedding model from application code. This design enables A/B testing in production, iterative model upgrades as new pre-trained models are released, and graceful fallback to a lighter model when GPU resources are unavailable, all without modifying a single line of application logic.
- **Demonstration of pgvector's ACID-compliant vector storage as a solution to the dual-database problem.** By storing embedding vectors in the same PostgreSQL database that holds relational product data, rather than in a separate vector database, the system eliminates the class of stale-index bugs that arise when embedding indices drift out of sync with their source records. Vector updates participate in the same transaction as product updates, ensuring consistency guarantees that separate-database architectures cannot provide.
- **A validated polyglot architecture pattern for .NET plus Python AI.** The sidecar integration, FastAPI service communicating with a .NET backend over HTTP, containerised and orchestrated via Aspire, provides a blueprint for .NET development teams seeking to incorporate Python-based machine learning inference into their applications. The pattern balances the strengths of each ecosystem: .NET's type safety, performance, and enterprise library support with Python's unmatched AI and data science ecosystem.

5.3 LIMITATIONS

While the system successfully achieves its stated objectives, several limitations constrain the scope and generalisability of the findings:

- **Dataset representativeness.** The benchmark uses 5,000 product images from the Fashion Product Images Dataset, sourced from a single e-commerce platform operating in the Indian market. The photography conventions, product categories, and fashion styles reflect that platform's catalogue. Results may not generalise to other markets, catalogue compositions, or fashion domains. Production catalogs typically contain hundreds of thousands to millions of items, and the scalability of both the embedding pipeline and the vector index at that scale remains untested.
- **Hardware specificity.** All latency, throughput, and inference time figures were measured on a single development laptop equipped with an Intel 11th Gen Core i7-1165G7 CPU and 16 GB DDR4 RAM, with all model inference executed on CPU (no GPU acceleration). This represents a standard consumer laptop configuration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially, and the relative ranking of models may shift across hardware platforms. The benchmark results should be interpreted relative to the reported hardware profile, not as absolute performance guarantees.
- **Category-level relevance criterion.** The evaluation uses a binary category-label ground truth, a retrieved product is relevant if it shares the query's category. This is a coarse proxy for visual similarity. Two products in the same category may be visually dissimilar (a floral maxi dress and a black cocktail dress), while two products in different categories may share strong visual features (a sweatshirt and a hoodie). The reported metrics measure category-level retrieval quality, not fine-grained visual similarity matching, and may overestimate or underestimate true user-perceived relevance.
- **No user experience evaluation.** Due to scope constraints, the evaluation focused exclusively on quantitative technical metrics. Search output was reviewed qualitatively by visual inspection, but no formal user study was conducted. The relationship between measured accuracy improvements, a 4.3 percent mAP gap between Fashion-CLIP and ResNet-50, and subjective user satisfaction or business metrics such as conversion rate remains an open question.
- **Pre-trained models only.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning on the target catalogue might further improve retrieval quality, particularly for models pre-trained on generic image corpora. This

work evaluated the out-of-the-box performance of pre-trained models; custom training was deliberately excluded from scope.

- **Image-only modality.** The evaluation is confined to image-to-image retrieval. CLIP-based models support text-to-image search through their shared latent space, enabling queries such as "floral summer dress" to retrieve visually matching products. This multimodal capability was not evaluated. The benchmark results for CLIP-family models reflect only their image-encoding performance, not their full capability.
- **K-value selection and labelling scheme.** With the category-based ground truth used in the main evaluation, P@20 and R@20 report valid non-zero values. However, the enriched-label evaluation in Appendix A.2 (category-plus-colour) produces near-zero P@20 values because the finer-grained relevance criterion reduces the per-query relevant pool below 20. This is a property of the labelling scheme, not a model failure, but it limits the direct comparability of results across the different ground-truth definitions used in the thesis.
- **Memory measurement limitations.** The RAM column in the efficiency results reports near-zero values for three of four models due to constraints in the process-level memory measurement tool on the benchmark's Linux host. Actual memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models) for model weights alone, with additional overhead from the PyTorch runtime. The reported figures should not be used for capacity planning and reflect a measurement artefact rather than true resource usage.

5.4 FUTURE WORK

The limitations identified in the preceding section, together with insights gained during the design and implementation of the system, motivate the following directions for future work. The items are prioritised from most actionable to most ambitious.

1. Fine-tune Fashion-CLIP on the target catalogue. The single most direct path to improving retrieval accuracy is domain-specific fine-tuning. Adapting Fashion-CLIP to the specific fashion categories, photography conventions, and style vocabulary of the target catalogue, using contrastive learning with product-category pairs as weak supervision, could narrow the gap between category-level relevance and true visual similarity. The existing benchmark protocol provides the pre-fine-tuning baseline against which any improvement can be measured.

2. Conduct a user experience study with A/B testing. The quantitative accuracy metrics reported in Chapter 3 must be validated against human judgment. A controlled A/B test, assigning half of storefront visitors to text-

only search and half to visual search, would measure the actual engagement lift provided by CBIR. Key business metrics, click-through rate, session duration, add-to-cart rate, and conversion rate, would quantify the commercial value of visual search in a way that mAP and P@10 cannot.

3. Implement multi-modal search combining text and image queries. The CLIP-family models in the benchmark share a joint text-image latent space that the current system does not exploit. Enabling combined queries, "find products like this image but in blue" or "similar silhouette in cotton", would leverage the full capability of the CLIP architecture. This requires extending the search endpoint to accept an optional text prompt alongside the query image and using CLIP's text encoder to refine the similarity computation.

4. Scale the benchmark to production-size catalogues. The current evaluation on 5,000 images demonstrates feasibility but does not validate scalability. Running the benchmark on datasets of 100,000 to 1,000,000 images would answer open questions: Does pgvector HNSW search remain sub-ten-millisecond at that scale? Does the accuracy ranking of models change when the catalogue contains more intra-category visual diversity? Do some models degrade gracefully while others collapse? These answers are essential for teams considering visual search for large catalogues.

5. Investigate ONNX Runtime optimisation. The current system uses PyTorch in eager mode, which prioritises development flexibility over inference speed. Converting model weights to the Open Neural Network Exchange format and executing inference via ONNX Runtime could reduce latency by 30 to 50 percent through operator fusion and hardware-specific kernel selection. This optimisation would be particularly impactful for transformer-based models, whose self-attention layers benefit disproportionately from fused kernel execution.

6. Add personalised re-ranking to search results. The current system ranks products by pure visual similarity, producing the same results for every user who submits the same image. Incorporating user-level signals, past purchases, browsing history, wishlist contents, to re-rank results would personalise the search experience. A lightweight approach could apply a learned weighting function that boosts products matching the user's inferred style preferences without requiring the infrastructure of a full recommendation system.

7. Develop a mobile application with on-device inference. The excluded scope of this thesis included a mobile frontend. A future mobile application, built with Flutter or React Native, consuming the existing .NET APIs, could use the device camera for a "snap-and-search" experience. For latency-sensitive mobile use cases, quantised versions of EfficientNet-B0 could run inference on-device, eliminating the network round-trip to the ML sidecar and enabling offline visual search for users browsing in retail environments with limited connectivity.

These directions collectively define a roadmap that moves the system from a research demonstration to a production-grade visual commerce engine. Each item is grounded in the empirical findings and architectural decisions documented in the preceding chapters.

5.5 REQUIREMENTS TRACEABILITY

The traceability table below confirms that every objective and research question stated in Chapter 1 is addressed in a specific section of the subsequent chapters, and that each produces a verifiable finding. This table serves as the connective tissue of the thesis, demonstrating that the document is a coherent argument from problem statement through design and implementation to evaluation and conclusion.

Table 61: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	A functional visual search pipeline was delivered, spanning Vue storefront, .NET backend, Python ML sidecar, and PostgreSQL pgvector. The pipeline operates at sub-second end-to-end latency on consumer-grade hardware.
Architect a polyglot system bridging .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	The sidecar pattern successfully isolates ML inference from transactional logic. The .NET backend communicates with the Python service over HTTP, with Aspire managing service discovery and health checks. Integration tests validate the cross-service contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.2.4, Section 2.3.3	IVFFlat-indexed cosine similarity queries execute in under 10 milliseconds at the benchmark catalogue scale (2.7–6.5 ms). Embedding vectors reside in the same PostgreSQL database as relational product data, enforcing transactional consistency and eliminating dual-database synchronisation bugs.
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.2–3.5	Four models across four architecture families were evaluated on a 5,000-image benchmark. Fashion-CLIP delivers the highest accuracy (mAP 0.8788); EfficientNet-B0 delivers the best efficiency (23.9 ms per inference). Deployment recommendations are provided for different operational contexts.

Objective / RQ	Addressed In	Key Finding
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.3; Chapter 3, Section 3.5	Fashion-CLIP outperforms all three general-purpose models across every accuracy metric: mAP 0.8788 vs 0.8120–0.8341. Domain-specific fine-tuning provides a consistent 5.4–8.2 percent mAP improvement. Fashion-CLIP also exhibits the lowest cross-fold variability (±0.0022).
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.3–3.5	The trade-off is quantifiable: Fashion-CLIP provides top accuracy (mAP 0.8788) at 92.0 ms; EfficientNet-B0 achieves 92.8 percent of that accuracy at 26.0 percent of the latency (23.9 ms). Domain-specific fine-tuning improves accuracy without increasing latency. ResNet-50 is the least competitive choice on both dimensions.
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.5 (synthesis)	The polyglot architecture is viable. The sidecar handles model inference at 23.9–92.9 ms per image depending on model, while the .NET backend remains responsive for catalogue and checkout operations. End-to-end search latency remains under one second. Independent scaling and fault isolation are achieved without the operational overhead of a full microservices deployment.
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer internal architecture (interface, Model Manager, PyTorch Runtime). Lazy-loading reduces cold-start memory pressure. Exposes POST /embeddings and GET /health endpoints, containerised via Docker and orchestrated by Aspire.
Set up vector search	Chapter 2, Section 2.2.4, Section 2.3.3	pgvector configured with cosine similarity on the embedding column. IVF-Flat queries execute in under 10 milliseconds. Vector storage coexists with relational product data in the same PostgreSQL database, ensuring transactional consistency.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	The .NET CBIR handler orchestrates the full pipeline: client-side validation, server-side magic-byte verification, cross-service HTTP to the ML sidecar with API-key authentication, pgvector similarity query with model-name filtering, and result deduplication with similarity-score conversion.
Create the user interface	Chapter 2, Sections 2.3.3 (flow)	Vue 3 storefront with drag-and-drop image upload, similarity score badges,

Objective / RQ	Addressed In	Key Finding
	and 2.3.5 (supporting modules)	and product-card result display. Client-side file-type and size validation reduces server load. Results are rendered as a grid with thumbnail images and navigable product links.
Evaluate the results	Chapter 3, Sections 3.2–3.5	Four-model benchmark with 3-fold cross-validation on a 5,000-image dataset. Seven accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (latency, throughput, load time, storage, RAM) across both original and enriched labelling schemes. Deployment recommendations provided.

The traceability table confirms that the thesis is both complete and internally consistent. Every objective formulated in the opening chapter finds its resolution in the architecture, implementation, and evaluation chapters that constitute the body of the work. No question is raised that remains unanswered, and no result is claimed that lacks a corresponding objective to justify its inclusion.

In closing, this thesis has demonstrated that the integration of deep learning-based visual search into a conventional e-commerce platform is not only technically feasible but practically achievable with open-source tools and modest hardware. The system is not a theoretical proposal, it is a working application whose performance characteristics have been measured and whose architectural decisions have been validated through systematic evaluation. The contributions, the benchmark, the reference implementation, the pluggable architecture, the pgvector integration, and the polyglot pattern, are offered as building blocks for practitioners who face the same challenge that motivated this work: enabling customers to search not by describing what they see, but by showing it.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[14] Z. Liu, P. Luo, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[15] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 6. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), and the full hardware specifications of the benchmark environment (C).

A APPENDIX A, FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 6, Section 6.2. Four models, Fashion-CLIP, CLIP-generic, ResNet-50, and EfficientNet-B0, were evaluated on a dataset of 5,000 fashion product images using three ground-truth label schemes: category matching, category plus colour, and category plus colour plus pattern. Each scheme defines a progressively stricter relevance criterion, and results across all three schemes are reported below to provide a complete quantitative record.

A.1 A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 62: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue, each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary

evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 63: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
EfficientNet-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth, requiring both category and colour agreement, better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 64: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
EfficientNet-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 65: Operational Efficiency, All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
EfficientNet-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
FashionCLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-

system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 66: Per-Fold Breakdown, Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B APPENDIX B, DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 6.

B.1 B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category,

subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 67: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/

Striped"). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C APPENDIX C, HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 6 and Appendix A were collected on a single workstation. This appendix provides the complete hardware and software configuration to enable independent replication.

C.1 C.1 COMPUTE HARDWARE

Table 68: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU acceleration). The Intel i7-1165G7 processor provides sufficient compute throughput for the evaluated models at interactive latencies: EfficientNet-B0 completes inference in 21.6 ms on average, while the transformer-based CLIP models complete in 84–106 ms. PyTorch executed in CPU mode with all model weights held in system memory. All inference timing measurements include preprocessing (resize, normalise) and postprocessing in the reported per-image latency.

C.2 C.2 SOFTWARE STACK

Table 69: Benchmark Software Environment

Component	Version and Details
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0 with CUDA 13.0 backend
TorchVision	0.28.0
Transformers (HuggingFace)	5.14.1
OpenCLIP	3.3.0
Fashion-CLIP	0.2.x
NumPy	2.5.1
CUDA Toolkit	13.0
cuDNN	8.9.x
NVIDIA Driver	580.173.02 (Linux)

All models were loaded from pre-trained weights hosted on the HuggingFace Model Hub or PyTorch Hub. Model weights were cached locally in `~/.cache/huggingface/` and `~/.cache/torch/` after the first download. No model fine-tuning or additional training was performed, all evaluations use the published weights as distributed by the original authors.

C.3 C.3 PRECISION AND DETERMINISM SETTINGS

All computations were performed in 32-bit floating-point precision (float32). Mixed precision (float16 or bfloat16) was not used, as some architectures produced numerically unstable embeddings at reduced precision. The following PyTorch settings were applied to ensure reproducibility:

- Random seed: 42 (Python `random`, NumPy, PyTorch CPU, PyTorch GPU)
- `torch.backends.cudnn.deterministic = true`
- `torch.backends.cudnn.benchmark = false`
- Model evaluation mode: `model.eval()` with `torch.no_grad()`

Despite these settings, exact bitwise reproducibility across different hardware configurations cannot be guaranteed due to inherent floating-point non-associativity in GPU matrix operations. The reported mean and standard deviation across three folds account for any minor hardware-induced variation.

C.4 C.4 REPRODUCIBILITY NOTES

Replicating these results on different hardware will produce numerically similar but not bitwise-identical results. The following factors contribute to cross-platform variation:

- **CPU architecture:** Inference latency is primarily determined by single-core performance and cache size, as the benchmark was executed on CPU without GPU acceleration. A processor with higher IPC (instructions per clock) or AVX-512 support would reduce inference times, particularly for the matrix operations in transformer models. The **relative ranking** of models (EfficientNet-B0 fastest, CLIP-based models slowest) should remain consistent across CPU architectures, though absolute values will vary.
- **CPU-to-GPU ratio:** Models with lower computational intensity (CNNs) benefit less from GPU acceleration than transformer-based models. On CPU-only systems, the latency gap between EfficientNet-B0 and FashionCLIP will narrow relative to GPU-accelerated deployments.
- **Memory capacity:** The 16 GB system memory of the development workstation exceeded the memory requirements of all individual models. Systems with less than 16 GB of system memory may experience swapping during concurrent model loading, particularly for the larger CLIP-based models which require over 600 MB each for model weights plus PyTorch runtime overhead.
- **Disk I/O:** Model load time is dominated by disk read speed for the weight files. An NVMe SSD (as used here) provides faster load times than a SATA SSD or HDD.

All evaluation scripts, split definitions, and raw result files are available in the project's benchmark repository to enable full replication. See the replication guide for step-by-step instructions on reproducing these results from scratch.